

List of Files

```
using Printf  
include("src/printmat.jl");
```

```
files = filter(x → endswith(lowercase(x), ".ipynb"), readdir()) #all .ipynb files  
filter!(x → x != "Ch00_ListOfFiles.ipynb", files)
```

```
printmat(files)
```

```
Ch14_FX.ipynb  
Ch15_Forwards.ipynb  
Ch16_Bonds1.ipynb  
Ch17_Bonds2.ipynb  
Ch18_Bonds3.ipynb  
Ch19_Options1.ipynb  
Ch20_Options2.ipynb  
Ch21_Options3.ipynb  
Ch22_Options4.ipynb
```

List of Files

```
using Printf
include("src/printmat.jl");
```

```
files = filter(x → endswith(lowercase(x),".ipynb"), readdir()) #all .ipynb files
filter!(x → x != "Tutorial_00_ListOfFiles.ipynb",files)
```

```
printmat(files)
```

```
Tutorial_01_FastTrack_Basics.ipynb
Tutorial_02_FastTrack_Stats.ipynb
Tutorial_03_FastTrack_Finance.ipynb
Tutorial_04_DataContainers.ipynb
Tutorial_05_Functions.ipynb
Tutorial_06_Plots.ipynb
Tutorial_07a_Arrays.ipynb
Tutorial_07b_MatrixAlgebraX.ipynb
Tutorial_07c_ArrayFunctions.ipynb
Tutorial_08_Types.ipynb
Tutorial_09_DaysAndDates.ipynb
Tutorial_10_LoadSaveData.ipynb
Tutorial_11_Strings.ipynb
Tutorial_12_Printing.ipynb
Tutorial_13_MissingValues.ipynb
Tutorial_14_Downloading.ipynb
Tutorial_21_FindingRoots.ipynb
Tutorial_22a_Optimization_QuadraticX.ipynb
Tutorial_22b_Optimization_GeneralX.ipynb
Tutorial_24_Derivatives.ipynb
Tutorial_25_Interpolation.ipynb
Tutorial_26_PythonCall_C-R.ipynb
Tutorial_27_Threads.ipynb
```

Tutorial_28a_Module.ipynb
Tutorial_28b_Project.ipynb
Tutorial_28c_Package.ipynb
Tutorial_31_PythonPlot.ipynb
Tutorial_33_DataFrames.ipynb
Tutorial_99_TrickyStuff.ipynb

Basic Julia Tutorial

(with a focus on coding in finance and econometrics).

This notebook gives instructions for installing Julia and summarizes the basics of the language in a fast-paced “get started” approach. Details are discussed in later notebooks.

Running/Installing Julia

This is a notebook. To run it you need Julia and Jupyter/VS Code.

1. To run Julia without a local installation, use, for instance, [CoCalc](#), [JuliaHub](#) or [Colab](#). CoCalc charges a subscription fee, JuliaHub charges per hour (with some free hours/month), while Colab is free. However, Colab requires reinstallation of (most) packages in each session and deletes local files as the session is closed. In contrast, CoCalc and JuliaHub have persistent storage (packages and local files are kept).
2. To install Julia on your machine, [download and install it](#).
3. You may also want to use VS Code with the Julia extension: (a) [download and install VS Code](#); (b) open VS Code and install the julia extension. This gives an IDE (editor and more) that can run script files and notebooks. It also works well with coding agents. Start by opening the right folder and then open/create the file. See [VS Code with the Julia extension](#) for further details.
4. As an alternative to VS code, you may consider Jupyter/JupyterLab: (a) [install the Julia package IJulia](#), but notice that the instructions differ depending on whether you have Python installed or not; (b) start Julia and run using `IJulia;jupyterlab(dir=pwd())`. Change the directory (from `pwd()`) as needed. For the traditional jupyter, replace `jupyterlab` by `notebook`.
5. To create and run script files instead of notebooks, see the comments towards the end of this notebook.

Documentation and Help

1. Cheat sheet at [QuantEcon](#)
2. [Wiki book](#)
3. [Short tutorials](#)
4. The [official Julia on-line manual](#)
5. Discussion lists are found at
 - <https://discourse.julialang.org/>
 - <https://stackoverflow.com/questions/tagged/julia-lang>
 - <https://www.reddit.com/r/Julia/>
6. In Julia, do `?cos` or `@doc cos` to get help with the `cos` function (see below for how to get help in VS Code).
7. Claude and ChatGPT often generate decent/good Julia code, which can serve as a starting point.

About Notebooks

This cell is a “Markdown” cell. This is meant for comments and documentation, not computations.

You can change a cell to “Code” or “Markdown” in the menu.

Markdown cells can handle LaTeX. An example: $\alpha = \beta/2$. A Markdown cell can also contain some *formatting*, like lists of this kind:

1. To insert a new cell, use the menu.
2. The next cell is “Code”. You can run it. Text after a `#` sign is treated as a comment.

```
a = 2          #this is a comment
               #run this cell by using the menu, or by Shift+Enter
```

2

Load Packages and Extra Functions

There are many packages for Julia, for instance, for plotting or statistical methods (see [julialang](#) for a list of lists). To install a package, you do either

1. (works everywhere) run `import Pkg` and then `Pkg.add("Packagename")`

2. (works in the Julia console, the REPL) enter the “package manager mode” by typing `]`, then run `add PackageName`. You leave the package manager mode by backspace.

Once a package is installed, you can use it by running `using PackageName`.

```
using Printf #installed by default, used for more control over printing
include("src/printmat.jl"); #just a function for prettier matrix printing, semicolon (;) to make
```

How to Get Help

Typing `?cos` or `@doc cos` prints the documentation for the `cos` function, except in a notebook in VS Code. We therefore use a work-around (`@doc2`, defined in `printmat.jl`) in the next cell.

```
@doc2(cos) #Needed in VS Code, works everywhere
# ? cos #works in REPL and jupyter, not in VS Code
```

```
cos(x::T) where {T <: Number} → float(T)
```

Compute cosine of `x`, where `x` is in radians.

Throw a `DomainError` if `isinf(x)`, return a `T(NaN)` if `isnan(x)`.

See also `cosd`, `cospi`, `sincos`, `cis`.

```
cos(A::AbstractMatrix)
```

Compute the matrix cosine of a square matrix `A`.

If `A` is symmetric or Hermitian, its eigendecomposition (`eigen`) is used to compute the cosine. Otherwise, the cosine is determined by calling `exp`.

Examples

```
julia> cos(fill(1.0, (2,2)))
2×2 Matrix{Float64}:
 0.291927 -0.708073
-0.708073  0.291927
```

Scalars and Matrices

Create a Scalar and a Matrix

```
q = 1                                #create a scalar
Q = [ 1 2 3;                         #create 2x3 matrix
      4 5 6 ]
println("q is a scalar. To print, use println() or printlnPs()")
println(q)

println("\nQ is a matrix. To print, use display() or printmat()")
printmat(Q)                          #case sensitive (q and Q are different)
                                     #the \n adds a line break
```

q is a scalar. To print, use println() or printlnPs()

1

Q is a matrix. To print, use display() or printmat()

1	2	3
4	5	6

Picking Out Parts of a Matrix

```
println("\n", "element [1,2] of Q: ",      #commands continue on
        Q[1,2])                             #the next line (until finished)

println("\ncolumns 2 and 3 of Q: ")
printmat(Q[:,2:3])

println("\nline 1 of Q (as a vector): ")
printmat(Q[1,:])
```

element [1,2] of Q: 2

columns 2 and 3 of Q:

2	3
5	6

line 1 of Q (as a vector):

1
2
3

Basic Linear Algebra

The syntax for linear algebra is similar to the standard text book approach. For instance, $Q'Q$ (or $Q' * Q$) multiplies the transpose (Q') with the matrix (Q) $A*B$ does matrix multiplication $100*Q$ multiplies each element of the matrix (Q) by 100. (You can also do $100Q$)

However, to add a scalar to each element of a matrix, use $100 \ .+ Q$. Notice the dot.

```
println("transpose of Q:")
printmat(Q')

println("Q'Q:")
printmat(Q'Q)

println("scalar * matrix:")
printmat(100*Q)

println("scalar .+ matrix:")      #notice the dot
printmat(100 .+ Q)
```

transpose of Q:

1	4
2	5
3	6

Q'Q:

17	22	27
22	29	36
27	36	45

scalar * matrix:

100	200	300
400	500	600

scalar .+ matrix:

101	102	103
104	105	106

Creating a Sequence and a Vector

```
θ = 1:10:21          #a range, type \theta[TAB] to get this symbol
println("\n", "θ is a sequence: ", θ)

ρ = collect(θ)        #make the sequence into a vector, \rho[TAB]
println("\n", "ρ is a vector: ")
printmat(ρ)
```

θ is a sequence: 1:10:21

ρ is a vector:

1
11
21

Comparing Things

To see if the scalar $z \leq 0$, do

```
vv = z <= 0
```

to get a single output (true or false).

Instead, if x is an array, do

```
vv = x .<= 0          #notice the dot.
```

to get an array of outputs (with the same dimensions as x)

```

x = [-1.5,-1.0,-0.5,0,0.5]           #this is a vector

vv = -1 .< x .<= 0                   #true for x values (-1,0], vv is a vector
println(x,vv;colNames=["x","is in (-1,0]"],width=15)

x2 = x[vv]                           #x values for which vv==true
println("x values that are in (-1,0]: ")
println(x2)

```

x	is in (-1,0]
-1.500	0
-1.000	0
-0.500	1
0.000	1
0.500	0

```

x values that are in (-1,0]:
-0.500
0.000

```

Finding Things (extra)*

can be done by indexing as shown before. However, it might be more efficient to use one of `findfirst()`, `findall()` and `indexin()`.

Also, if you just need to check if the number `z` is in an array (or any collection) `x`, then use `z in x` or `in(z,x)`.

```

x = [-1.5,-1.0,-0.5,0,0.5]           #this is a vector

println("x values: ")
println(x)

v1 = findfirst(x.==0)
println("(first) index v in x such x[v]==0: ",v1)

v2 = findall(x.>=0)
println("\nall indices v in x such x[v]>=0: ")
println(v2)

```

```

y = [-1.0,0]
v3 = indexin(y,x)
println("\nindices in x so that x[v] equals the vector y=$y: ")
printmat(v3)

v4 = 0 in x          #or in(0,x)
println("\ntesting if 0 is in x: ",v4)

```

```

x values:
-1.500
-1.000
-0.500
0.000
0.500

```

```
(first) index v in x such x[v]==0: 4
```

```

all indices v in x such x[v]>=0:
4
5

```

```

indices in x so that x[v] equals the vector y=[-1.0, 0.0]:
2
4

```

```
testing if 0 is in x: true
```

Simple Functions

The next cell defines two new functions, `fn0()` and `fn1()`. They take a scalar input `x` and return a scalar output `y`.

If you instead use a vector as the input, then the computations in these functions fail. (The reason is that you cannot do x^2 on a vector. You would need $x.^2$.)

However, using the “dot” syntax

```
y = fn1.(x)
```

gives an array as output where element $y[i,j] = \text{fn1}(x[i,j])$.

```
fn0(x) = (x-1.1)^2 - 0.5      #define a one-line function

function fn1(x)              #define a function that could have many lines
    y = (x-1.1)^2 - 0.5
    return y
end
```

fn1 (generic function with 1 method)

```
y = fn1(1.5)
printlnPs("result from fn1(1.5): ",y," and compare with fn0(1.5) ",fn0(1.5))

x = [1;1.5]
#y = fn1(x)                  #would give an error
y = fn1.(x)                  #calling on the function, dot. to do for each element in x
printlnPs("\nresult from fn1.(x): ")
printmat(y)
```

result from fn1(1.5): -0.340 and compare with fn0(1.5) -0.340

result from fn1.(x):
 -0.490
 -0.340

if-else-end

allows you to run different commands depending on a condition that you specify.

(extra*) There are also other (more compact) possibilities for the if-else case

1. $y = z \leq 2 ? z : 2$ or
2. $y = \text{ifelse}(z \leq 2, z, 2)$

The second cell demonstrates how to also use elseif.

```

z = 1.05

if z <= 2          #(a) if true, run the next command (y=z) and then jump to end
    y = z
else              #(b) if (a) is false, do this instead
    y = 2
end

#y = z <= 2 ? z : 2      #these two versions also work
#y = ifelse(z <= 2,z,2)

println(y)

```

1.05

```

if z < 1          #(a) if true, run the next command (y=1) and then jump to end
    y = 1
elseif 1 <= z <= 2  #(b) if (a) is false, try this instead
    y = z
else              #(c) if also (b) is false, do this
    y = 2
end

println(y)

```

1.05

Loops

There are two types of loops: “for loops” and “while loops”.

The *for loop* is best when you know how many times you want to loop, for instance, over all m rows in a matrix.

The *while loop* is best when you want to keep looping until something happens, for instance, that x_0 and x_1 get really close.

The default behaviour in *Julia* (notebooks), *inside functions* and also at the *REPL* prompt is that assignments of x inside the loop overwrite x defined before the loop. To get the same behavior in scripts, you

need to add global `x` somewhere inside the loop. In contrast, a variable (here `z2`) that does not exist before the loop is local to the loop.

To make sure that the `y` calculated inside the loop *does not* affect `y` outside the loop, say local `y` inside the loop.

A Simple “for loop”

The “for loop” in the next cell iterates over all elements in a vector and changes them.

```
x = zeros(3)
for i in 1:length(x)      #1,2,3
    x[i] = 100 + i
end

printmat(x)
```

```
101.000
102.000
103.000
```

A More Complicated “for loop”

The “for loop” in the next cell makes 3 iterations and changes a global `x` variable. It also illustrates difference between local and global variables.

```
x = 0
y = -999
println("\nBefore loop: x=$x and y=$y \n")    #"$x" inserts the value of x into a string#

for i in 3:3:9                                #or `i = 3:3:9`, or `v=[3,6,9]; i in v`
    #global x                                  #only needed in script
    local y                                    #don't overwrite y outside loop
    x = x + i                                  #adding i to the "old" x
    y = i
    z2 = -998                                  #notice: z2 has not been used before
    println("i=$i, x=$x, y=$y and z2=$z2")    #x prints the value of x
end

println("\nAfter loop: x=$x and y=$y \n")
```

```

try                                #to "drive around" errors
    println(z2)                    #does not work: z2 is local to the loop
catch
    println("z2 is not defined outside the loop")
end

```

Before loop: x=0 and y=-999

i=3, x=3, y=3 and z2=-998
i=6, x=9, y=6 and z2=-998
i=9, x=18, y=9 and z2=-998

After loop: x=18 and y=-999

z2 is not defined outside the loop

A Double “for loop”

The next cell uses a double loop to fill a matrix.

```

(m,n) = (4,3)                     #same as m=4;n=3
x = fill(-999,m,n)                #to put results in, initialized as -999
for i in 1:m, j in 1:n            #can be written on the same line
    x[i,j] = 10*i + j
end

println("new x matrix: \n")
printmat(x)

```

new x matrix:

11	12	13
21	22	23
31	32	33
41	42	43

A Simple “while loop”

There are also `continue` to skip the rest of the current iteration and move on the next one, as well as `break` to break out of the loop.

```
x = 5

while x > 0.05
    #global x           #only needed in script
    x = x/2
    println("x=$x")
end

println("\nAfter loop: x=$x")
```

```
x=2.5
x=1.25
x=0.625
x=0.3125
x=0.15625
x=0.078125
x=0.0390625
```

After loop: x=0.0390625

A First Plot

With the [Plots.jl](#) package you create a a simple plot like this:

1. Plot two curve by using the `plot([x1 x2],[y1 y2])` command
2. Configure curves by `linecolor =`, `linestyle =` etc
3. Add titles and labels by `title =`, `xlabel =`, etc

Notice: the *first plot is slow*.

```
using Plots           #this loads the Plots package
default(size = (480,320),fmt = :png) # :svg gives prettier plots
```



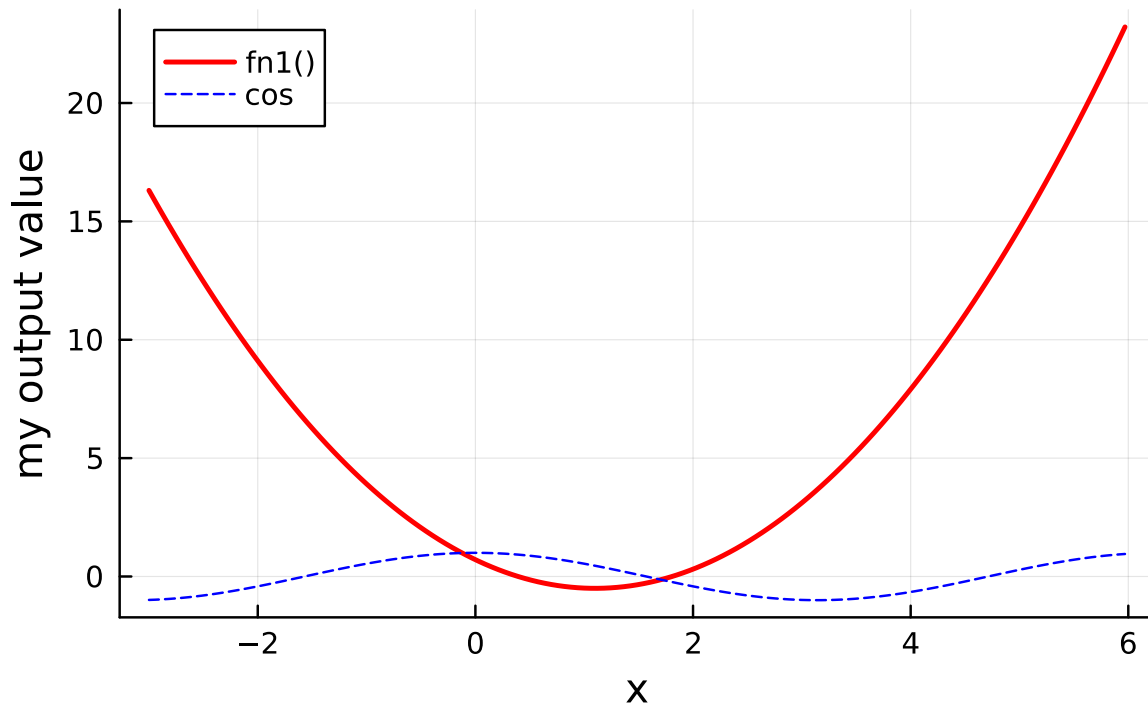
```

x = -3:6/99:6

p1 = plot( [x x],[fn1.(x) cos.(x)],
           linecolor = [:red :blue],           #"row vectors" of values
           linestyle = [:solid :dash],
           linewidth = [2 1],
           label = ["fn1()" "cos"],
           title = "My Results",
           xlabel = "x",
           ylabel = "my output value" )
display(p1)           #not needed in notebook, but useful in script

```

My Results



Types: Integers, Floats, Booleans and Others

Julia has many different types of variables: signed integers (like 2 or -5), floating point numbers (2.0 and -5.1), booleans (false/true), bitarrays (similar to booleans, but with more efficient use of memory), strings ("hello"), Dates (2017-04-23) and many more types.

Integers and Floats

```
a = 2           #integer, Int (Int64 on most machines)
b = 2.0         #floating point, (Float64 on most machines)
A = [1,2]
B = [1.0,2.0]

println("Finding the type of a, b, A and B:")
println(typeof(a)," ",typeof(b)," ",typeof(A)," ",typeof(B))
```

Finding the type of a, b, A and B:
Int64 Float64 Vector{Int64} Vector{Float64}

Bools and BitArrays

Bools are “true” or “false”. BitArrays are (more memory efficient) versions of this.

```
c = 2 > 1.1
C = A .> 1.5      #A is an array, so C is too

println("Finding the type of c and C:")
println(typeof(c)," ",typeof(C))
```

Finding the type of c and C:
Bool BitVector

Calculations with Mixed Types

A calculation like “integer” + “float” works and the type of the result will be a float (the more flexible type). Similarly, “bool” + “integer” will give an integer. These promotion rules make it easy to have mixed types in calculations, and also provide a simple way of converting a variable from one type to another. (There are also an explicit `convert()` function that might be more efficient.)

```
println(1+2.0)      #integer + Float
println((1>0) + 2)  #bool + integer
```

3.0
3

Running Script Files

instead of notebooks.

Once you have created a script file (called “myscript.jl” below), then you can, for instance, run it from VS Code or directly from the Julia REPL. For the latter, do as follows:

1. Open the Julia REPL and perhaps run `cd("folder_name")` to set the working directory (typically to where your script file is).
2. Run `include("myscript.jl")`

As an alternative, instead do the following:

0. Add this close to the top of the script file: `cd(dirname(@__FILE__))`
1. Open the Julia REPL
2. Run `include("foldername/myscript.jl")`

Performance Tips (extra)*

To benchmark your code, install/use the [BenchmarkTools.jl](#) package.

```
using BenchmarkTools

function fn2(x)
    n = length(x)
    z = zero(x)          #array of zeros, same size as x
    for i in 2:n
        z[i] = (x[i] + x[i-1])/2  #moving average
    end
    return z
end

x = [1;zeros(101)]
@btime fn2($x)          #notice the $
println()
```

54.049 ns (2 allocations: 928 bytes)

Basic Statistic

This notebook loads some data, reports simple descriptive statistics (means, standard deviations etc) and shows a number of useful plots (scatter plots, histograms, time series plots).

The approach is fast-paced, and details are left for later notebooks.

Most of the descriptive stats use the standard package `Statistics`. The plots rely on the [Plots.jl](#) package and the pdf and quantiles are from the [Distributions.jl](#) package.

For more stat functions, see the [StatsBase.jl](#) package (not used here).

Statistical calculations are often reported in tables. This notebook uses my own `printmat()` function. For more powerful alternatives, consider the [PrettyTables.jl](#) package.

Load Packages and Extra Functions

```
using Statistics, Printf, Dates, LinearAlgebra, DelimitedFiles, Distributions
include("src/printmat.jl")
```

@doc2

```
using Plots
default(size = (480,320),fmt = :png)    #change to :svg for sharper plots
```

Load Data from a csv File

The next cell loads data (excess returns) from a csv. See the header line for definitions. To keep it simple, the cell creates the variables in a rather crude way (picking out columns).

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,R) = (Date.(x[:,1]),Float64.(x[:,2:4])) #make sure dN is a Date, and data is Float
printlnPs("R: ",size(R)) #Re: excess returns on two sectors and the equity market

RNames = ["HiTec","Utils","Market"]; #names of the (columns) in the 'R' matrix, see header line
printmat(RNames)
(T,n) = size(R) #no. of periods and variables
```

```

      R:  (660, 3)
      HiTec
      Utils
      Market
```

```
(660, 3)
```

Descriptive Statistics

Means and Standard Deviations

The next few cells estimate means, standard deviations, quantiles, covariances and correlation.

```
μ = mean(R,dims=1) #,dims=1 to calculate average along a column
σ = std(R,dims=1) #do \sigma[Tab] to get σ

printmat([μ;σ];colNames=RNames,rowNames=["mean","std"])
```

	HiTec	Utils	Market
mean	0.700	0.547	0.608
std	6.538	4.151	4.599

Quantiles

The quantile() function has no dims argument, so we need to loop over the columns. This can be done in several ways, and the cell uses a very straightforward approach.

```

qM = fill(NaN,2,n)
for i in 1:n
    qM[:,i] = quantile(R[:,i],[0.05,0.95])
end

printmat(qM;colNames=RNAMES,rowNames=["0.05","0.95"])

```

	HiTec	Utils	Market
0.05	-9.891	-6.333	-7.684
0.95	11.533	6.750	7.191

Correlations

```

printblue("\n","correlation matrix:")
printmat(cor(R);colNames=RNAMES,rowNames=RNAMES)

```

```

correlation matrix:
      HiTec  Utils  Market
HiTec  1.000  0.333  0.870
Utils  0.333  1.000  0.576
Market 0.870  0.576  1.000

```

Regressions

The function defined in the next cell calculates the OLS estimates, with standard errors based on iid (Gauss-Markov) assumptions.

The second cell applies the function.

```

"""
    OLS(y,x)

Does OLS of the regression y = x*b + u

"""
function OLS(y,x)
    b = x\y                                #OLS

```

```

    u    = y - x*b                #residuals
    covb = inv(x'x)*var(u)        #cov(b), see any textbook
    stdb = sqrt.(diag(covb))     #std(b)
    R2    = 1 - var(u)/var(y)
    return b, stdb, R2
end

```

OLS

```

(T,n) = size(R)
x = [ones(T) R[:,3]]           #x is a Tx2 matrix, constant and 'market'
K = size(x,2)                  #no. of regressors, here 2

(b,tstat) = (fill(NaN,K,n),fill(NaN,K,n)) #to fill with results
for i in 1:n
    local stdb
    (b[:,i],stdb,) = OLS(R[:,i],x)
    tstat[:,i] = b[:,i]/stdb
end

printblue("OLS coefficients, regressing R on a constant and Rme:")
printmat(b;colNames=RNAMES,rowNames=["c","Rme"],width=15)

printblue("t-stats, regressing R on a constant and Rme:")
printmat(tstat[:,1:2];colNames=RNAMES[1:2],rowNames=["c","Rme"],width=15)

printred("The t-stats for the third regression are uninteresting and extreme")

```

OLS coefficients, regressing R on a constant and Rme:

	HiTec	Utils	Market
c	-0.052	0.231	-0.000
Rme	1.237	0.520	1.000

t-stats, regressing R on a constant and Rme:

	HiTec	Utils
c	-0.412	1.735
Rme	45.301	18.109

The t-stats for the third regression are uninteresting and extreme

Distributions

Probability Density Function (pdf)

The cells below calculate and plot pdfs and cdfs of some distributions. The `Distributions.jl` package has many more distributions.

A Remark on the Code

- Notice that the `Distributions.jl` package wants `Normal( $\mu$ , $\sigma$ )`, where σ is the standard deviation.
- `pdf.(Normal(0,1),x)` calculates the pdf of a standard normal variable at each value in the array `x`. Notice the dot `(.)`.

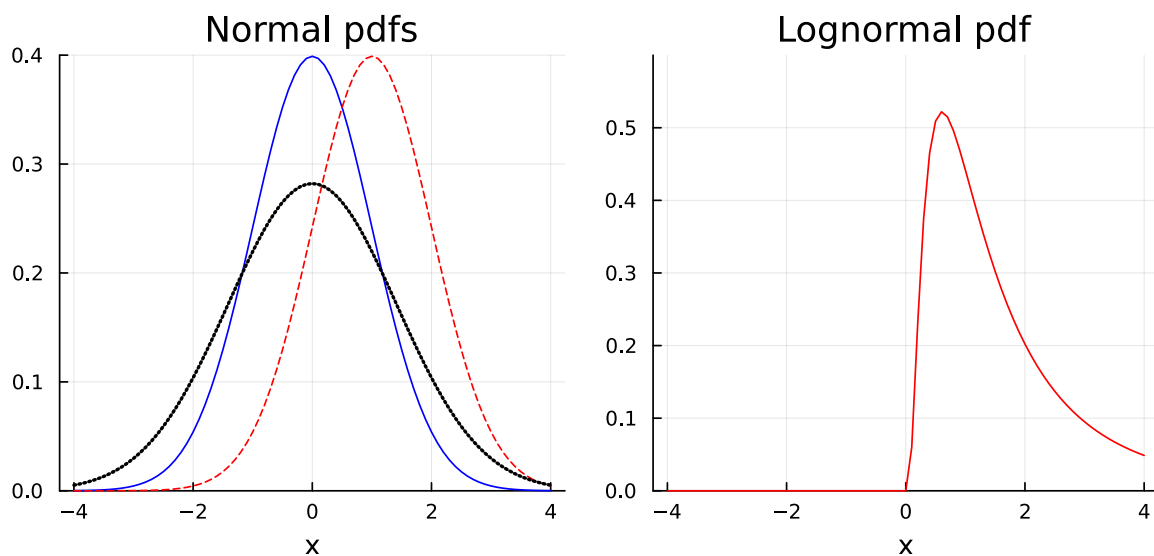
```
x = -4:0.1:4
pdf1a = pdf.(Normal(0,1),x)           #Normal
pdf1b = pdf.(Normal(1,1),x)
pdf1c = pdf.(Normal(0,sqrt(2)),x)

pdf2 = pdf.(LogNormal(0.25,sqrt(0.75)),x)   #LogNormal

p1 = plot( x,hcat(pdf1a,pdf1b,pdf1c),
            linecolor = [:blue :red :black],
            linestyle = [:solid :dash :dot],
            linewidth = [1 1 2],
            legend = nothing,
            ylims = (0,0.4),
            title = "Normal pdfs",
            xlabel = "x")

p2 = plot( x,pdf2,
            linecolor = :red,
            legend = nothing,
            ylims = (0,0.6),
            title = "Lognormal pdf",
            xlabel = "x")

pAll = plot( p1,p2,                      #combine the subplots
             layout = @layout[a a],
             size = (700,320) )
display(pAll)
```

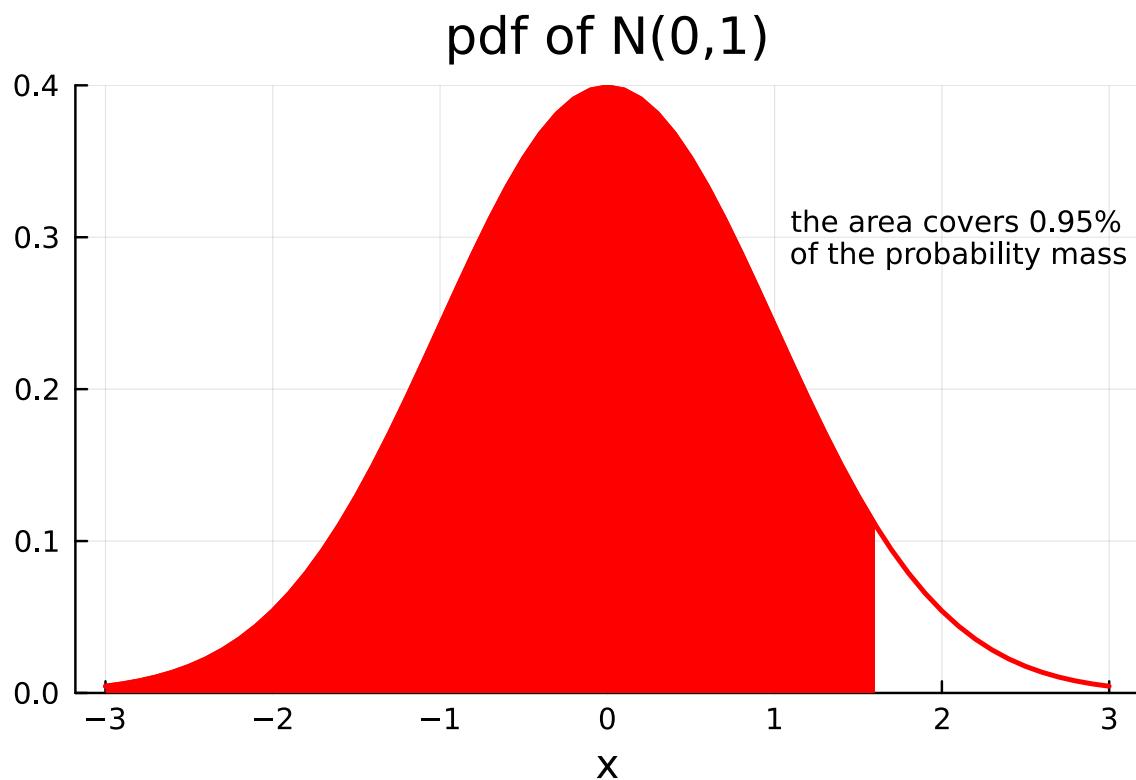



```
x = -3:0.1:3
xb = x[x.<=1.645]                                #pick out x values <= 1.645

pdfx = pdf.(Normal(0,1),x)                       #calculate the pdf of a N(0,1) variable
pdfxb = pdf.(Normal(0,1),xb)                     #colour the area below this curve
Prb = cdf(Normal(0,1),1.645)
printlnPs("Pr(x<=1.645) ",Prb)

p1 = plot( x,pdfx,                                #plot pdf
           linecolor = :red,
           linewidth = 2,
           legend = nothing,
           ylims = (0,0.4),
           title = "pdf of N(0,1)",
           xlabel = "x",
           annotation = (1.1,0.3,text("the area covers $(round(Prb,digits=2))%\nof the probability
plot!(xb,pdfxb,linecolor=:red,linewidth=2,legend=nothing,fill=(0,:red)) #plot area under pdf
display(p1)
```

Pr($x \leq 1.645$) 0.950



Quantiles (“critical values”) of Distributions

```
N95      = quantile(Normal(0,1),0.95)           #from the Distributions package
Chisq90 = quantile(Chisq(2),0.9)

printblue("\n","95th percentile of  $N(0,1)$  and 90th of  $\chi^2(2)$ :")
printmat([N95 Chisq90])
```

95th percentile of $N(0,1)$ and 90th of $\chi^2(2)$:
1.645 4.605

Drawing Random Numbers

Random Numbers: Independent Variables

```
T = 100
x = randn(T,2)    #T x 2 matrix, N(0,1) distribution

printblue("\n", "mean and std of random draws: ")
μ = mean(x,dims=1)
σ = std(x,dims=1)

colNames = ["x1", "x2"]
printmat([μ;σ];colNames,rowNames=["mean","std"])

printblue("covariance matrix:")
printmat(cov(x);colNames,rowNames=colNames)
```

mean and std of random draws:

	x ₁	x ₂
mean	-0.091	0.026
std	0.911	0.949

covariance matrix:

	x ₁	x ₂
x ₁	0.830	-0.041
x ₂	-0.041	0.900

Random Numbers: Correlated Variables

```
μ = [-1,10]      #vector of means
Σ = [1 0.5;      #covariance matrix
     0.5 2]

T = 100
x = rand(MvNormal(μ,Σ),T)' #drawn from bivariate N(μ,Σ), transpose to get T x 2
colNames = ["x1", "x2"]
```

```
printblue("covariance matrix:")
printmat(cov(x);colNames,rowNames=colNames)
```

covariance matrix:

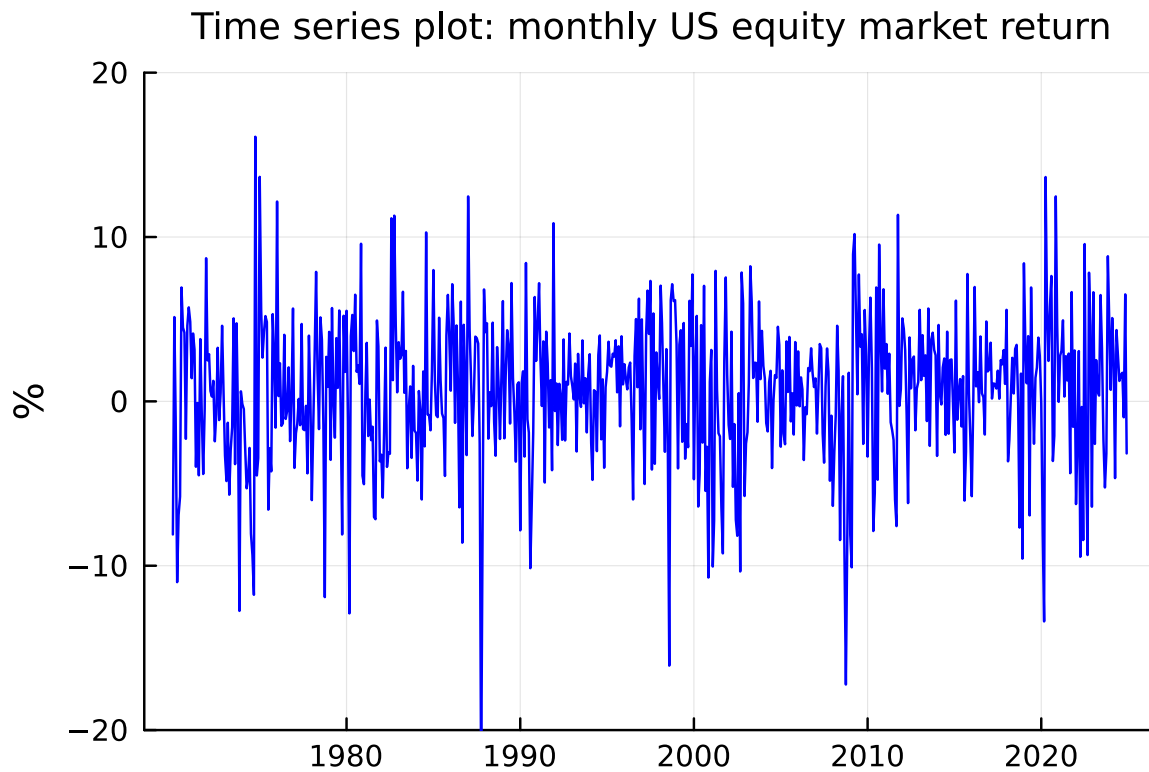
	x_1	x_2
x_1	1.222	0.656
x_2	0.656	2.004

Statistical Plots

The next few cells demonstrates some useful statistical plots.

```
xTicksLoc = Date(1980):Year(10):Date(2020)    #a sequence, to get good x-axis labels
xTicksLab = Dates.format.(xTicksLoc,"Y")

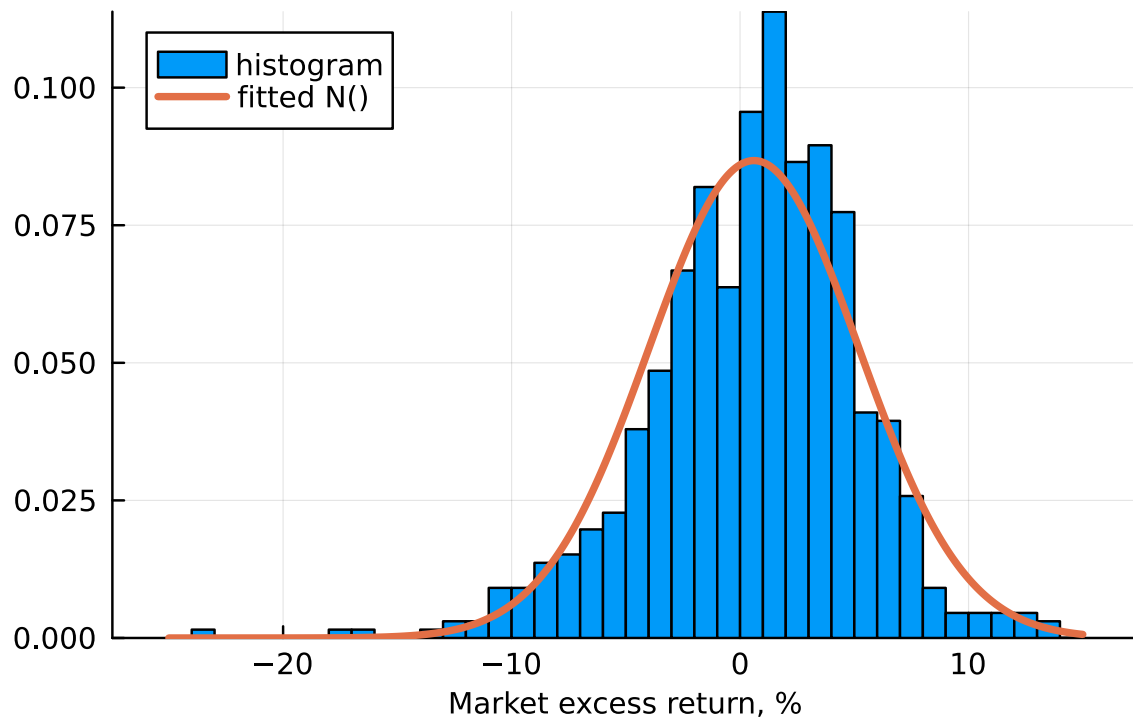
p1 = plot( dN,R[:,3],                                #time series plot
           linecolor = :blue,
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
           ylim = (-20,20),
           title = "Time series plot: monthly US equity market return",
           titlefontsize = 10,
           ylabel = "%" )
display(p1)
```



```
xGrid = -25:0.1:15
pdfX = pdf.(Normal(mean(R[:,3]),std(R[:,3])),xGrid) #"Distributions" wants  $\sigma$ , not  $\sigma^2$ 

p1 = histogram( R[:,3],bins = -25:1:15,      #histogram
               normalized = true,          #normalized to have area=1
               label = "histogram",
               title = "Histogram: monthly US equity market excess return",
               titlefontsize = 10,
               xlabel = "Market excess return, %",
               legend = :topleft,
               guidefontsize = 8 )          #font size of axis labels
plot!(xGrid,pdfX,linewidth=3,label="fitted N()")
display(p1)
```

Histogram: monthly US equity market excess return



```
p1 = scatter( R[:,3],R[:,1],                                #scatter
              fillcolor = :blue,
              label = "scatter",
              legend = true,
              xlim = (-20,20),
              ylim = (-20,20),
              title = "Scatter plot: $(RNames[1]) vs market ",
              titlefontsize = 10,
              xlabel = "Market return, %",
              ylabel = "Returns on $(RNames[1]) sector, %",
              guidefontsize = 8 )
Plots.abline!(1,0,color=:black,linewidth=0.5,label="45 degree")      #45-degree line
Plots.abline!(b[2,1],b[1,1],linecolor=:magenta,line=:dash,label="regression line")  #b is from OLS

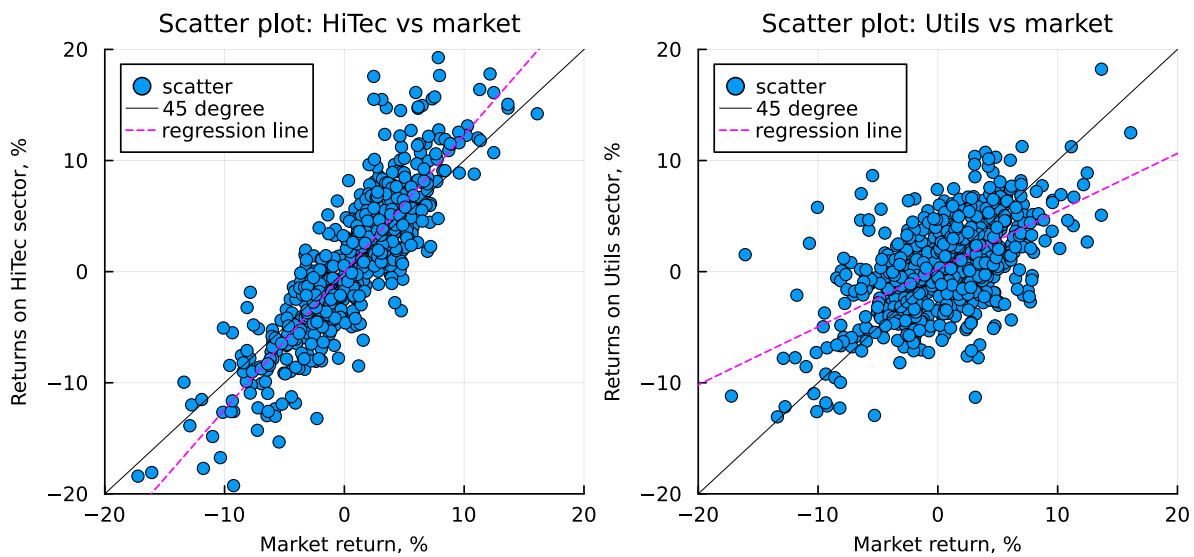
p2 = scatter( R[:,3],R[:,2],
              fillcolor = :blue,
              label = "scatter",
              legend = true,
              xlim = (-20,20),
```

```

ylim = (-20,20),
title = "Scatter plot: $(RNames[2]) vs market",
titlefontsize = 10,
xlabel = "Market return, %",
ylabel = "Returns on $(RNames[2]) sector, %",
guidelfontsize = 8 )
Plots.abline!(1,0,color=:black,linewidth=0.5,label="45 degree")
Plots.abline!(b[2,2],b[1,2],linecolor=:magenta,line=:dash,label="regression line")

pAll = plot( p1,p2,                                     #combine the subplots
             layout = @layout[a a],
             size = (700,320) )
display(pAll)

```



Typical Financial Calculations

This notebook showcases some classical finance calculations, which requires regressions, root solving, optimisation, and more. The approach is fast-paced, and details are left to later notebooks.

The notebook (a) estimates CAPM equations and autocorrelations; (b) implements a simple trading strategy; (c) calculates Value at Risk using a simple model for time-varying volatility; (d) calculates the Black-Scholes option price and implied volatility; (e) calculates and draws the mean-variance frontier.

Load Packages and Extra Functions

The [Roots.jl](#) package solves a non-linear equation, [Distributions.jl](#) defines distribution functions for a large number of distributions, and [StatsBase.jl](#) has methods for estimating autocorrelations etc. The other packages are Standard Libraries (they come with the Julia installer).

```
using Printf, Dates, DelimitedFiles, LinearAlgebra, Roots, Distributions, StatsBase
include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data

The returns loaded below are in excess of the riskfree rate.

```
x = readlm("Data/TwoIndustries.csv",'',skipstart=1)

(dN,R) = (Date.(x[:,1]),Float64.(x[:,2:4])) #make sure dN is a Date, and data is Float
printlnPs("R: ",size(R)) #Re: excess returns on two sectors and the equity market
T = size(R,1)
```



```
RNames = ["HiTec","Utils","Market"];      #names of the (columns) in the 'R' matrix, see header line
(Re,Rm) = (R[:,2],R[:,3]);                  #pick out Utils and the Market, separate vectors
```

R: (660, 3)

CAPM

The CAPM regression is

$$R_t^e = \alpha + \beta R_{mt}^e + \varepsilon_t,$$

where R_t^e is the excess return of test asset and R_{mt}^e is the market excess return. Theory says that $\alpha = 0$, which is easily tested.

A Remark on the Code

- $b[1]$ corresponds to α and $b[2]$ to β .
- There are clearly packages that can do OLS, but we do not use them here.

```
"""
    OLS(y,x)

Does OLS of the regression y = x*b + u

"""
function OLS(y,x)
    b    = x\y                #OLS, intercept and slope
    u    = y - x*b            #residuals
    covb = inv(x'x)*var(u)    #cov(b), see any textbook
    stdb = sqrt(diag(covb))   #std(b)
    R2   = 1 - var(u)/var(y)
    return b,stdb,R2
end
```

OLS

```
(b, stdb, R2) = OLS(Re, [ones(T) Rm]) #regressing Re on Rm

printblue("Regression results:\n")
printmat([b stdb b./stdb], colNames=["coeff", "std", "t-stat"], rowNames=["α", "β"])
printlnPs("R2: ", R2)
printlnPs("no. of observations: ", T)
```

Regression results:

	coeff	std	t-stat
α	0.231	0.133	1.735
β	0.520	0.029	18.109

R2:	0.332
no. of observations:	660

Return Autocorrelation

That is, the correlation of R_t and R_{t-s} .

It can be shown that (under strong assumptions) the t-stat of an autocorrelation is $\sqrt{T}$ times the autocorrelation.

A Remark on the Code

- `autocor(Re, plags)` from the `StatsBase` package estimates the autocorrelations for the lags defines in the rang (or vector) `plags`.

```
plags = 1:5
ρ = autocor(Rm, plags) #autocorr of the market

printblue("Autocorrelations of the market:")
printmat([ρ sqrt(T)*ρ], colNames=["autocorr", "t-stat"], rowNames=string.(plags), cell00="lag")
```

Autocorrelations of the market:

lag	autocorr	t-stat
1	0.040	1.038
2	-0.049	-1.253
3	0.032	0.826

4	0.004	0.112
5	0.045	1.166

A Trading Strategy

The next cell implements a very simple momentum trading strategy.

1. If $R_{t-1} \geq 0$, then we hold the market index and shorten the riskfree from $t - 1$ to t . This means that we will earn R_t (assuming R_t is an excess return).
2. Instead, if $R_{t-1} < 0$, then we do the opposite. This means that we will earn $-R_t$.

This simple strategy could be coded without using a loop, but “vectorization” does not speed up much.

We compare with a “passive” portfolio, which is just holding the asset all the time (and earning R_t^e in each period).

```
(w,Rp) = (fill(NaN,T),fill(NaN,T))
for t in 2:T
    w[t] = ifelse(Rm[t-1] < 0,-1,1)      #w is -1 or 1
    Rp[t] = w[t]*Rm[t]
end

R_all = hcat(Rp[2:end],Rm[2:end])        #Tx2, matrix of both returns
μ      = mean(R_all,dims=1)
σ      = std(R_all,dims=1)

printblue("The annualized mean excess return and Sharpe ratios of the strategy and a passive portfolio")
xx = [μ*12;sqrt(12)*μ./σ]
printmat(xx;rowNames=["mean","SR"],colNames=["strategy","passive"])

printred("A pretty bad strategy...")
```

The annualized mean excess return and Sharpe ratios of the strategy and a passive portfolio:

	strategy	passive
mean	4.288	7.455
SR	0.268	0.469

A pretty bad strategy...

Value at Risk

The next cell constructs an simple estimate of σ_t^2 as a backward looking moving average:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda)(R_{t-1}^e - \mu_{t-1})^2, \text{ where } \mu_t = \lambda \mu_{t-1} + (1 - \lambda)R_{t-1}^e$$

Then, we calculate the 95% VaR by assuming a $N(\mu, \sigma_t^2)$ distribution:

$$\text{VaR}_t = -(\mu_t - 1.64\sigma_t).$$

```
function EWMA(x,λ,μ₀,σ₀)
    T = length(x)
    (μ,σ²) = (fill(μ₀,T),fill(σ₀^2,T))    #vectors, time-varying mean and variance
    for t in 2:T
        μ[t] = λ*μ[t-1] + (1-λ)*x[t-1]
        σ²[t] = λ*σ²[t-1] + (1-λ)*(x[t-1]-μ[t-1])^2    #RiskMetrics approach
    end
    return μ, σ²
end
```

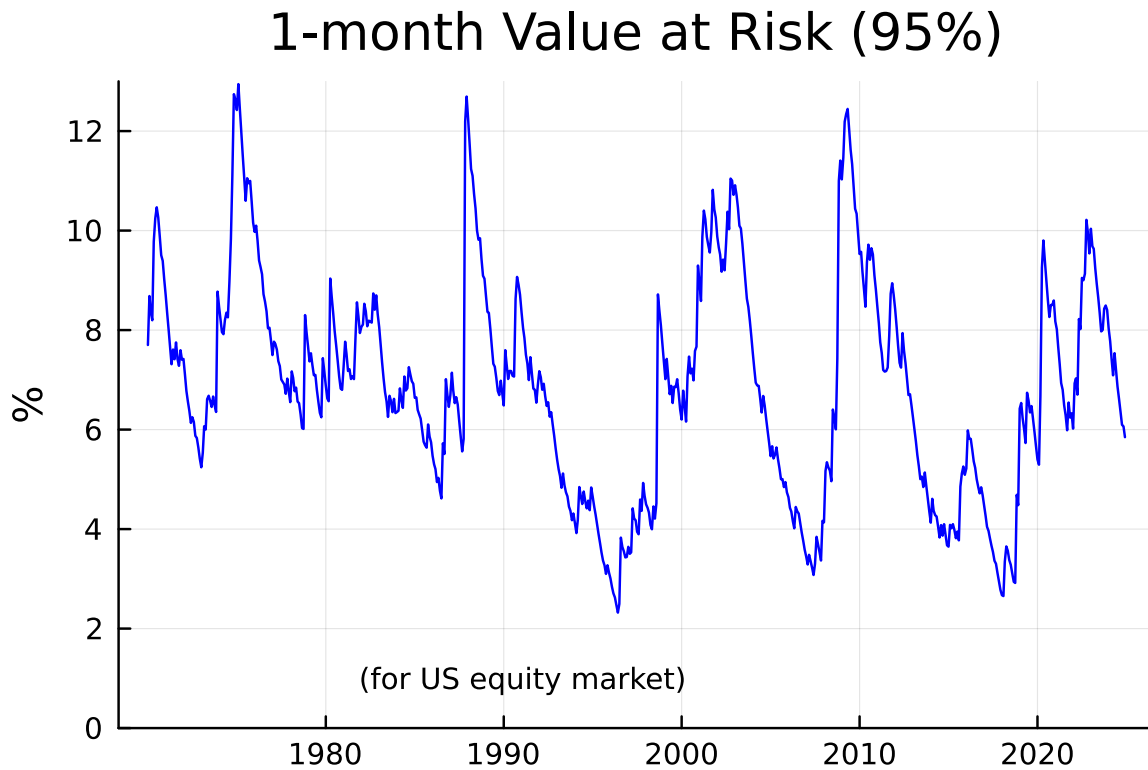
EWMA (generic function with 1 method)

```
(λ,μ₀,σ₀) = (0.94,0.5,5.0)    #λ and some starting values for obs t=0

(μ,σ²) = EWMA(Rm,λ,μ₀,σ₀)
VaR95 = -(μ .- 1.64*sqrt.(σ²));    #VaR at 95% level, assuming R is N()
```

```
xTicksLoc = Date(1980):Year(10):Date(2020)
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN,VaR95,
    color = :blue,
    legend = false,
    xticks = (xTicksLoc,xTicksLab),
    ylim = (0,13),
    title = "1-month Value at Risk (95%)",
    ylabel = "%",
    annotation = (Date(1982),1,text("(for US equity market)",8,:left)) )
display(p1)
```



Options

Black-Scholes Option Price

Let S be the current spot price of an asset and y be the interest rate.

The Black-Scholes formula for a European call option with strike price K and time to expiration m is

$C = S\Phi(d_1) - e^{-ym}K\Phi(d_2)$, where

$$d_1 = \frac{\ln(S/K) + (y + \sigma^2/2)m}{\sigma\sqrt{m}} \quad \text{and} \quad d_2 = d_1 - \sigma\sqrt{m}$$

and where $\Phi(d)$ denotes the probability of $x \leq d$ when x has an $N(0, 1)$ distribution. All variables except the volatility (σ) are directly observable.

A Remark on the Code

- `cdf(Normal(0,1),x)` defines the cdf of a $N(0, 1)$ distribution, using the `Distributions.jl` package.

```

"""
Pr(z<=x) for N(0,1)
"""
Φ(x) = cdf(Normal(0,1),x)      #a one-line function, to get standard notation

"""
Calculate Black-Scholes european call option price
"""
function OptionBlackSPs(S,K,m,y,σ)

    d1 = ( log(S/K) + (y+1/2*σ^2)*m ) / (σ*sqrt(m))
    d2 = d1 - σ*sqrt(m)
    c   = S*Φ(d1) - exp(-y*m)*K*Φ(d2)
    return c
end

```

OptionBlackSPs

```

(S,K,m,y,σ) = (10,10,0.5,0.1,0.4)

c1 = OptionBlackSPs(S,K,m,y,σ)      #call price for a single K value
printlnPs("\n","call price according to Black-Scholes for a single K value: ",c1)

K_range = range(7,13,length=51)      #call prices for several K values
c = OptionBlackSPs.(S,K_range,m,y,σ);

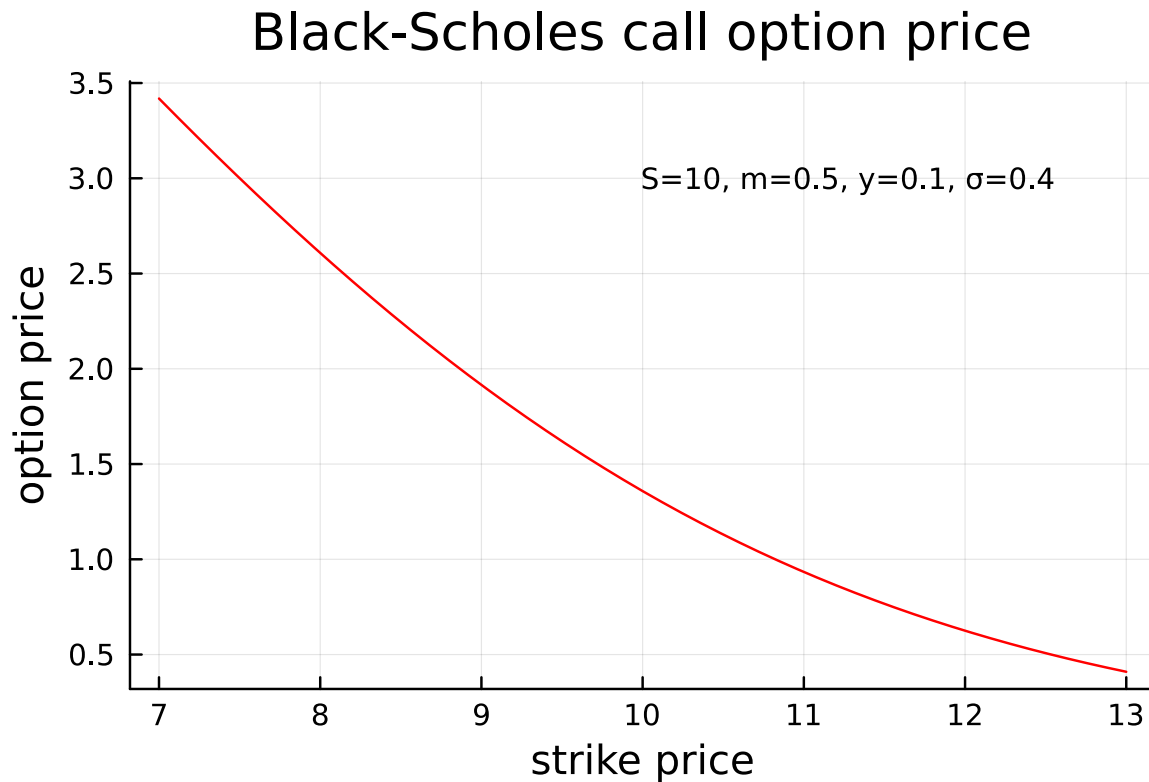
```

call price according to Black-Scholes for a single K value: 1.358

```

p1 = plot( K_range,c,
            color = :red,
            legend = false,
            title = "Black-Scholes call option price",
            xlabel = "strike price",
            ylabel = "option price",
            annotation = (10,3,text("S=$S, m=$m, y=$y, σ=$σ",8,:left)) )
display(p1)

```



Implied Volatility

...is the σ value that makes the Black-Scholes equation give the same option price as observed on the market. It is often interpreted as the “market uncertainty.”

The next cell uses the call option price calculated above as the market price. The implied volatility should then equal the volatility used above (this is a way to check your coding).

The subsequent cells instead use some data on options on German government bonds and calculate the σ for each of them.

A Remark on the Code

- the `find_zero()` function from the `Roots.jl` package solves a non-linear function for its root (where the function is zero). The function should only have one (effective) argument.
- In the cell below, we define `σKFn(σ,K)` to make the code more readable and then solve for the root of `σKFn(σ,K)-c1`.

```

                                #solve for implied vol
σKFn(σ,K) = OptionBlackSPs(S,K,m,y,σ)    #function of σ and K only

iv = find_zero(σ → σKFn(σ,K)-c1,(0.00001,5))    #here: should be the same as used for ca

printlnPs("Implied volatility: ",iv,", which should equal $σ")

```

Implied volatility: 0.400, which should equal 0.4

Solve for iv From Some Data on German Government Bonds

...overwriting the previous values for S,K,m,y,σ

```

# LIFFE Bunds option data, trade date April 6, 1994
K = [                                #strike prices
      92.00; 94.00; 94.50; 95.00; 95.50; 96.00; 96.50; 97.00;
      97.50; 98.00; 98.50; 99.00; 99.50; 100.0; 100.5; 101.0;
      101.5; 102.0; 102.5; 103.0; 103.5 ];
C = [                                #call prices
      5.13; 3.25; 2.83; 2.40; 2.00; 1.64; 1.31; 1.02;
      0.770; 0.570; 0.400; 0.280; 0.190; 0.130; 0.0800; 0.0500;
      0.0400; 0.0300; 0.0200; 0.0100; 0.0100 ];
S = 97.05                            #spot price
m = 48/365                           #time to expiry in years
y = 0.0                               #Interest rate: LIFFE⇒no discounting
N = length(K)

```

21

```

iv = fill(NaN,N)    #looping over strikes
for i in 1:N
    iv[i] = find_zero(σ → σKFn(σ,K[i])-C[i],(0.00001,5))
end

#printblue("From Bunds options data:\n")
#printmat(K,iv,colNames=["strike","σ"])

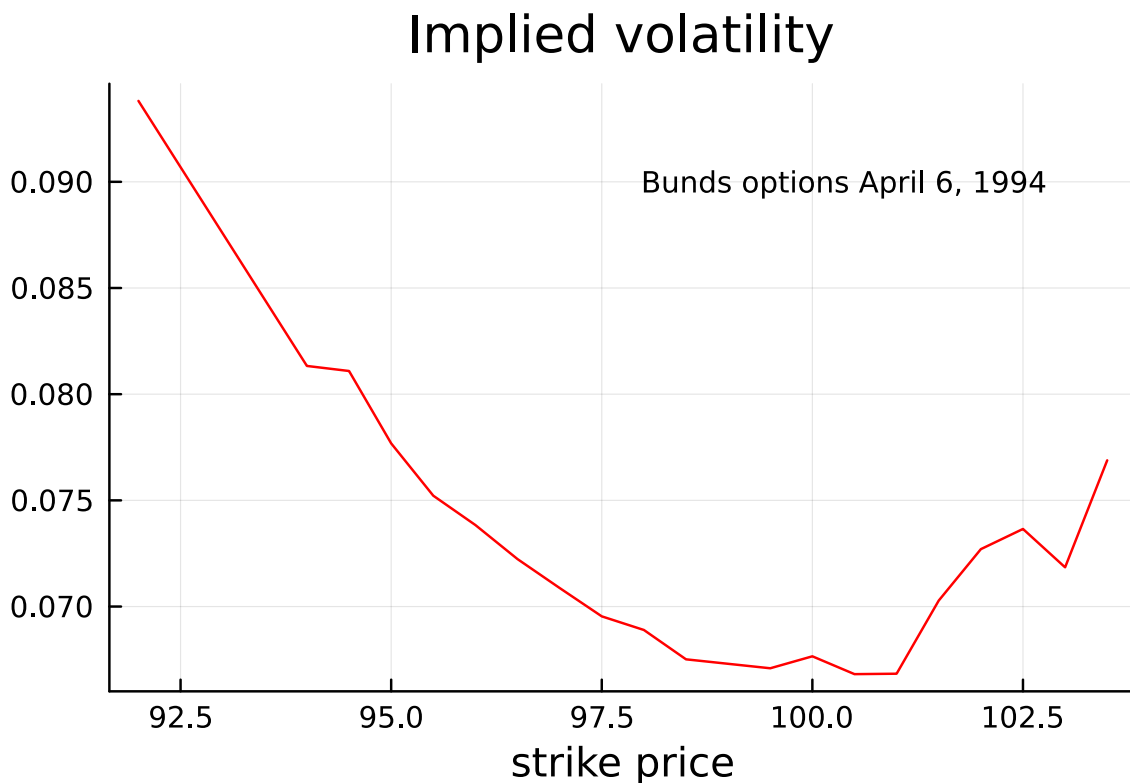
```



```

p1 = plot( K,iv,
           color = :red,
           legend = false,
           title = "Implied volatility",
           xlabel = "strike price",
           annotation = (98,0.09,text("Bunds options April 6, 1994",8,:left)) )
display(p1)

```



Mean-Variance Frontier

Given a vector of average returns (μ) and a variance-covariance matrix (Σ), the mean-variance frontier shows the lowest possible portfolio uncertainty for a given expected portfolio return (denoted μ^* below).

It is thus the solution to a quadratic minimization problem. The cells below will use matrix formulas for this solution, but we often have to resort to numerical methods (especially when there are portfolio restrictions).

The mean-variance frontier is typically plotted with the portfolio standard deviation (*not* the variance) on the horizontal axis and the portfolio expected return μ^* on the vertical axis.

We calculate and plot two different mean-variance frontiers: (1) when we only consider risky assets; (2) when we also consider a risk-free asset.

A Remark on the Code

The calculations for the mean-variance frontiers are in two functions (for the case of only risky assets, and for risky and a riskfree asset). The calculations involve solving a system of linear equations (or calculating a matrix inverse) for each value of the expected return μ^* . To speed up the calculations we use the `factorize()` function from the `LinearAlgebra` standard library.

```
μ = [11.5, 9.5, 6]/100          #expected returns
Σ = [166 34 58;                #covariance matrix
     34 64 4;
     58 4 100]/100^2
Rf = 0.03                      #riskfree return (an interest rate)

println("μ: ")
printmat(μ)
println("Σ: ")
printmat(Σ)
println("Rf: ")
printmat(Rf)
```

μ:

```
0.115
0.095
0.060
```

Σ:

```
0.017    0.003    0.006
0.003    0.006    0.000
0.006    0.000    0.010
```

Rf:

```
0.030
```

```

"""
    MVCalc( $\mu^x, \mu, \Sigma$ )

Calculate the std and weights of a portfolio (with mean return  $\mu^x$ ) on MVF of risky assets.

# Input
- ' $\mu^x::\text{Vector}$ ':    K vector, mean returns to calculate results for
- ' $\mu::\text{Vector}$ ':    n vector, mean returns
- ' $\Sigma::\text{Matrix}$ ':  nxn, covariance matrix of returns, can contain riskfree assets

# Output
- ' $\text{StdRp}::\text{Vector}$ ':  K vector, standard deviation of mean-variance portfolio (risky only) with mean  $\mu^x$ 
- ' $w_p::\text{Matrix}$ ':    Kxn, portfolio weights of      ""

"""
function MVCalc( $\mu^x, \mu, \Sigma$ )
    (K,n) = (length( $\mu^x$ ),length( $\mu$ ))
    A = [ $\Sigma$        $\mu$  ones(n);   #A is just a name of this matrix, it's not asset A
           $\mu'$       0 0;
          ones(n)' 0 0];
    Af = factorize(A)           #factorize so we can solve  $w\lambda\delta_i$  repeatedly (and fast)
    (w,StdRp) = (fill(NaN,K,n),fill(NaN,K))
    for i in 1:K                #loop over  $\mu^x[i]$ 
         $w\lambda\delta_i$  = Af\[zeros(n); $\mu^x[i]$ ;1]
         $w_i$  =  $w\lambda\delta_i[1:n]$ 
        StdRp[i] = sqrt( $w_i'\Sigma w_i$ )
         $w[i,:]$  =  $w_i$ 
    end
    return StdRp, w
end

"""
Calculate the std of a portfolio (with mean  $\mu^x$ ) on MVF of (Risky,Riskfree)
"""
function MVCalcRf( $\mu^x, \mu, \Sigma, Rf$ )
    (K,n) = (length( $\mu^x$ ),length( $\mu$ ))
     $\mu^e = \mu .- Rf$ 
    A = [ $\Sigma$        $\mu^e$ ;
           $\mu^e'$  0]
    Af = factorize(A)
    (w,StdRp) = (fill(NaN,K,n),fill(NaN,K))

```

```

    for i in 1:K
        wλ_i      = Af\[zeros(n);(μ*[i]-Rf)]
        w_i       = wλ_i[1:n]
        StdRp[i]  = sqrt(w_i'*Σ*w_i)
        w[i,:]    = w_i
    end
    return StdRp, w
end

```

MVCalcRf

```

μ* = range(Rf,0.15,length=201)      #a range of different expected portfolio returns

StdRp,   = MVCalc(μ*,μ,Σ)            #risky assets only
StdRpRf, = MVCalcRf(μ*,μ,Σ,Rf);      #with riskfree too

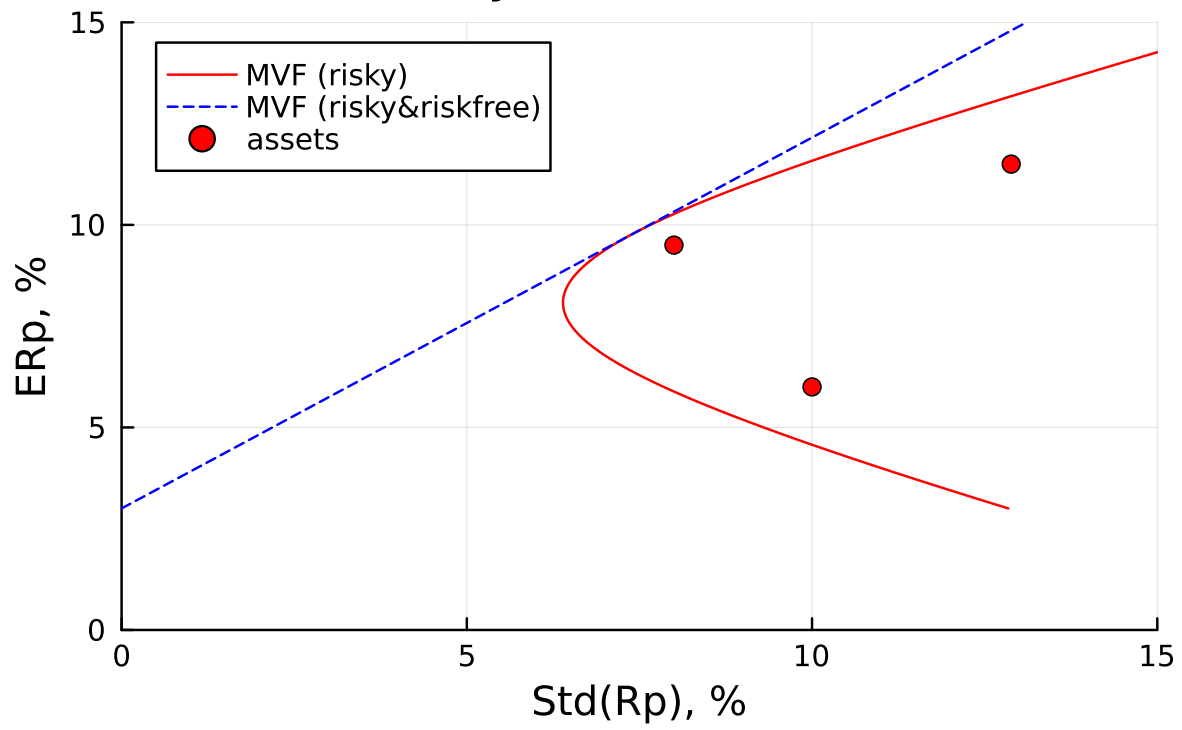
```

```

p1 = plot( [StdRp StdRpRf]*100,[μ* μ*]*100,
    label = ["MVf (risky)" "MVf (risky&riskfree)"],
    legend = true,
    linestyle = [:solid :dash],
    linecolor = [:red :blue],
    xlim = (0,15),
    ylim = (0,15),
    title = "MVf, risky and riskfree assets",
    xlabel = "Std(Rp), %",
    ylabel = "ERp, %" )
scatter!(sqrt.(diag(Σ))*100,μ*100,color=:red,label="assets")
display(p1)

```

MVF, risky and riskfree assets



Data Containers

This notebook shows how to put data into different types of “containers” (arrays, tuples, named tuples, and dictionaries).

(However, it does not discuss [DataFrames.jl](#) or [Tables.jl](#).)

Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl");
```

Quick Summary

of how to create some data containers.

```
a  = 1:3
C  = [11 12; 21 22]    #an array

t = (a,C)              #a tuple
display(t)

nt = (a2=a,C2=C)       #a named tuple
display(nt)

D = Dict{:a2⇒a,:C2⇒C} #dictionary
display(D)
```

(1:3, [11 12; 21 22])

```
(a2 = 1:3, C2 = [11 12; 21 22])
```

Dict{Symbol, AbstractArray{Int64}} with 2 entries:

```
:a2 => 1:3
```

```
:C2 => [11 12; 21 22]
```

Arrays

are used everywhere in finance and statistics/econometrics.

Arrays: Create

(later notebooks will show many more ways of creating arrays)

```
A = [100,101]      #a vector  
printmat(A)        #or display(A)
```

```
100
```

```
101
```

Arrays: Access and Change Elements

To access a vector element, just do `A[2]` or similarly. Also, you can change an matrix element as in `B[2,1] = -999`, that is, arrays are mutable.

```
println("A[2] is ",A[2])      #access an element of a vector  
  
A[2] = -999                   #change an element of a matrix  
println("\nAfter a change of A[2], the vector is")  
printmat(A)
```

```
A[2] is 101
```

After a change of `A[2]`, the vector is

```
100
```

```
-999
```

Arrays of Arrays or Other Types (extra)*

You can store very different things (a mixture of numbers, matrices, strings) in an array. For instance, if `a` is a vector, `str` is a string and `C` is a matrix, then `x = [a,str,C]` puts them into a vector, where `x[1]` equals the string in `a`, etc.

If you later change elements of the matrix `C` then it will affect `x` (discussed at the end of the notebook).

To first allocate an array of arrays and later fill it, use, for instance, `y = Vector{Any}(undef,3)` can be filled with any sort of elements, while `y = Vector{Array}(undef,3)` can be filled with arrays.

```
a = 1:3
str = "Hazel"
C = [11 12; 21 22]
x = [a,str,C]          #element 1 of x is a
foreach(printmat,x)    #loops over the elements of x

#=
println("\nAlternative approach:")
y = Vector{Any}(undef,3)
y[1] = 1:3
y[2] = "Hazel"
y[3] = [11 12; 21 22]
foreach(printmat,y)
=#
```

```
1
2
3
```

```
Hazel
```

```
11      12
21      22
```

Tuples and Named Tuples

are very useful for collecting very different types of data (a number, a string, and a couple of vectors, say). They are also often used as inputs or outputs of functions.

Once created, you cannot change tuples (they are immutable). (Exception: *changing an element of an array* that belongs to the tuple will affect the tuple too.)

The next few cells show how to create tuples and named tuples, how to extract parts of them, merge them and what happens when you try to change them.

Tuples and Named Tuples: Create

```
a = 1:3          #will be used in the tuple/named tuple below
str = "Hazel"
C = [11 12;21 22]

t = (a,str,C)    #a tuple, or tuple(a,str,C)
display(t)

nt = (a=a,str=str,C=C) #a named tuple, can also give new name like (a2=a,...
display(nt)

nt_b = (;a,str,C); #also a named tuple, names are given by variables
display(nt_b)
```

```
(1:3, "Hazel", [11 12; 21 22])
```

```
(a = 1:3, str = "Hazel", C = [11 12; 21 22])
```

```
(a = 1:3, str = "Hazel", C = [11 12; 21 22])
```

Tuples and Named Tuples: Extract Elements

```
(a2,str2,C2) = t          #extract the tuple into variables ("destructuring")
println("a2 and str2 are: $a2 $str2 \n") #a2 is the same as t[1]

println("t[3] is:")       #can index into (tuple) t
printmat(t[3])

println("nt.C is:")       #we can use nt.C as a name (nt is a named tuple)
printmat(nt.C)

(;C,a) = nt               #extract by symbol name, not position
println("C is:")
printmat(C)
```

a2 and str2 are: 1:3 Hazel

t[3] is:

11	12
21	22

nt.C is:

11	12
21	22

C is:

11	12
21	22

Tuples and Named Tuples: Add and Merge

```
t = (t...,3.14)          #add an element like this to a tuple
display(t)

nt = (;nt...,abc=3.14)   #and to a named tuple
display(nt)

nt_c = nt[:,a,:C]        #create a new named tuple as a subset of another one
display(nt_c)

nt_d = merge(nt,(abc=3.14,x2="a")) #merge named tuples to create a new one
display(nt_d)
```

(1:3, "Hazel", [11 12; 21 22], 3.14)

(a = 1:3, str = "Hazel", C = [11 12; 21 22], abc = 3.14)

(a = 1:3, C = [11 12; 21 22])

(a = 1:3, str = "Hazel", C = [11 12; 21 22], abc = 3.14, x2 = "a")

```
#t[1] = -999          #cannot change the tuple, uncomment to get an error
#t[4] = 34            #cannot add elements like this, uncomment to get an error
```

Create a Tuple Dynamically (extra)*

when values and/or names are created dynamically in the program, not hard-coded as above.

Using `tuple(values...)` and `NamedTuple{names}(values)` creates tuples/named tuples.

```
values = [a,str,C]                                #the values to be used below

t2 = tuple(values...)                             #or just (values...,)
display(t2)

names = (:a, :b, :c)                             #should be a tuple of symbols (:a)
nt2    = NamedTuple{names}(values)                #or (;zip(names,values)...)
display(nt2)
```

```
(1:3, "Hazel", [11 12; 21 22])
```

```
(a = 1:3, b = "Hazel", c = [11 12; 21 22])
```

Dictionaries

offer a flexible way to collect different types of data. Dictionaries can (in contrast to tuples) be changed: they are mutable. In contrast, they are often a bit slower.

(As usual, changing elements of an array that belongs to the dictionary will affect the dictionary too.)

A dictionary is organised as (key,value) pairs, where the key is the name of the element. You can loop over the elements (see below) and also change/add elements in a loop.

By using `get.(Ref(D),[:a,:C],missing)` you can extract several variables at once.

```
a    = 1:10
str  = "Hazel"
C    = [11 12; 21 22]

D = Dict{:a⇒a,:str⇒str,:C⇒C}          #dictionary, "a" instead of :a works too
#D = Dict{(:a,a),(:str,str),(:C,C)}   #alternative syntax
display(D)

println("\n","D[:C] is ",D[:C])       #extract an element

D[:a] = -999                          #can change an element
```

```
D[:verse2] = "Stardust"           #can add an element this way
display(D)

D_b = merge(D,Dict(:abc=>3.14));   #merge Dicts
display(D_b)
```

```
Dict{Symbol, Any} with 3 entries:
 :a    => 1:10
 :str  => "Hazel"
 :C    => [11 12; 21 22]
```

D[:C] is [11 12; 21 22]

```
Dict{Symbol, Any} with 4 entries:
 :a      => -999
 :verse2 => "Stardust"
 :str    => "Hazel"
 :C      => [11 12; 21 22]
```

```
Dict{Symbol, Any} with 5 entries:
 :a      => -999
 :abc    => 3.14
 :verse2 => "Stardust"
 :str    => "Hazel"
 :C      => [11 12; 21 22]
```

Dictionary: A Potential Pitfall in Adding (extra)*

If you have created a dict with only numbers by

```
D = Dict(:aa=>1)
```

then you cannot add eg. a string by `D[:cc] = "hello"` since D is only set up to accept variables of the type Int.

```

D = Dict{:aa=>1}
#D[:cc] = "hello"           #error since D only accepts Int

D = Dict{Any,Any}(:aa=>1)    #this works
D[:cc] = "hello"
display(D)

```

Dict{Any, Any} with 2 entries:

```

:aa => 1
:cc => "hello"

```

Dictionary: Create Dynamically (extra)*

See below for examples.

Remark: if you have the names as an array of strings (names = ["a","b","c"]), but want symbol names (:a etc), then use Symbol.(names).

```

names = (:a, :b, :c)           #or ["a","b","c"]
values = [a,str,C]

D = Dict(zip(names,values))
display(D)

#=                               #alternative approach
D = Dict()                       #empty dictionary
for i = 1:length(values)         #loop
    D[names[i]] = values[i]      #add this to the dictionary
end
display(D)
=#

```

Dict{Symbol, Any} with 3 entries:

```

:a => 1:10
:b => "Hazel"
:c => [11 12; 21 22]

```

From a Dict to a NamedTuple and Back Again (extra)*

```
nt = (;D...)           #create a named tuple from a dict
display(nt)

println()
D2 = Dict{pairs(nt)}   #create a dict from a named tuple
display(D2)
```

```
(a = 1:10, b = "Hazel", c = [11 12; 21 22])
```

Dict{Symbol, Any} with 3 entries:

```
:a => 1:10
:b => "Hazel"
:c => [11 12; 21 22]
```

Loop over a Tuple/Named Tuple/Dictionary

by using (key,value) in pairs()

```
for (key,value) in pairs(nt)           #or over `t` or `D`
    println("$key: $value")
end
```

```
a: 1:10
b: Hazel
c: [11 12; 21 22]
```

Your Own Tailor Made Data Type (struct)

It is sometime convenient to define your own struct as a container. The struct command creates an immutable type (you cannot change it, except for elements of arrays that belong to it). There is also a mutable struct approach.

```

a   = 1:10
str = "Hazel"
C   = [11 12; 21 22]

struct MyType          #change to `mutable struct` to be able to change it later
    x                  #can be anything
    s::String          #has to be a String
    z::Array           #has to be an Array
end

x1 = MyType(a,str,C)    #has to specify all arguments

println("x1: ",x1,"\n")
println("x1.s: ",x1.s)

#x1 = MyType(1:10,10,[1;2])    #error since 10 is not a string
#x1.x = 3                     #error since we cannot change

```

```

x1: MyType{1:10, "Hazel", [11 12; 21 22]}

```

```

x1.s: Hazel

```

Functions

This notebook shows how to define functions with one (or several) inputs and outputs. It also shows how to use the dot (.) syntax to apply a function to each element of an array.

Load Packages and Extra Functions

```
using Printf  
  
include("src/printmat.jl");
```

Functions with One Output

The Basic Approach

The basic approach to define a function with the name `fn` is

```
function fn(x,b)  
    ...(some code that calculates y)  
    return y  
end
```

Once you have defined a function, you can use it (call on it) by, for instance, `y1 = fn(2,1)` or by `y1 = fn(b1,1)` if `b1=2` is already defined. This will generate a `y1` variable (not a `y` variable) in the workspace. Inside the function, `x` is 2 and `b` is 1.


```

function fn1(x,b)           #x and b are the inputs
    c = 0.5                 #c is only "seen" inside the function
    y = b*(x-1.1)^2 - c
    return y                #this is the output
end

y1 = fn1(2,1)
printlnPs("result from fn1(2,1): ",y1)

```

```

result from fn1(2,1):      0.310

```

Performance Tips (extra)*

- Wrapping performance critical code in functions often speeds up the computation, since it helps Julia/the compiler to make optimizations.
- Do not use any global variables inside a function. It slows down the computation and can lead to surprising behaviour. Instead, make all the inputs explicit function arguments. (Alternatively, declare the global variables to be constants by `const a = 2`).

Default Values for the Inputs

You can change the first line of the function to specify default values as

```

function fn2(x,b=1)

```

In this case you can call on the function as `fn2(x)` and the value of `b` will default to 1. (Inputs with default values should be towards the end of the list of inputs.)

```

function fn2(x,b=1)         #b=1 is the default in case we call as fn2(x)
    y = b*(x-1.1)^2 - 0.5
    return y
end

printlnPs("result from fn2(2,1) and fn2(2): ",fn2(2,1),fn2(2))

```

```

result from fn2(2,1) and fn2(2):      0.310      0.310

```

Elementwise Evaluation

To apply the function to each element of arrays x and b , use the dot syntax (broadcasting):

```
y = fn.(x,b)
```

This calculates $\text{fn}(x[i], b[i])$ for each pair $(x[i], b[i])$.

Instead, with $\text{fn.}(x, 2)$, you calculate $\text{fn}(x[i], 2)$ for each element $x[i]$.

```
x1 = [1,1.5]
b1 = [2,3]

println("\nresult from fn2.(x1,2): ")
printmat(fn2.(x1,2))

println("\nresult from fn2.(x1,b1): ")
printmat(fn2.(x1,b1))
```

```
result from fn2.(x1,2):
-0.480
-0.180
```

```
result from fn2.(x1,b1):
-0.480
-0.020
```

Short Form (extra)*

We can also use short forms of a function as in the cell below.

The first version (`fn3`) is just a single expression. It could span several lines, but it should be a single expression. The second version (`fn3b`) is a sequence of expressions (a “compound expression”) separated by semicolons (`;`), where the last expression is the function output.

As a matter of taste, I think `fn3b()` is better written as a traditional function.

```
fn3(x,b) = b*(x-1.1)^2 - 0.5           #short form of a function

fn3b(x,b) = (c = 0.5;b*(x-1.1)^2 - c)  #this works too. Notice the ;

printlnPs("result from fn3(1.5,1) and fn3b(1.5,1): ",fn3(1.5,1),fn3b(1.5,1))
```

```
result from fn3(1.5,1) and fn3b(1.5,1):    -0.340    -0.340
```

Explicit Names of the Inputs: Keyword Arguments

You can also define functions that take *keyword arguments* like in

```
function fn(x;b,c)
```

Notice the semi-colon (;). You can also specify default values as `fn(x;b=1,c=0.5)`

In this case, you *call* on the function by `fn(x;c=3,b=2)` or just `fn(x)` if you want to use the default values. The explicit naming helps remembering/interpreting what the arguments represent. When calling on the function, you can pass the keyword arguments in *any order* and you can use comma (,) instead of semi-colon (;). Using the semi-colon has an advantage, though: if you already have `b` defined, then you can call as `fn(x;c=3,b)`.

```
function fn4(x;b=1,c=0.5)
    y = b*(x-1.1)^2 - c
    return y
end

b = 2
printlnPs("result from fn4(1,c=3,b=2): ",fn4(1;c=3,b))
```

```
result from fn4(1,c=3,b=2):    -2.980
```

(Extra)* You can also use a `Dict()` to supply the keyword arguments as in

```
opt = Dict{:b=>1,:c=>0.5}
fn(x;opt...)
```

Notice that the dictionary must use symbols and that you need to use `; and ...` in the call. It is also possible to mix traditional keywords with a dictionary as in `opt = Dict{:c=>0.5}; fn(x;b=1,opt...)`

Variable Number of Inputs (extra)*

When *defining a function*, we can use the `x...` syntax (also called “slurping”) on the last non-keyword input to capture a variable number of inputs. Later, when calling on this function, several inputs will be combined into a vector `x`.

For instance, `fn5(-9,1,2,b=10)` in the next cell will create the 2-element vector

`-9 + 1*10`

`-9 + 2*10`

```
function fn5(a,x...;b=1)    #a is traditional input, x... captures several inputs; b=1 is a keyword
    n = length(x)           #how many inputs are there in 'x'?
    y = zeros(n)
    for i in 1:n            #loop over the elements in x
        y[i] = a + x[i]*b
    end
    return y
end

y = fn5(-9,1,2;b=10)
println("result from fn5(-9,1,2,b=10): ")
printmat(y)
```

result from `fn5(-9,1,2,b=10)`:

1.000

11.000

Conversely, you can also use the `x...` syntax (also called “splatting”) when *calling on a function* that requires several inputs.

In the next cell, when running `fn5b([1,2]...)`, `x1` will be 1 and `x2` will be 2.

```
function fn5b(x1,x2)
    println(x1," and then also ",x2)
    return nothing
end

fn5b([1,2]...)
```

1 and then also 2

Functions with Several Outputs

Several Outputs 1: Basic Approach

A function can produce a “tuple” like (y_1, y_2, y_3) as output.

In case you only want the first two outputs, call as $(y_1, y_2,) = \text{fn}(x)$. Instead, if you only want the second and third outputs, call as $(_, y_2, y_3) = \text{fn}(x)$

You can also extract the second output as $y_2 = \text{fn}(x)[2]$

If Y_1 is an already existing array, then $(Y_1[3], y_2,) = \text{fn}(x)$ will change element 3 of Y_1 (and also assign a value to y_2).

```
function fn11(x,b=1)
    y1 = b*(x-1.1)^2 - 0.5
    y2 = b*x
    y3 = 3
    return y1, y2, y3
end

(y1,y2,) = fn11(1,2)
printlnPs("The first 2 outputs from fn11(1,2): ",y1,y2)

y2 = fn11(1,2)[2]          #grab the second output
printlnPs("\nThe result from calling fn11(1,2)[2]: ",y2)

Y1 = zeros(5)              #create an array
(Y1[3],y2,) = fn11(1,2)    #put result in existing array
printlnPs("\nY1 array after calling fn11(1,2): ",Y1)
```

The first 2 outputs from fn11(1,2): -0.480 2

The result from calling fn11(1,2)[2]: 2

Y1 array after calling fn11(1,2): 0.000 0.000 -0.480 0.000 0.000

Several Outputs 2: Named Tuples (extra)*

Instead of returning several values, it might be easier to combine them into a named tuple and then export that.

You could end the function with

```

        y = (a=y1,b=y2,c=y3)                #named tuple
        return y
end

```

Several Outputs 3: Elementwise Evaluation (extra)*

...can be tricky, because you get an array (same dimension as the input) of tuples instead of a tuple of arrays (which is perhaps what you want).

One way around this is to reshuffle the output to get a tuple of arrays, using `ntuple()` as below.

Alternatively, you could loop over the function calls or write the function in such a way that it directly handles array inputs (without the dot). The latter is done in `fn14()`.

```

y = fn11.([1,1.5],2)
println("y, a 2-element vector of tuples (3 elements in each):")
printmat(y)

(y1,y2,y3) = ntuple(i->getindex.(y,i),3)    #split up into 3 vectors

println("\nthe vectors y1, y2 and y3:")
printmat([y1 y2 y3])

```

```

y, a 2-element vector of tuples (3 elements in each):
(-0.48, 2.0, 3)
(-0.180000000000000016, 3.0, 3)

```

```

the vectors y1, y2 and y3:
-0.480      2.000      3.000
-0.180      3.000      3.000

```

```

function fn14(x,b=1)                #x can be an array
    y1 = b*(x.-1.1).^2 .- 0.5
    y2 = b*x
    y3 = 3
    return y1, y2, y3
end

(y1,y2,y3) = fn14(x1,2)             #function written to handle arrays
println("result from fn14(x1,2): ")

```

```
printmat([y1 y2])
printmat(y3)
```

```
result from fn14(x1,2):
```

```
-0.480    2.000
-0.180    3.000
```

3

Documenting Your Function

To use Julia's help function (`?FunctionName`, `@doc FunctionName`) or `@doc2 FunctionName` (in VS Code, using the macro from `printmat.jl`), put the documentation in triple quotes, just above the function definition. (No empty lines between the last triple quote and the start of the function.) The cell below illustrates a simple case.

```
"""
    fn101(x,b=1)

Calculate `b*(x-1.1)^2 - 0.5`.

### Arguments
- `x::Number`:    an important number
- `b::Number`:    another number

"""
function fn101(x,b=1)
    y = b*(x-1.1)^2 - 0.5
    return y
end
```

fn101

```
@doc2 fn101      #get help on the fn101 function that works also in VS Code
#?fn101          #works in REPL and jupyter, not in VS Code
```

```
fn101(x,b=1)
```

Calculate $b*(x-1.1)^2 - 0.5$.

Arguments

- `x::Number`: an important number
- `b::Number`: another number

Sending a Function to another Function

You can use a function name (for instance, `cos`) as an input to another function. Alternatively, use an anonymous function (see `→` below).

```
function fn201(f,a,b)    #f is a function, could also write f::Function
    y = f(a) + b
    return y
end

#here f is the cos function
y1 = fn201(cos,3.145,10)
printlnPs("result from fn201(cos,3.145,10): ",y1)

#here f is z→mod(z,2), an anonymous function
y2 = fn201(z→mod(z,2),3.145,10)
printlnPs("\nresult from fn201(z→mod(z,2),3.145,10): ",y2)
```

```
result from fn201(cos,3.145,10):      9.000
```

```
result from fn201(z→mod(z,2),3.145,10):  11.145
```


Plots

This file demonstrates how to create plots in Julia by using the [Plots.jl](#) package.

The notebook uses the default backend (GR), but Plots.jl supports several alternatives. (See the package document for details.)

See [Plots Gallery](#) for examples (with code snippets available by clicking on a figure).

Load Packages and Extra Functions

```
using Dates, Plots

default(size = (480,320),fmt = :png) #:svg gives prettier plots, but may not work well on GitHub
#pythonplot()                       #to use another backend
```

A First Plot

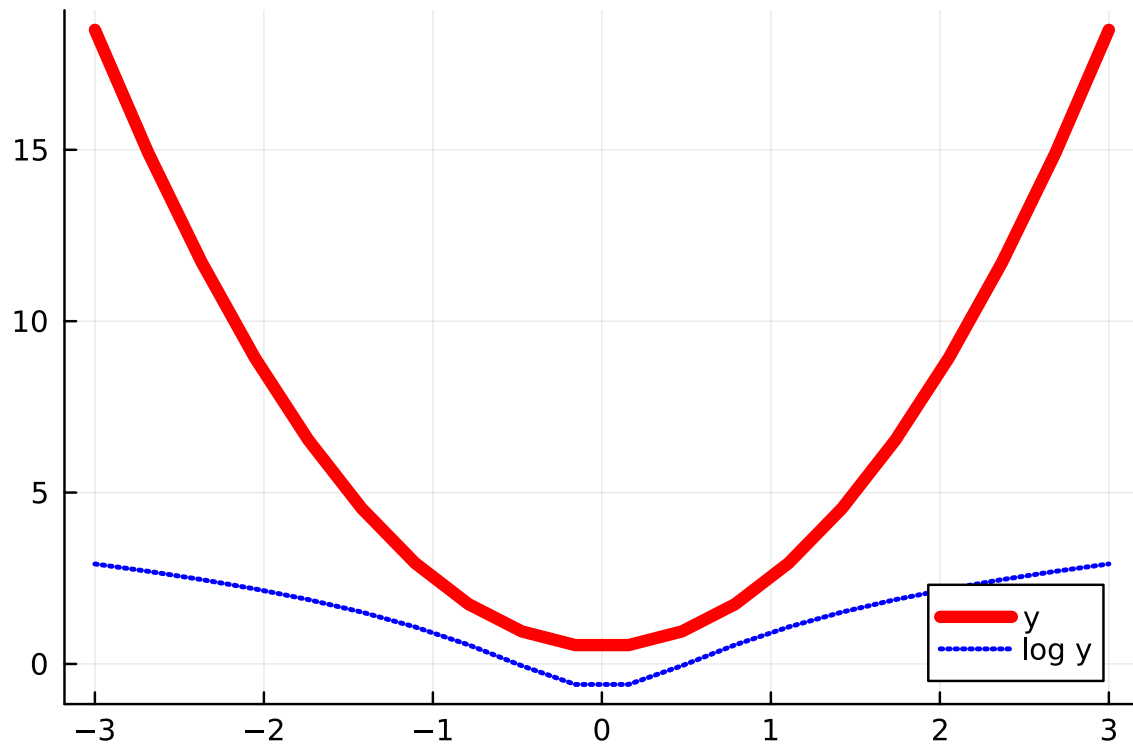
The next cell creates and shows a first plot.

The first plot takes a bit of time. Subsequent plots are much quicker.

The `savefig()` command (see below) saves the plot to a graphics file.

```
x = range(-3,3,length=20)      #something to plot
y = 2*x.^2 .+ 0.5

p1 = plot( x,[y log.(y)],      #same x axis
           label = ["y" "log y"], #row vector, one element for each column in [y log.(y)]
           linecolor = [:red :blue],
           linestyle = [:solid :dot],
           linewidth = [5 2] )
display(p1)                    #not needed in notebook, but useful in script, not needed with pythonplot
```



```
plotattr("linestyle") #see all available options
```

:linestyle

Style of the line (for path and bar stroke). Choose from [:auto, :solid, :dash, :dot, :dashdot, :dashdot]

Aliases: (:linestyles, :ls, :s, :style).

Type: Symbol.

'Series' attribute, defaults to 'solid'.

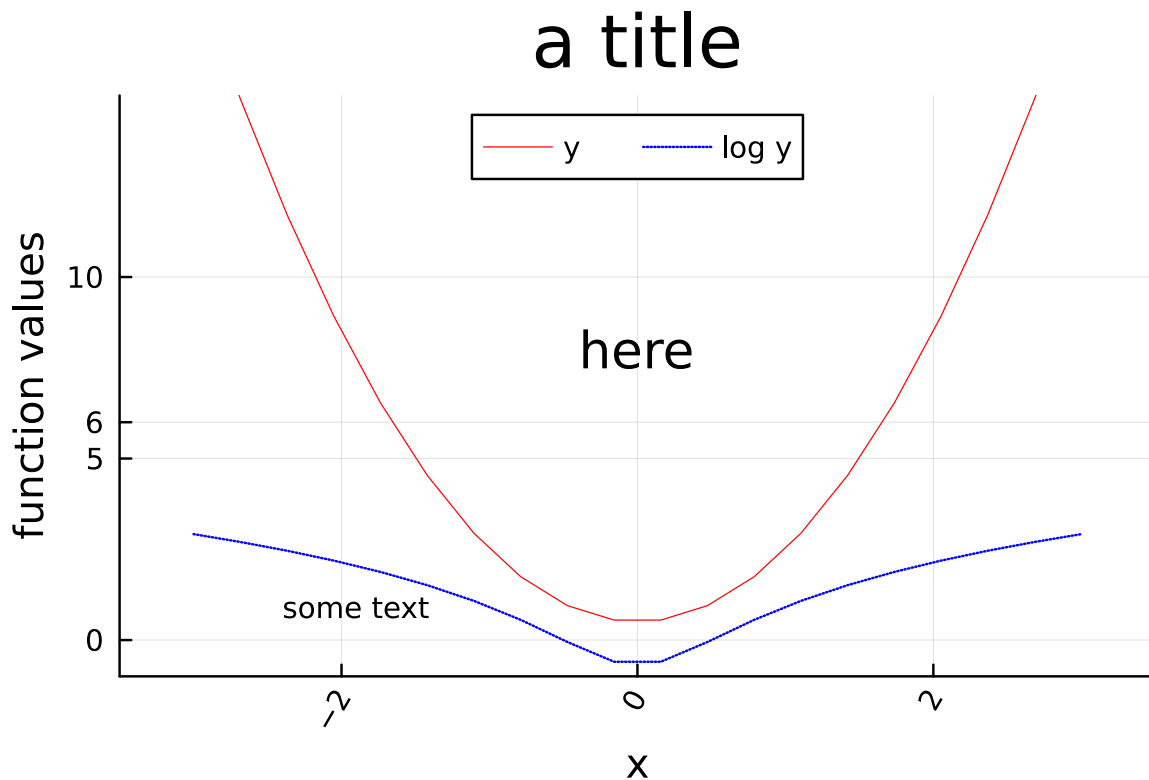
#now with title, labels and more

```
p1 = plot( x,[y log.(y)],
           label = ["y" "log y"],
           legendcolumns = 2,
           legend = :top,
           linecolor = [:red :blue],
```

```

linestyle = [:solid :dot],
linewidth = [0.5 1],
title = "a title",
titlefontsize = 20,
xlabel = "x",
ylabel = "function values",
xrotation = 60,
xlims = (-3.5,3.5),
ylims = (-1,15),
xticks = [-2;0;2],
yticks = [0;5;6;10],
annotation = ([-1.9,0],[0.9,8],[text("some text",8),"here"] )
display(p1)

```



```

savefig("AFirstPlot.pdf");           #change to .svg or .png
println("Check that the graphics file is now in the same folder as the notebook.")

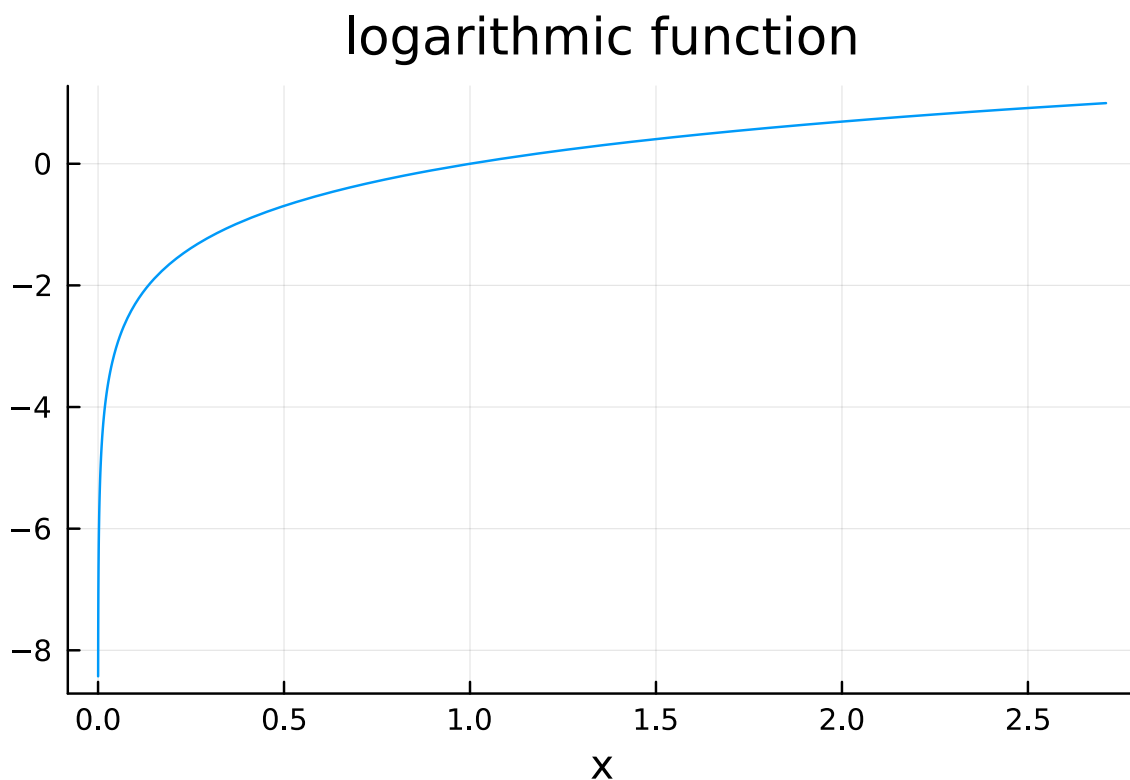
```

Check that the graphics file is now in the same folder as the notebook.

Plotting a Function

...can also be done by, for instance, `plot(log,0,2.71)`. This plots the $\log(x)$ function for x from 0 to 2.71.

```
p1 = plot( log,0,2.71,  
           legend=false,  
           title = "logarithmic function",  
           xlabel = "x" )  
display(p1)
```



Subplots

Use `layout = (2,2)` to create a 2x2 plot and `layout = @layout[a a;a _]` to create a 2x2 plot where the last subplot is blank. In the latter case, a just indicates that there should be a visible subplot (you could use another symbol, eg b), while `_` indicates that the subplot should be blank.

```

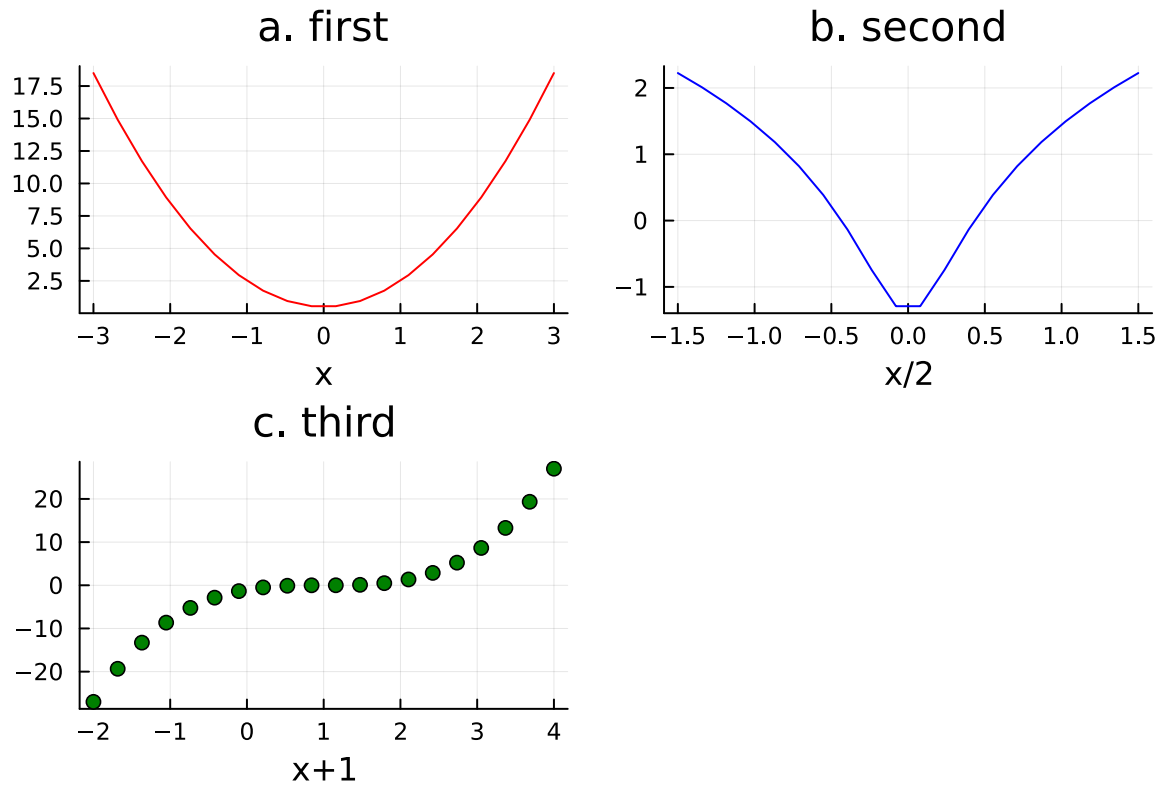
p1 = plot( x,y,                                #define subplot 1
           linecolor = :red,
           title = "a. first",
           xlabel = "x",
           legend = false )

p2 = plot( x/2,log.(y/2),
           linecolor = :blue,
           title = "b. second",
           xlabel = "x/2",
           legend = false )

p3 = scatter( x.+1,x.^3,
              markercolor = :green,
              title = "c. third",
              xlabel = "x+1",
              legend = false )

pAll = plot( p1,p2,p3,                        #combine the subplots
             layout = @layout[a a;a _],
             size = (600,400) )
display(pAll)

```



Adding Lines (Horizontal, Vertical and More)

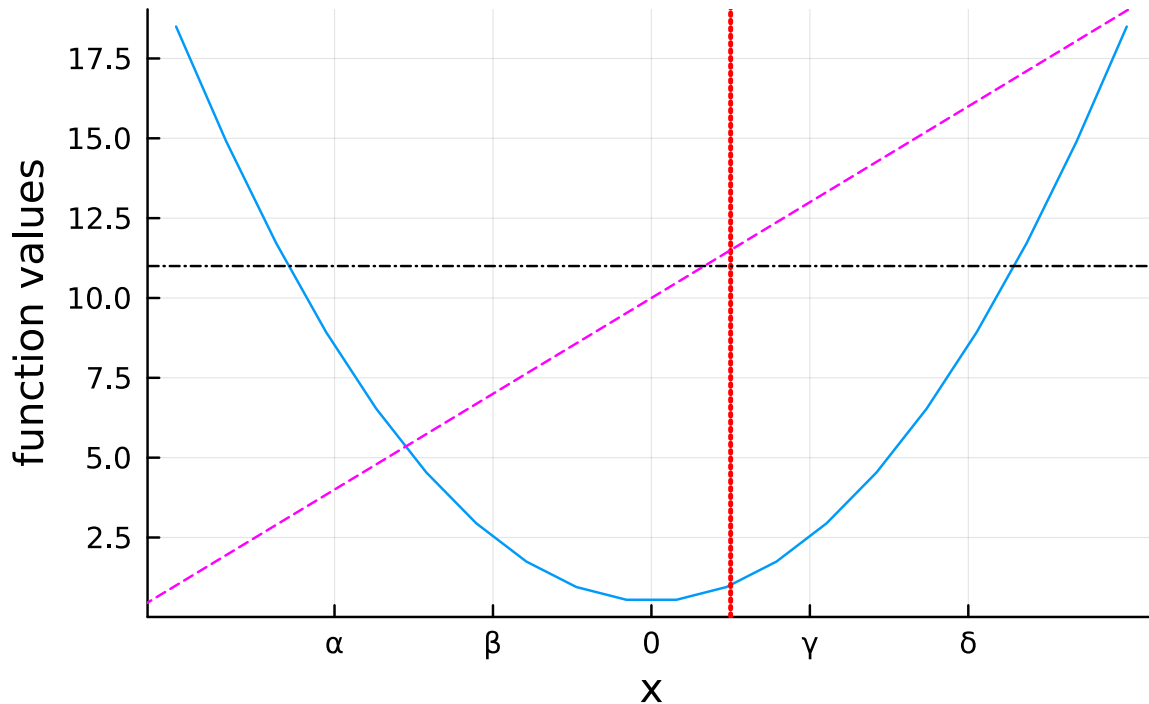
and changing the x-tick marks.

```
xtickStr = ["α","β","θ","γ","δ"]

p1 = plot( x,y,
           legend = false,
           title = "With reference lines",
           xlabel = "x",
           ylabel = "function values",
           xticks = (-2:2,xtickStr) )

vline!([0.5],linecolor=:red,line=:dot,2)) #easiest to not integrate this in plot()
hline!([11],linecolor=:black,line=:dashdot)
Plots.abline!(3,10,linecolor=:magenta,line=:dash) #yes, it currently needs the Plots. prefix
display(p1)
```

With reference lines



LaTeX in the Figure

`gr()` can include LaTeX elements. Such strings need to be just LaTeX code, so you need a work-around to combine it with text: see `title` in the cell below. In particular, notice that `\mathrm{}` creates ordinary text and that `\` gives a space. To make all text (including tick labels etc) use the traditional TeX font, use `fontfamily="Computer Modern"`.

To insert the value of the variable `z` into the LaTeX string (“string interpolation”), use `%%z` instead of the standard `$z`.

(You may also consider using the `pythonplot()` backend which has extensive support for LaTeX.)

```
using LaTeXStrings          #add some LaTeX to the figure

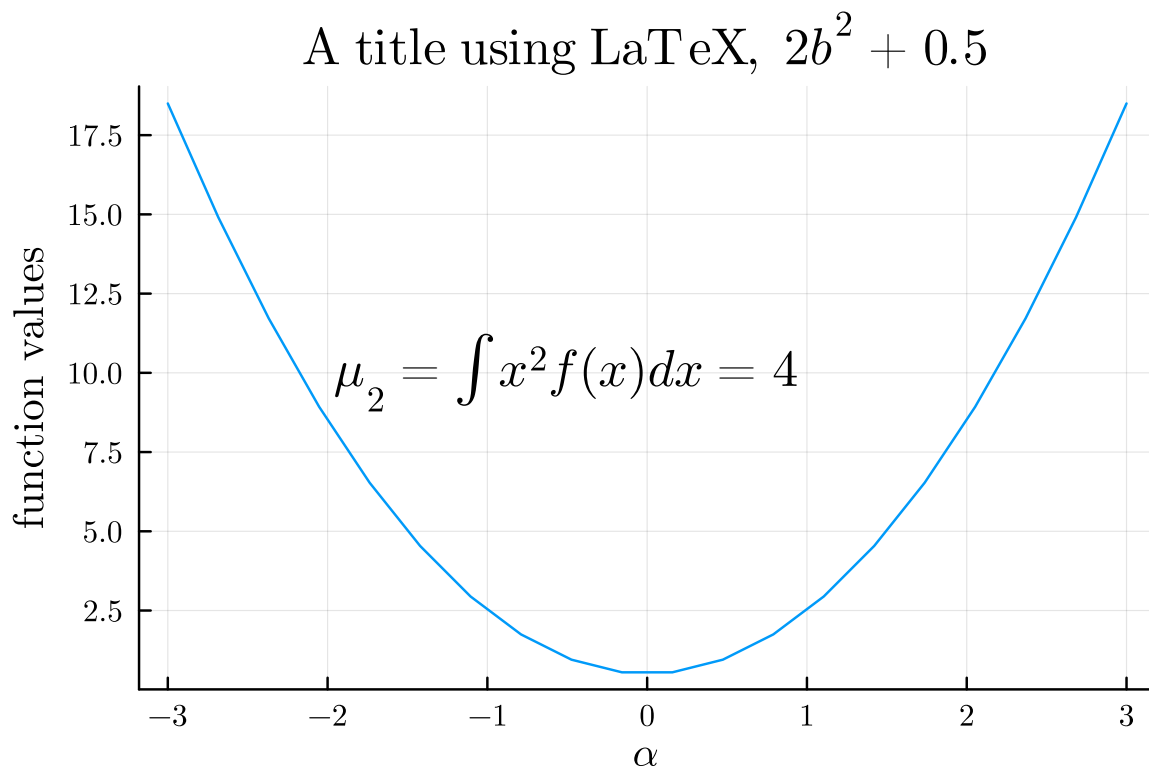
z = 4                        #a value to be used inside the string

p1 = plot( x,y,legend = false,
            title = L"$\mathrm{A \ title \ using \ LaTeX, \ }^2 b^2 + 0.5$",
            xlabel = L"$\alpha$",
```

```

ylabel = "function values",
annotation = (-0.5,10,L"\mu_2 = \int x^2 f(x) dx = %z $"), #notice the % in %z
fontfamily = "Computer Modern" ) #comment out this if it does not work
display(p1)

```



```

savefig("ASecondPlot.pdf");

```

Bars and Stairs Plots

(For the `bar()` plot, the option `orientation=:h` makes the bars horizontal instead.)

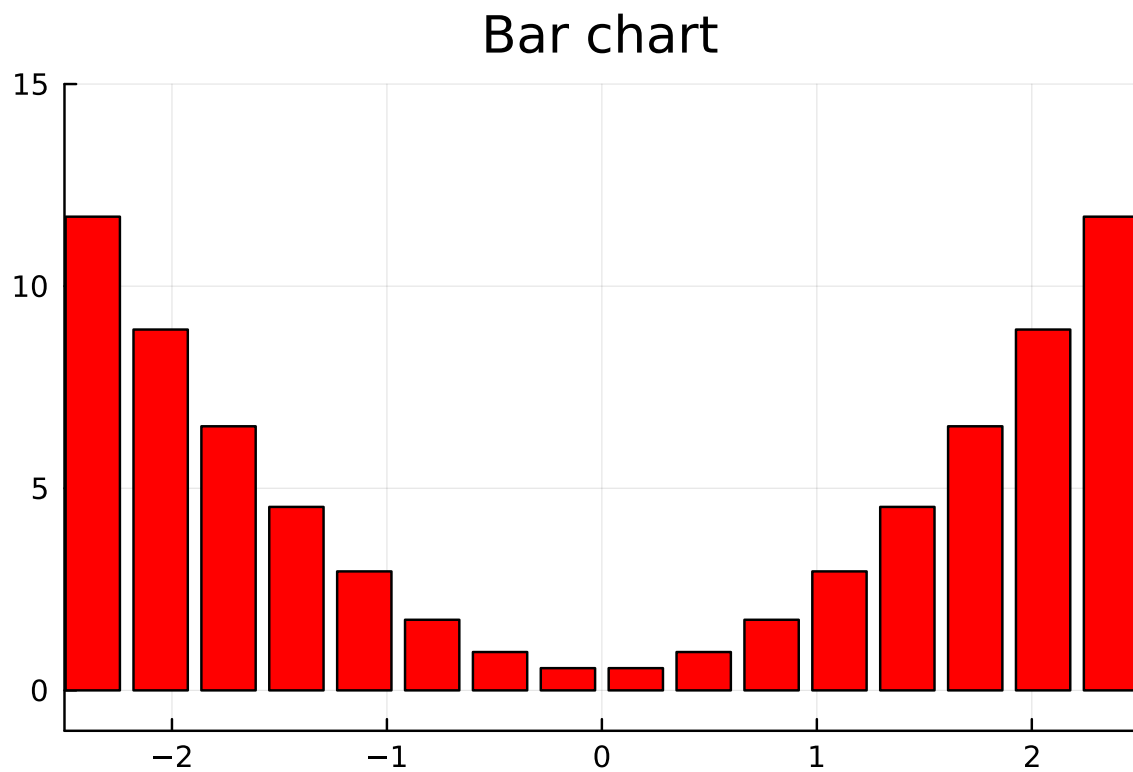
```

p1 = bar( x,y,
          legend = false,
          fillcolor = :red,
          xlims = (-2.5,2.5),
          ylims = (-1,15),

```

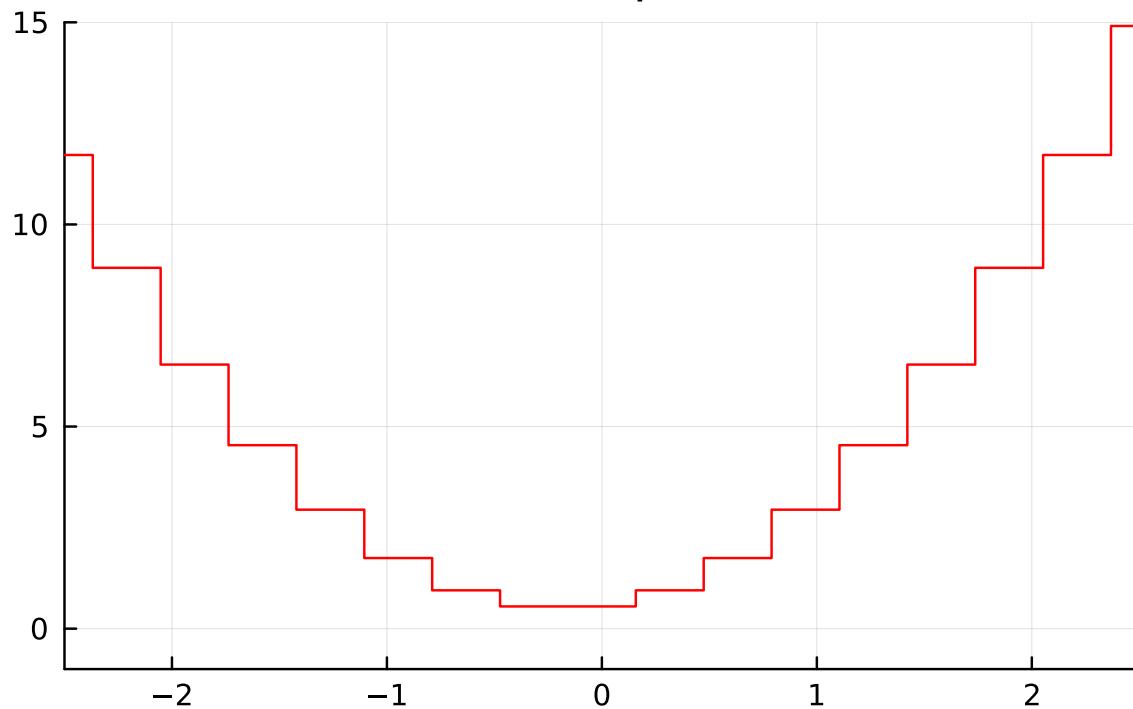


```
title = "Bar chart" )  
display(p1)
```



```
p1 = plot( x,y,  
           linestyle = :steppre,  
           linecolor = :red,  
           legend = false,  
           xlims = (-2.5,2.5),  
           ylims = (-1,15),  
           title = "Stairs plot" )  
display(p1)
```

Stairs plot



Surface Plots

The camera = (60,30) in the second figure rotates the figure.

```
x = range(-3,3,length=20)      #create some "data" to plot
y = range(1,7,length=25)
z = 2*x.^2 .+ (y' .-4).^2

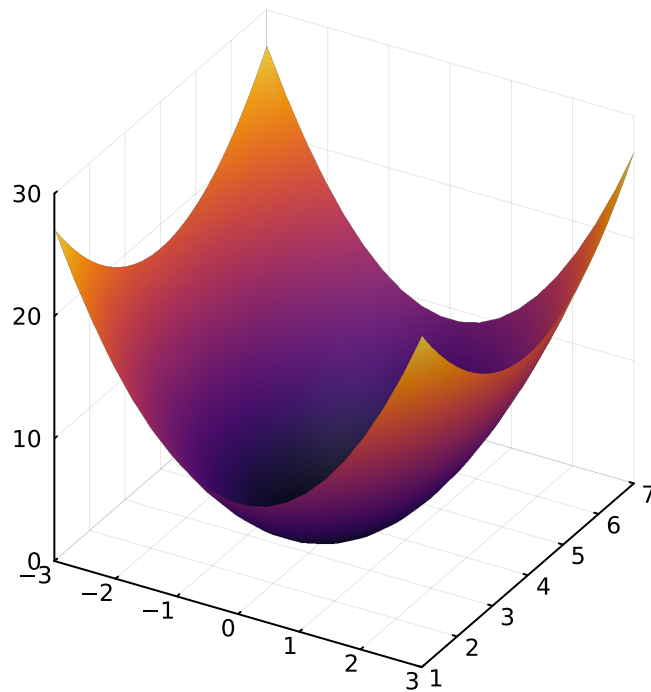
#notice the arguments: x,y,z'
println(size(x),size(y),size(z'))

p1 = surface( x,y,z',
              size = (600,433),
              legend = false,
              xlims = (-3,3),
              ylims = (1,7),
              zlims = (0,30),
```

```
title = "Surface plot" )  
display(p1)
```

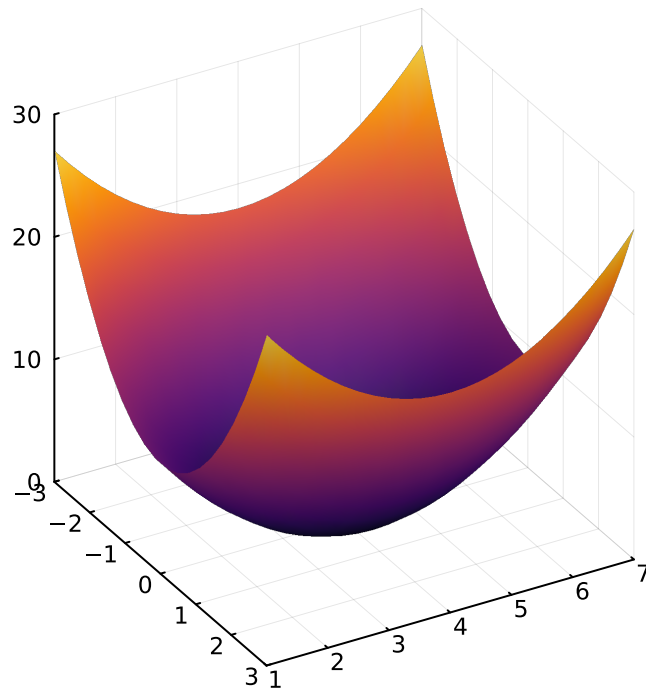
```
(20,)(25,)(25, 20)
```

Surface plot



```
p1 = surface( x,y,z',  
              size = (600,433),  
              camera = (60,30),  
              legend = false,  
              xlims = (-3,3),  
              ylims = (1,7),  
              zlims = (0,30),  
              title = "Surface plot, rotated" )  
display(p1)
```

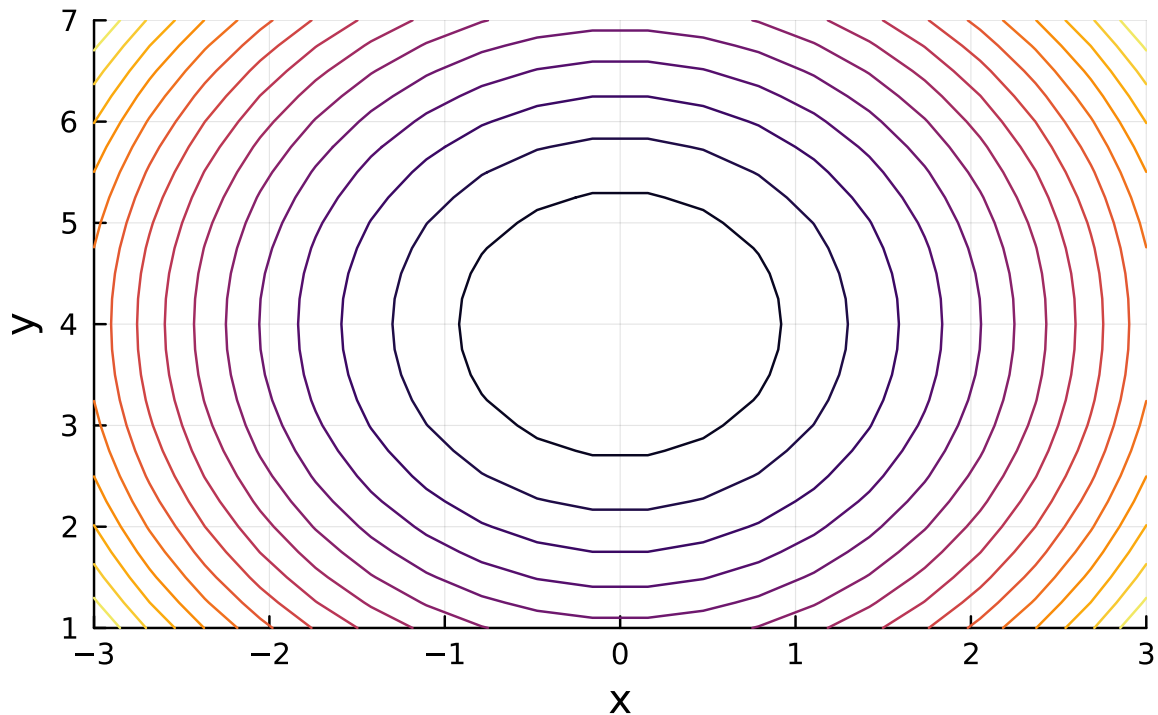
Surface plot, rotated



Contour Plot

```
p1 = contour(x,y,z',                                #notice the transpose: z'  
            legend = false,  
            title = "Contour plot of loss function",  
            xlabel = "x",  
            ylabel = "y")  
display(p1)
```

Contour plot of loss function



Scatter and Histogram

Scatter plots allows many customisations, for instance, to let the marker size vary across data points (see `markersize`).

For the histogram, `normalize = true` makes the area equal to one, so the histogram becomes directly compatible to a probability density function.

```
plotattr("markershape")           #to see available options
```

`:markershape`

Choose from `[ :none, :auto, :circle, :rect, :star5, :diamond, :hexagon, :cross, :xcross, :utriangle, :dt`

Aliases: `(:markershapes, :shape)`.

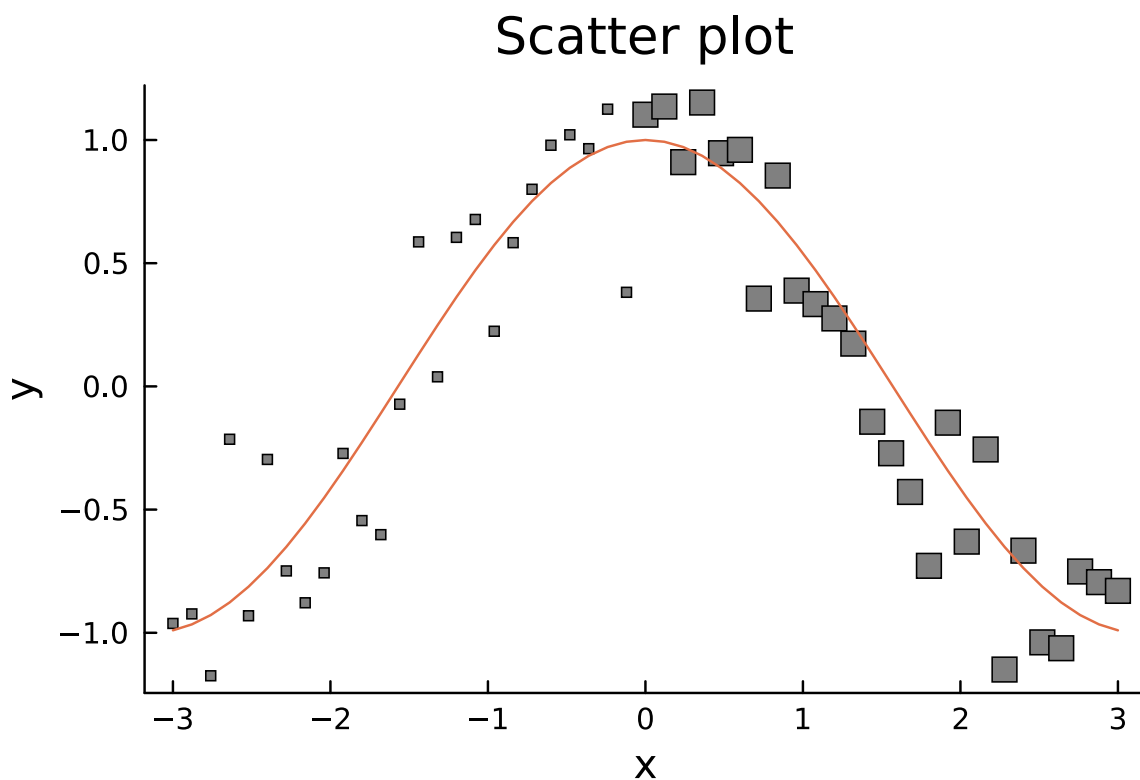
Type: `Union{Symbol, Shape, AbstractVector}`.

'Series' attribute, defaults to 'none'.

```
N = 51
x = range(-3,3,length=N)
y = cos.(x) + randn(N)/3

ms = vcat(2*ones(25),5*ones(N-25))    #marker size, N or a scalar

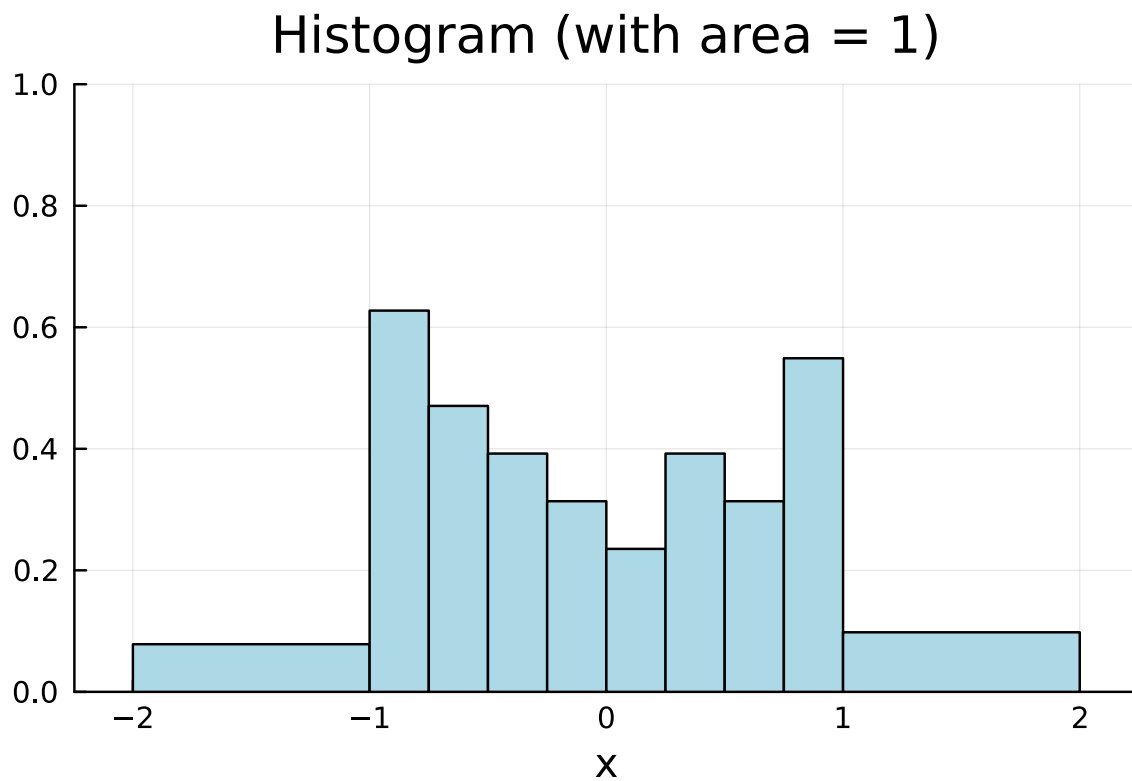
p1 = scatter( x,y,
              markersize = ms,
              markercolor = :grey,
              markershape = :rect,
              legend = false,
              grid = false,
              title = "Scatter plot",
              xlabel = "x",
              ylabel = "y" )
plot!(x,cos.(x))
display(p1)
```



```
bins = vcat([-2,-1],[-0.75:0.25:1,2]) #bins for the histogram, here of different sizes (for illustrat

p1 = histogram( y,
                bins = bins,
                normalize = true,
                fillcolor = :lightblue,
                ylims = (0,1),
                legend = false,
                title = "Histogram (with area = 1)",
                xlabel = "x" )

display(p1)
```



Time Series Plots

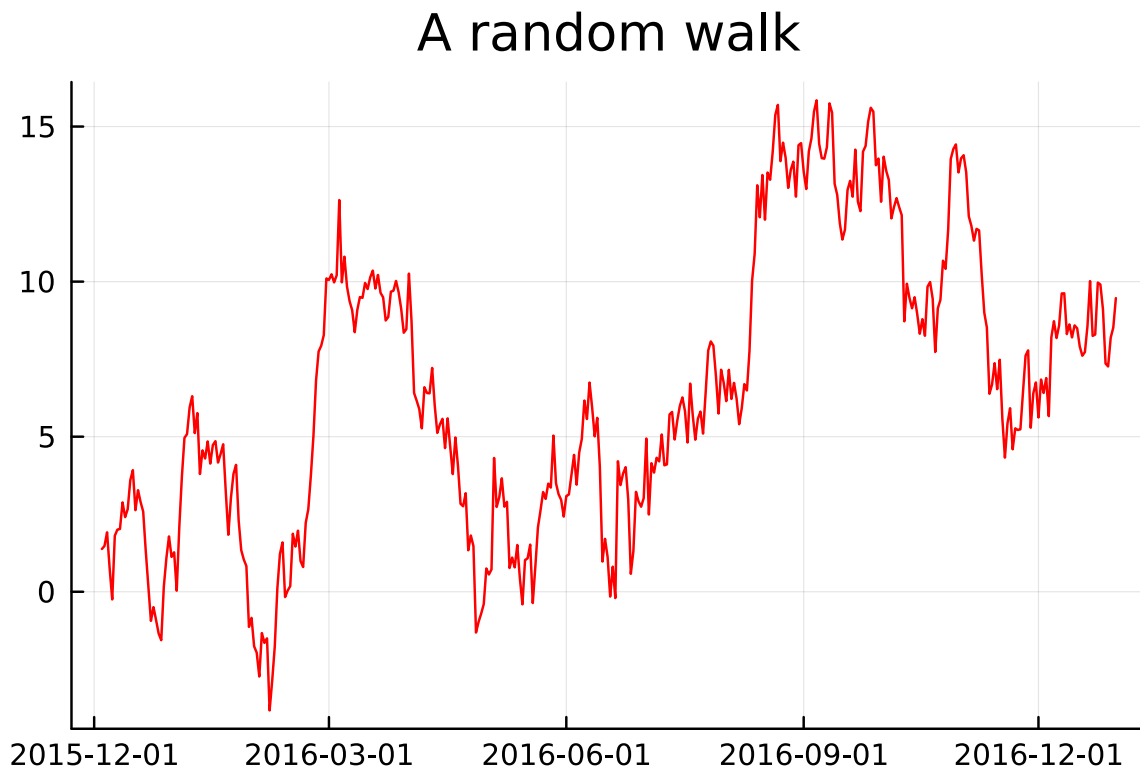
We can use a `Date()` vector as the x-axis variable.

To insert lines, annotations and tick marks, the `x` values are defined as, for instance, `Date(2016,8,15)`.

It is often useful to configure the printing of the dates (shorter, not too many). This is done in the second figure.

```
dN = Date(2015,12,4):Day(1):Date(2016,12,31) #just faking some dates
y = randn(length(dN)) #some random numbers to plot

p1 = plot( dN,cumsum(y),
           linecolor = :red,
           legend = false,
           title = "A random walk" )
display(p1)
```



```
xTicksLoc = [Date(2016,1),Date(2016,7),Date(2017,1)]
xTicksLab = Dates.format.(xTicksLoc,"Y:m") #see Dates.format for more options

p1 = plot( dN,cumsum(y),
           linecolor = :red,
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
```



```

title = "A random walk, with better tick marks",
annotation = (Date(2016,5),0,text("some text",8)) )

vline!([Date(2016)],linecolor=:black,line=(:dash,1))
display(p1)

```



Animations

The following cell animates the probability density function (pdf) of a normally distributed variable x with mean $x_0 + m\mu$ and variance $m\sigma^2$ as m increases.

(Background: this shows how the distribution of a random walk with drift would change as the horizon m is extended. This is, for instance, used in pricing of options which expire at different horizons.)

```
using Distributions
```

```
(x0,μ,σ) = (0.25,0.1,0.2) #parameters of the N() distribution
```

```

x      = range(-0.25,0.75,length=101)
m_range = range(1/52,1/2,step=1/52)      #different horizons

anim = @animate for m in m_range          #create an animation, like a loop
    dist = Normal(x_0+m*μ,sqrt(m*σ^2))    #computations
    pdf_m = pdf.(dist,x)
    txt = "m = $(round(m,digits=2))"
    plot(x,pdf_m,                          #create the plot
        ylims = (0,7),
        title = "Pdf of x",
        xlabel = "x",
        annotation = (-0.1,6.5,text(txt,8)),
        legend = false)
    vline!([x_0+m*μ],linecolor=:black,line=(:dot,1))
end

gif(anim; fps=7,show_msg=false)           #show the animation

```

```
Plots.AnimatedGif("c:\\Users\\PSoderlind\\PSDataE\\JL\\JuliaTutorial\\tmp.gif")
```

Plot Themes (extra)*

...to give the entire plot another style.

See [the Plots documentation on colors](#) for instructions on how to change colours, and [PlotsThemes](#) for changing the overall plot theme (to dark, say).

```

using PlotsThemes
theme(:lime)

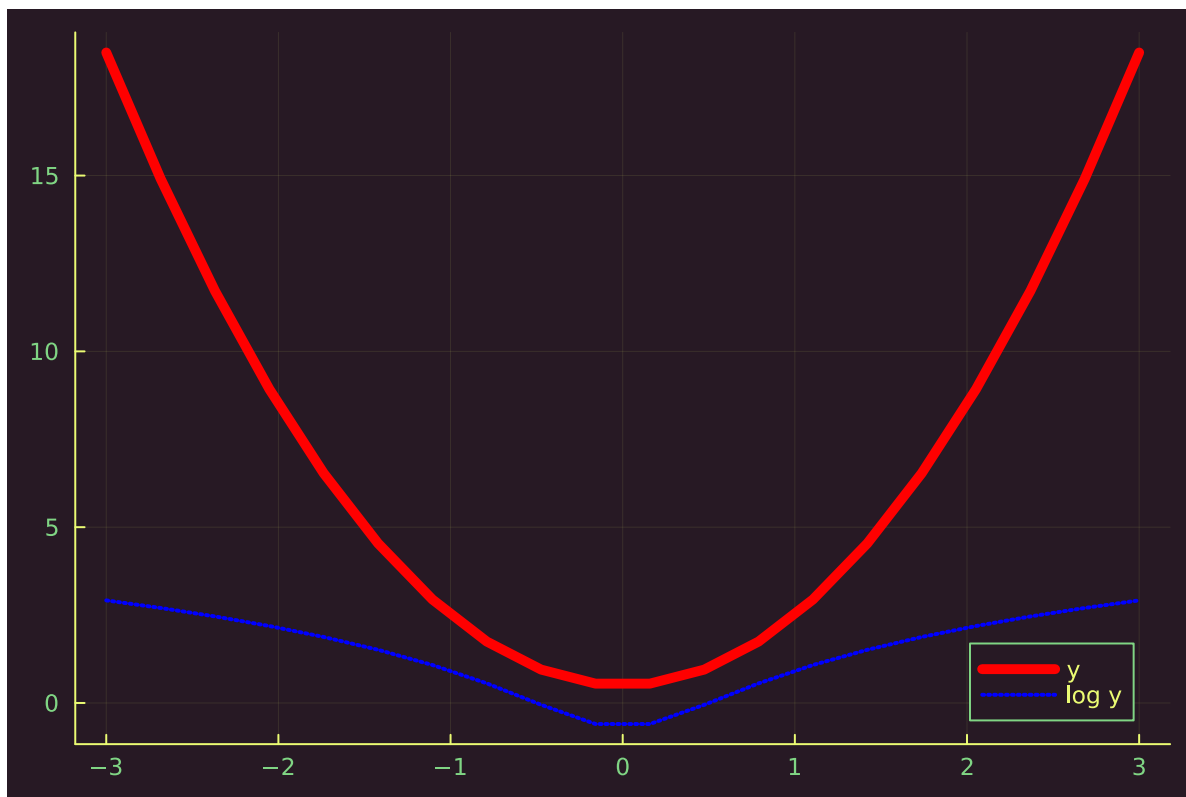
```

```

x = range(-3,3,length=20)      #something to plot
y = 2*x.^2 .+ 0.5

p1 = plot( [x x],[y log.(y)],
    label = ["y" "log y"],
    linecolor = [:red :blue],
    linestyle = [:solid :dot],
    linewidth = [5 2] )
display(p1)

```



Arrays

This notebook illustrates how to create and reshuffle arrays. Other notebooks focus on matrix algebra and functions applied to arrays.

Load Packages and Extra Functions

```
using Printf, DelimitedFiles  
  
include("src/printmat.jl");
```

Scalars, Vectors and Multi-dimensional Arrays

are treated as different things in Julia, even if they happen to “look” similar. For instance, a 1×1 array is not a scalar and an $n \times 1$ array is not a vector.

However, we first present some common features of all arrays (vectors or multi-dimensional arrays).

Creating Arrays

The typical ways of getting an array are

- hard coding the contents
- reading in data from a file
- as a result from computations
- allocating the array and then changing the elements
- (often not so smart) growing the array by adding rows (or columns,...)
- by list comprehension

The next few cells give simple examples.

1. Hard Coding the Contents or Reading from a File

```
z = [11 12;                                     #typing in your matrix
     21 22]
printblue("A matrix that we typed in:")
printmat(z)

x = readlm("Data/MyData.csv",'',skipstart=1) #read matrix from file
printblue("First four lines of x from csv file:")
printmat(x[1:4,:])

#to create a vector: [1,2] or [1;2]
#to create a 2x3x2 array: [1 2 3;4 5 6;;;11 12 13;14 15 16]
```

A matrix that we typed in:

```
11      12
21      22
```

First four lines of x from csv file:

```
197901.000    4.180    0.770    10.960
197902.000   -3.410    0.730    -2.090
197903.000    5.750    0.810    11.710
197904.000    0.050    0.800     3.270
```

2a. Allocating an Array and Then Changing the Elements: Fill

An easy way to create an array is to use the `fill()` function.

```
A = fill(0,10,2)          #10x2 matrix, integers (0)
B = fill(0.0,10)          #vector with 10 elements, floats (0.0)
C = fill(NaN,10,1)        #10x1 matrix, floats (NaN), not a vector
D = fill("",3)            #vector with 3 elements, strings ("")
E = fill(Date(1),3)       #vector with 3 elements, dates (0001-01-01)
```

In contrast, do *not* use `fill([1,2],7)`, since all 7 arrays will refer to the same underlying array (changing one changes all). Instead, use a comprehension (see below).

```

x = fill(0.0,3,2)      #creates a 3x2 matrix filled with 0.0
printblue("so far, x is filled with 0.0. For instance, x[1,1] is $(x[1,1])")

for i in 1:size(x,1), j in 1:size(x,2)    #loop over rows and columns
    x[i,j] = i/j
end

printblue("\nx after some computations:")
printmat(x)

```

so far, x is filled with 0.0. For instance, x[1,1] is 0.0

x after some computations:

```

1.000    0.500
2.000    1.000
3.000    1.500

```

2b. Allocating an Array and Then Changing the Elements: A More General Approach (extra)*

You can also create an array by

```

A = Array{Int}(undef,10,2)      #10x2 matrix, integers
F = Array{Any}(undef,3)         #vector with 3 elements, can include anything

```

The undef signals that the matrix is yet not initialized. This is more cumbersome than fill(), but sometimes more flexible.

```

F = Array{Any}(undef,3)
F[1] = [1;2;3;4]      #F[1] contains a vector
F[2] = "Sultans of Swing"  #F[2] a string
F[3] = 1978            #F[3] an integer

printmat(F)

```

```

[1, 2, 3, 4]
Sultans of Swing
1978

```

3a. Growing an Array

Growing an array (vector, matrix or higher-dimensional array) is done by `[A;B]` to stack vertically, `[A;;B]` (which is the same as `[A B]`) to stack horizontally, or `[A;;;B]` which is to stack along dimension 3. Alternatively, use the `vcat`, `hcat` and `cat` functions to do the same things. Still, this is not very fast, so avoid it in a tight loop.

```
A = [11 12;
      21 22]
B = [1 2;
      0 10]

z = [A;B]          #same as vcat(A,B)
printblue("\n","stacking A and B vertically")
printmat(z)

z2 = [A B]         #same as hcat(A,B)
printblue("\n","stacking A and B horizontally")
printmat(z2)
```

stacking A and B vertically

11	12
21	22
1	2
0	10

stacking A and B horizontally

11	12	1	2
21	22	0	10

3b. Growing a Vector

There are special (faster) functions for growing a *vector* (not a matrix):

```
push!(old_vector,new_element_1,new_element_2)    #or pushfirst!()
```

If you instead want to append all elements of a vector, then do

```
append!(old_vector,vector1_to_append,vector2_to_append)    #or prepend!()
```

```

B = Float64[]           #empty vector, to include floats
for i in 1:3
    x_i = i + 10^i
    push!(B,x_i)         #adding an element at the end
end
printblue("a vector with 3 elements:")
printmat(B)

```

a vector with 3 elements:

```

11.000
102.000
1003.000

```

4. List Comprehension and map (extra)*

List comprehension is a simple way of creating an array from repeated calculations. It is similar to the combination of pre-allocation and a “for loop.”

(You can achieve the same thing with map (for instance, by `map(i→collect(1:i),1:3)`)).

```

A = [collect(1:i) for i=1:3]      #this creates a vector of vectors

printblue("A[1] is vector with 1 element, A[2] a vector with 2 elements,...")
printmat(A)

```

A[1] is vector with 1 element, A[2] a vector with 2 elements,...

```

[1]
[1, 2]
[1, 2, 3]

```

Using Parts of a Matrix

The most common way to use parts of an array is by indexing. For instance, to use the second column of A, do `A[:,2]`.

Notice that `x = A[1,:]` gives a (column) vector (yes, it does), while `z = A[1:1,:]` gives a $1 \times k$ matrix.

Also notice that `z = A[1,:]` creates an independent copy, so changing (elements of) z will *not* change A.

A shortcut to loop over all rows of A is `for i in eachrow(A)`. There is also `eachcol()`.

```
A = [11 12;
     21 22]
printblue("A:")
printmat(A)

printblue("\nsecond column of A:")
printmat(A[:,2])

printblue("\n", "first row of A (as a vector): ")
printmat(A[1,:]) #notice 1 makes it a vector

printblue("\n", "first row of A (as a 1x2 matrix): ")
printmat(A[1:1,:]) #use 1:1 to keep it as a 1x2 matrix

printblue("\n", "looping over `eachrow`: ")
for z in eachrow(A) #looping over all rows
    printmat(z)
end
```

A:

11	12
21	22

second column of A:

12
22

first row of A (as a vector):

11
12

first row of A (as a 1x2 matrix):

11	12
----	----

looping over `eachrow`:

11

12

21

22

Arrays are Different...

Vectors and matrices (arrays) can take lots of memory space, so **Julia is designed to avoid unnecessary copies of arrays**. This means that if A and B are names of the same underlying array, then changing an element of B changes also A. Details are given in the next few cells.

Examples of “Arrays are Different” (extra)*

For instance, notice the following. Suppose you do one of the following computations:

1. $B = A$, $B = \text{reshape}(A, n, m)$, $B = \text{vec}(A)$, or $B = A'$, followed by $B[1] = -999$
2. $B = f1!(A)$ where $f1!$ is a function like

```
function f1!(B)
    B[1] = -999    #change some elements of B inside the function
    return B
end
```

3. $B = [A, A]$ (an array of arrays), followed by $B[1][1] = -999$

In each case, *also A will change*.

Notice that in all cases you are changing some *elements* of B, not redefining the entire B (like in $B = [1, 2, 3]$). Other ways to change some *elements* are $B[:] = [1, 2]$ or $B .= [1, 2]$ so the same behaviour applies to those cases.

If you do not like this behaviour, then use $\text{copy}(A)$ to create an independent copy of the array.

```
function f1!(B)                #! is a convention for indicating that the function changes the input
    B[1] = -999                #changing ELEMENTS of B, affects outside value
    #B = B/2                   #this would NOT affect the outside value
    return B
end

A = [1.0, 2.0]
printblue("original A:")
```

```
printmat(A)

B = f1!(A)
printblue("A after calling f1!(A): ")
printmat(A)
```

original A:
 1.000
 2.000

A after calling f1!(A):
 -999.000
 2.000

Performance Tip: Use view Instead of Creating New Arrays (extra)*

In case you do not need an independent copy, then `y = view(A,1,:)` creates a *view* of the first row of A. This saves memory and is sometimes faster.

To make a *copy* or a *view*? If you need to save memory: a view. Instead, if you need speed: try both. (Copies are often quicker when you need to do lots of computations on the matrix, for instance, in a linear regression.)

```
A = [11 12;
     21 22]

printblue("\n","view of first row of A (although it prints like a column vector): ")
y = view(A,1,:)
printmat(y)
```

view of first row of A (although it prints like a column vector):
 11
 12

Performance Tip: Avoid Creating and Destroying Lots of Arrays (extra)*

Avoid creating and destroying lots of arrays in loops: it takes time. If possible re-use the existing arrays instead. The next cell provides an illustration.

```
function fn1(N)
    for i in 1:N
        tmp = zeros(N,N)           #create a new tmp in each loop
        tmp[i,i] = i              #do something with tmp
    end
    return nothing
end

function fn2(N)
    tmp = zeros(N,N)              #create tmp only once
    for i in 1:N
        tmp .= 0.0                #re-use the existing tmp, reset to zeros
        tmp[i,i] = i              #do something with tmp
    end
    return nothing
end

using BenchmarkTools              #a package for benchmarking computations
@btime fn1(300)                   #timing
@btime fn2(300)

printred("fn2 is much faster")
```

```
9.630 ms (900 allocations: 206.02 MiB)
2.815 ms (3 allocations: 703.21 KiB)
fn2 is much faster
```

Splitting up an Array (extra)*

Sometimes you want to create new variables from the columns (or rows) of a matrix. The next cell shows an example.

```

A = [11 12;
     21 22]

printblue("A simple way...which works well when you want to create a few variables")
x1 = A[:,1]
x2 = A[:,2]      #or (x1,x2) = (A[:,1],A[:,2])
printmat(x2)

printblue("Another way")
(z1,z2) = [A[:,i] for i in 1:2]
printmat(z2)

```

A simple way...which works well when you want to create a few variables

```

12
22

```

Another way

```

12
22

```

An Array of Arrays (extra)*

If x_1 and x_2 are two arrays, then $y=[x_1, x_2]$ is a vector (of arrays) where $y[1] = x_1$ and $y[2] = x_2$.

In this case $y[1]$ is actually a view of x_1 so changing elements of one changes the other.

If you instead want to stack x_1 and x_2 into a single matrix, use $[x_1 \ x_2]$, $[x_1; x_2]$ or one of the `cat` functions discussed above.

```

x1 = ones(3,2)
x2 = [1,2,3]
y = [x1,x2]      #a vector of arrays

for i in 1:length(y)
    printblue("y[$i]:")
    printmat(y[i])
end

```

```

y[1]:
1.000    1.000
1.000    1.000

```

1.000 1.000

y[2]:

1

2

3

Basic Matrix Algebra

See Ch23_1_MatrixAlgebra.ipynb in the ModernFinanceTheory repository.

Functions of Arrays

This notebook illustrates how to apply a function to arrays and how to iterate over array elements.

Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl"); #a function for prettier matrix printing
```

Elementwise Functions of Arrays: The Dot (.)

Let X be an array, and a and b be scalars. Then, $y = \text{fn.}(X, a, b)$ generates an array y where $y[i, j] = \text{fn}(X[i, j], a, b)$

(You would achieve the same thing with $\text{map}(xi \rightarrow \text{fn}(xi, a, b), x)$.)

```
fn(x,a,b) = a/x + b #x has to be a scalar for this to work

X = [1 2;
      0 10]

printmat(fn.(X,100,10)) #notice the dot (.)
```

```
110.000    60.000
   Inf     20.000
```


Looping over Elements of an Array

There are several ways of looping over all elements in an array. The next cell summarises some of them. We cover `for z in x`, `for i in 1:length(x)` and `for i in eachindex(x)`. The `enumerate()` approach is discussed further below.

```
x = [11 12;
     21 22]

printblue("elements of x")
for z in x                #when we do not need the index
    println(z)
end
println()

y = similar(x)
for i in 1:length(x)      #when we want to use the index i for an output
    y[i] = x[i] + i*100    #works only with traditional arrays (first index is 1)
end                        #`for i = 1:length(x)` is the same
printblue("y:")
printmat(y)

y = similar(x)
for i in eachindex(x)     #again when we want to use the index i for an output
    y[i] = x[i] + i*100    #more general and would work with non-traditional arrays
end
printblue("y calculated in another way:")
printmat(y)
```

elements of x

```
11
21
12
22
```

y:

```
111    312
221    422
```

y calculated in another way:

```
111    312
221    422
```

Looping over Columns or Rows

Suppose you want to calculate the sum of every row of a matrix, giving a single value for each row. This can clearly be done by `sum(x,dims=2)`, but here we consider more iterative approaches in order to illustrate general ideas.

The classical way of doing that is to extract row `i` as `X[i,:]` (or `view(X,i,:)`) and `sum`, and repeat for each row (loop over rows).

But, there is also a direct way to loop over all rows by using `for r in eachrow(X)`. To also get the row number `i`, we use `for (i,row) in enumerate(eachrow(X))`. For looping over columns, use `eachcol()`.

```
X = [1 2;
      0 10]
printblue("X:")
printmat(X)

m = size(X,1)

z = zeros(Int,m)           #to fill with results, zeros(Int,m) to get Int
for i in 1:m               #traditional loop over rows
    z[i] = sum(X[i,:])     #or sum(view(X,i,:)) to save memory
end
printblue("sum of each row:")
printmat(z)

z = zeros(Int,m)
for (i,row) in enumerate(eachrow(X)) #enumerate to get both index and value
    z[i] = sum(row)
end
printblue("sum of each row:")
printmat(z)
```

X:

1	2
0	10

sum of each row:

3
10

sum of each row:

3
10

Functions with Built-in dims and Predicates Arguments

Several functions have a `dims` argument that allows you to avoid looping, for instance, `sum`.

In other cases, the functions allow you to also apply an elementwise function to `X` before doing the rest of the calculations, for instance, `any`, `all`, `sum`, `prod`, `maximum`, and `minimum`. This speeds up things since it avoids creating intermediate/temporary arrays.

```
printmat(sum(X,dims=2))  
  
printmat(sum(abs2,X,dims=2))           #same as sum(abs2.(X),dims=2) but faster
```

3
10

5
100

Apply Your Own Function on Each Column: `mapslices (extra)*`

...or each row (or some other dimension).

You can always loop, but there are alternatives. The `mapslices(fun,x,dims=1)` applies `fun(x[:,i])` to each column of a matrix `x`. This is a flexible alternative to looping over the columns, since it easily allows you to change the dimension over which too loop.

The cell below illustrates this by calling a function which calculates the moving average of `x[t]` and `x[t-1]` for each column (or row) of a matrix `X`.

```
function MovingAvg2(x)           #moving average of t and t-1  
    T = length(x)  
    y = fill(NaN,T)  
    for t in 2:T  
        y[t] = (x[t] + x[t-1])/2  
    end  
    return y  
end
```

```

X = [1:5 101:105]
Y = mapslices(MovingAvg2,X,dims=1)          #could change to dims=2
printblue("Y and MA(1) of X:")
printmat([X Y],colNames=["x1", "x2", "MA(x1)", "MA(x2)"])

```

Y and MA(1) of X:

x ₁	x ₂	MA(x ₁)	MA(x ₂)
1.000	101.000	NaN	NaN
2.000	102.000	1.500	101.500
3.000	103.000	2.500	102.500
4.000	104.000	3.500	103.500
5.000	105.000	4.500	104.500

Types

This notebook provides (a) a brief introduction to types (for instance, integers, bools and strings), (b) a discussion of how to test types and (c) also how to convert from one type to another.

Load Packages and Extra Functions

```
using Printf  
  
include("src/printmat.jl");
```

Quick Start

...demonstrating some key aspects.

```
println("Int + Float64: ",1+2.0,"\n")  
  
println("Convert 3.0 to Int: ",convert{Int,3.0},"\n")  
  
x = 3  
println("test if $x is an Int: ",x isa Int)
```

Int + Float64: 3.0

Convert 3.0 to Int: 3

test if 3 is an Int: true

Some Important Types

Julia has many different types of variables: signed integers (like 2 or -5), floating point numbers (2.0 and -5.1), bools (false/true), bitarrays (similar to bools, but with more efficient use of memory), strings (“hello”), Dates (2017-04-23) and many more.

The numerical types also come with subtypes for different precisions, for instance, Float16, Float32 and Float64. Unless you specify otherwise, code like

```
a = 2
b = 2.0
```

gives an Int64 and a Float64 respectively (at least on the 64 bit version of Julia).

Integers and Floats

```
a = 2          #integer, Int (Int64 on most machines)
b = 2.0        #floating point, (Float64 on most machines)
A = [1,2]
B = [1.0,2.0]

println("a: ",typeof(a))
println(a)

println("\nb: ",typeof(b))
println(b)

println("\nA: ",typeof(A))
printmat(A)

println("B: ",typeof(B))
printmat(B)
```

```
a: Int64
2
```

```
b: Float64
2.0
```

```
A: Vector{Int64}
```

```
1
2
```

```
B: Vector{Float64}
 1.000
 2.000
```

Why Use Int When There Are Floats?

That is, why bother with sometimes using 3 when you could use 3.0 everywhere? Mostly because you cannot. For instance, you cannot pick out element `x[3.0]` from a vector. It has to be `x[3]`.

(In contrast, `1.0 + 2.0` will be exactly `3.0` in Julia, without a floating point rounding error, so that is not a main concern. For instance, try `1.0 + 2.0 == 3`)

```
x = [1,10,100,1000]

#println(x[3.0])    #uncomment and run. Will give an error

println(x[3])
```

```
100
```

Bools and BitArrays

```
c = 2 > 1.1                #Bool
println("c: ",typeof(c))
println(c)

C = A .> 1.5                #BitArray since A is an array
println("\nC: ",typeof(C))
printmat(C)

println("A BitArray is a more economical array version of Bool, but prints as 0/1 by printmat.\n",
        "Notice that typeof(C[1]) gives: ",typeof(C[1]))
```

```
c: Bool
true
```

```
C: BitVector
  0
  1
```

A BitArray is a more economical array version of Bool, but prints as 0/1 by printmat. Notice that `typeof(C[1])` gives: Bool

Char and Strings

```
t = 'a'                                #Char, just one letter
println(typeof(t))

txt = "Dogs are nicer than cats."      #String, could be a long novel
println(typeof(txt))
```

```
Char
String
```

Calculations with Mixed Types and Converting Types

A calculation like “integer” + “float” works automatically. The type of the result is a float (the more flexible type). Similarly, “Bool” + “Int” will give an integer.

There are also direct ways of converting a variable from one type to another using the `convert()` function.

Some Calculations with Mixed Types (“promotion”)

```
println("Int + Float64: ", 1+2.0)
println("Bool + Int: ", (1 > 0) + 2)
```

```
Int + Float64: 3.0
Bool + Int: 3
```


Converting from Int to Float and Vice Versa

by `convert(Int,x)` or `convert(Float64,y)`.

Notice that conversion to Int works if `x=1.0`, but not if `x=1.1`. In the latter case, `round(Int,x)` can be used.

```
x = [1.0;10]
println("x: ",typeof(x))
printmat(x)

x_to_Int = convert.(Int,x)           #Float64 → Int, Int(x) also works
println("converting x to Int: ",typeof(x_to_Int))
printmat(x_to_Int)
```

```
x: Vector{Float64}
 1.000
10.000
```

```
converting x to Int: Vector{Int64}
 1
10
```

Converting from Booleans and BitArrays to Int and Vice Versa

```
C = A .> 1.5
C_to_Int = convert.(Int,C)           #BitArray → Int, Int.(C) also works
println(typeof(C_to_Int))
printmat(C_to_Int)

D = [1;0]
D_to_Boolean = convert.(Boolean,D)   #Int → BitArray, Boolean.(D) also works
println(typeof(D_to_Boolean))
printmat(D_to_Boolean)
```

```
Vector{Int64}
 0
 1
```

```
BitVector
```

1
0

From Booleans and BitArrays to Int: A Tricky Case (extra)

false is a “strong zero” in the sense that `false*NaN == 0` and `false*Inf == 0`.

If you do not want that behaviour in your code, transform false to 0 and then multiply.

```
println(false*NaN)
println(false*Inf)

println(convert(Int,false)*NaN)
println(convert(Int,false)*Inf)
```

0.0
0.0
NaN
NaN

Testing the Type

The perhaps easiest way to test the type is by using the `isa(variable,Type)` function, which can also be called as `variable isa Type`. The type can be a union of other types (see below for an example).

Notice that an array has the type `Array`; more specifically `Array{Float64}` if it is an array with `Float64` numbers.

```
x = 1.2
z = [1.2,1.3]

println("$x is a Number: ",isa(x,Number))
println("$x is an Int: ",x isa Int)
println("$x is an Int or a Float64: ",isa(x,Union{Int,Float64}))

println("$z is a Float64: ",isa(z,Float64))
println("$z is an Array: ",isa(z,Array))
```

```
1.2 is a Number: true
1.2 is an Int: false
1.2 is an Int or a Float64: true
[1.2, 1.3] is a Float64: false
[1.2, 1.3] is an Array: true
```

Performance Tips (extra)

Your code will often run faster if your variables do not change type in the computations (that is, if they are “type stable”). For instance, do something like this

```
x = 0.0                #better than using x = 0 here
x = x + 0.1
```

If you want to initialise an array of zeros with the same type and size as `z`, use `zero(z)`. This is particularly useful inside function where the input `z` might sometimes contain floats and other times integers, etc. To get another size than `z`, do eg. `zeros(elttype(z),3,2)`. There are corresponding `one()` and `ones()` functions. Instead, if you just need a array of the same type as `z`, use `similar(z)`. (It will be filled with garbage.)

Days and Dates

This notebook (a) introduces the Date type; (b) discusses how to do date arithmetics; (c) and how to convert to/from other date formats (for instance, from matlab).

Load Packages and Extra Functions

```
using Printf, Dates  
  
include("src/printmat.jl");
```

Building a Calendar

```
dNb = Date(2014,1,31):Month(1):Date(2014,7,31)    #build a monthly calendar  
  
printmat(dNb;colNames=["date"],width=13)
```

```
      date  
2014-01-31  
2014-02-28  
2014-03-31  
2014-04-30  
2014-05-31  
2014-06-30  
2014-07-31
```

Time Arithmetics

You can add and subtract Dates from each other.

```
d1 = Date(2016,3,31)
d2 = Date(2016,4,30)

dif      = d2 - d1                                #count the number of days between d2 and d1
difRel = Dates.value(dif)/daysinyear(d1)          #Dates.value() is the datenumber, needs prefix Dates

println("difference between two dates: ",dif)
printlnPs("as a fraction of the year (approx): ",difRel)
```

```
difference between two dates: 30 days
as a fraction of the year (approx):      0.082
```

```
d3 = d1 + Month(1)                                #one month after d1

println("d1 and one month later: ",d1," ",d3)
```

```
d1 and one month later: 2016-03-31 2016-04-30
```

Looking up Day of the Week and More

```
println("day of the week of date: ",d1," ",dayofweek(d1))
println("day of the year of date: ",d1," ",dayofyear(d1))

(y,m,d)= yearmonthday(d1)                        #splitting up a date
println("\nSplitting up a date: ",y," ",m," ",d)
```

```
day of the week of date: 2016-03-31 4
day of the year of date: 2016-03-31 91
```

```
Splitting up a date: 2016 3 31
```

Printing a Date

with your own formatting (see the manual for many other formatting options)

```
println(Dates.format(d1,"d u yyyy"))           #needs prefix Dates
println(Dates.format(d1,"dd-mm-yyyy"))
```

```
31 Mar 2016
31-03-2016
```

Converting from/to Other Date Formats

Converting from/to Day Number

`datetime2rata()` creates a Julia day *number* from a Julia Date. (You can also use `Dates.value()` for that.) It counts the number of days since 0000-12-31. Day 1 is thus 0001-01-01. This could, for instance, be useful for communicating with other software.

The inverse is `rata2datetime()`, which creates `DateTime()` from a Julia day number. To drop the time part, apply the `Date()` function on the result.

```
d          = Date(2016,3,31)
daynumber  = datetime2rata(d)           #day number from date, or Dates.value()
d2         = Date(rata2datetime(daynumber)) #date from day number

printmat(d,daynumber,d2;colNames=["Original date","Julia day no","new date"],width=13)
```

```
Original date Julia day no    new date
2016-03-31    736054         2016-03-31
```

Converting from `yyyymmdd`

Downloaded financial data (eg. from `finance.yahoo`) often have dates like 20160331. The next cell shows how to create a Julia Date.

```
csvdate = 20160331
jldate  = Date(string(csvdate),"yyyymmdd") #convert to string and then Julia Date
csvdate2 = Dates.format(d1,"yyyymmdd")    #Julia date to csv format

printmat(csvdate,jldate,csvdate2;colNames=["Original date","Julia date","new date"],width=13)
```

Original date	Julia date	new date
20160331	2016-03-31	20160331

Converting from/to Matlab's datenum()

Matlab's `datenum()` also counts days since a starting date, but day 1 is here 0000-1-1, which is 366 days before day 1 in Julia. Conversion therefore requires subtracting/adding 366. In addition, mat files typically save the date number as a floating point number.

The conversion uses the approach for day numbers discussed above and is coded up in two functions in the next cell.

```
mlNum2jlDate(mlnum) = Date(rata2datetime(round(Int,mlnum) - 366))    #to Julia Date from matlab dat
jlDate2mlNum(jldate) = datetime2rata(jldate) + 366.0
```

`jlDate2mlNum` (generic function with 1 method)

```
mlnum = 736420.0          #to be converted, 2016-03-31
jldate = mlNum2jlDate(mlnum)
mlnum2 = jlDate2mlNum(jldate)

printmat([mlnum jldate mlnum2];colNames=["matlab","Julia","matlab 2"],width=13)
```

matlab	Julia	matlab 2
736420.000	2016-03-31	736420.000

Loading and Saving Data

to/from csv, jld2, xlsx, hdf5, and mat files.

The focus is on typical financial data sets, with dates and data. This mixture of types often needs conversion into Julia dates and integers/floating point numbers. In case your data set is simpler, skip those parts of the code.

There are also packages for reading R data files ([RData.jl](#)), numpy data files ([NPZ.jl](#)), [JSON.jl](#), [Arrow.jl](#), and more. Alternatively, use `RCall.jl`/`PythonCall.jl` to read such data. This is not covered in this notebook.

Load Packages and Extra Functions

The packages are loaded in the respective sections below. This allows you to run parts of this notebook without having to install all packages.

The data files created by this notebook are written to and loaded from the subfolder `Results`.

```
using Printf, Dates
include("src/printmat.jl")

if !isdir("Results")
    error("create the subfolder Results before running this program, perhaps with mkdir()")
end
```

csv Files

csv: Loading

The csv (“comma-separated values”) format provides a simple and robust method for moving data and it can be read by most software.

For *reading* a data file delimited by comma (,) and where the first line of the file contains variable names, use the following

```
(x,header) = readdlm(FileName,',',header=true)
```

Alternatively, use

```
x = readdlm(FileName,',',skipstart=1)
```

to disregard the first line. Extra arguments control the type of data (Float64, Int, etc), suppression of comment lines, and more.

When reading a file with mixed data types, you typically get data of the Any type, so an explicit conversion might be useful.

If you need more powerful write/read routines, try the [CSV.jl](#) package.

```
using DelimitedFiles

(x,header) = readdlm("Data/CsvFile.csv",',',header=true) #read csv file
printblue("x is: ",summary(x),"\\n")
printmat(x;colNames=vec(header)) #printmat() wants a vector of column names
printblue("There are mixed types and missings in this matrix. Maybe convert?")
```

```
x is: 6×3 Matrix{Any}
```

	GSPC	DTB3
02/01/1979	96.730	
03/01/1979		9.310
04/01/1979	98.580	9.310
05/01/1979	99.130	9.340
08/01/1979	98.800	9.360
09/01/1979	99.330	9.270

```
There are mixed types and missings in this matrix. Maybe convert?
```

csv: Convert Dates and Missing Values (extra)*

The function `datesFix()` below converts dates to Julia dates, while the function `readdlmFix()` replaces all non-numbers by NaN (use with `Float64`) or optionally missing (use when data is not `Float64`) and finally converts the whole matrix to `Float64` (or a type you decide).

Most financial data is floating point, so we typically choose to convert to NaN rather than to missing as the former is much more easily handled in many computations.

```
"""
    datesFix(z,fmt="dd/mm/yyyy")

Convert to Julia dates, or `Time()` and `DateTime()`
"""
datesFix(z,fmt="dd/mm/yyyy") = Date.(string.(z),fmt)

"""
    readdlmFix(x,Typ=Float64,missVal=NaN)

Change elements with missing data (' ') to either NaN or missing.
`x` is the input matrix, `Typ` is the type of the output (Float64, Int, etc) and
`missval` is your choice of either `NaN` or `missing`
"""
function readdlmFix(x,Typ=Float64,missVal=NaN)
    y = replace(z→!isa(z,Number) ? missVal : z,x)
    ismissing(missVal) && (Typ = Union{Missing,Typ}) #allow missing
    y = convert.(Typ,y)
    return y
end;

dN = datesFix(x[:,1],"dd/mm/yyyy") #to proper Julia Date(s)
x2 = readdlmFix(x[:,2:end])        #convert to Float64 and replace empty by NaN

println("after datesFix() and readdlmFix():\n")
printmat(dN,x2;colNames=vec(header)) #printmat() wants a vector of column names
```

after `datesFix()` and `readdlmFix()`:

	GSPC	DTB3
1979-01-02	96.730	NaN
1979-01-03	NaN	9.310

1979-01-04	98.580	9.310
1979-01-05	99.130	9.340
1979-01-08	98.800	9.360
1979-01-09	99.330	9.270

csv: Saving

To *write* csv data, the simplest approach is to create the matrix you want to save and then run

```
writedlm(FileName,matrix)
```

Alternatively, to write several matrices to the file (stacked vertically), use

```
open(Filename, "w") do fh writedlm(fh,matrix1,','') writedlm(fh,matrix2,','') end
```

For instance, `'matrix1'` might be a row vector with variable names (the header) and `'matrix2'` the data.

Another syntax which does the same thing is

```
fh = open(Filename, "w") writedlm(fh,matrix1,','') writedlm(fh,matrix2,','') close(fh)
```

If you only need/want limited precision in the file, round the numbers first, for instance, `'round.(matrix1, 2)'`

```
::: {.cell tags='[]' execution_count=5}
``` {.julia .cell-code}
x = [Date(1990) 1.1 1.2 1.3;
 Date(1991) 2.1 2.2 2.3]
header = [" " "X" "Y" "Z"]
```

```
open("Results/NewCsvFile.csv","w") do fh #write csv file
 writedlm(fh,header,",") #the header
 writedlm(fh,x,",") #the actual data
end
```

```
printblue("NewCsvFile.csv has been created in the subfolder Results. Check it out.")
```

NewCsvFile.csv has been created in the subfolder Results. Check it out.

...

## jld2

### jld2: loading

jld2 files can store very different types of data: dates, integers, floats, strings, dictionaries, etc. It is a dialect of hdf5, designed to save different Julia objects. This avoids most type conversions.

The basic syntax of the [JLD2.jl](#) package is

```
(A,B) = load(FileName,"A","B") #load some data
xx = load(FileName) #load all data into a Dict()
save(FileName,"Aaaa",A,"B",B) #save data
jldsave(FileName;Aaaa=A,B) #also to save data, notice the semicolon (;)
```

(It also possible to use the same syntax as for HDF5 below, except that we use `jldopen` instead of `h5open`.)

```
using FileIO, JLD2

xx = load("Data/JldFile.jld2") #load entire file
println("The variables are: ",keys(xx),"\n") #list contents of the file
(dN,x) = (xx["d"],xx["x"])

println("loaded from jld2 file:\n")
printmat([dN x])

#(x,dN) = load("Data/JldFile.jld2","x","d"); #alternative way to read some of the data
```

The variables are: ["B", "C", "x", "d"]

loaded from jld2 file:

2019-05-14	1.100	1.200	1.300
2019-05-15	2.100	2.200	2.300

## jld2: saving

```
x = [1.1 1.2 1.3; #to save in a JLD2 file
 2.1 2.2 2.3]
d = [Date(2019,5,14); #Julia dates
 Date(2019,5,15)]
B = 1
C = "Nice cat"

save("Results/NewJldFile.jld2","xxxx",x,"d",d,"B",B,"C",C) #write jld2 file
#jldsave("Results/NewJldFile.jld2";xxxx=x,d,B,C) #alternative way, notice the ;

println("NewJldFile.jld2 has been created in the subfolder Results")
```

NewJldFile.jld2 has been created in the subfolder Results

## xlsx

### xlsx: Loading

The [XLSX.jl](#) package allows you to read and write xlsx files.

When reading from a sheet with mixed data types, you again get data of the Any type, which needs conversion (see the cell further below). Otherwise, try the `XLSX.gettable()` method.

```
using XLSX

data1 = XLSX.readxlsx("Data/xlsxFile.xlsx") #reading the entire file
x = data1["Data!A2:C5"] #extracting a part of the sheet "Data"
println("x is: ",summary(x),"\n")
#x = XLSX.readdata("Data/xlsxFile.xlsx","Data","A2:C5") #does the same thing

println("part of the xlsx file:")
printmat(x)

#xx1 = XLSX.gettable(data1["Data"],infer_eltypes=true) #to get types automatically
#(a1,a2,a3) = xx1.data #extract "columns" from xx1
#printmat(a1,a2,a3)
```

```
x is: 4×3 Matrix{Any}
```

part of the xlsx file:

1950-01-06	16.980	-999.990
1950-01-09	17.080	-999.990
1950-01-10	17.030	7
1950-01-11	17.090	8

### xlsx: Convert Dates and Missing Values (extra)\*

using the same functions as for csv. (See above. You need to run that cell once before using the functions.)

Noice that the xlsx file here uses -999.99 to indicate a missing value. This approach is fairly common approach. For that reason we replace those with NaN.

```
dN = datesFix(x[:,1], "yyyy-mm-dd") #to proper Julia Date(s)
x2 = readlmFix(x[:,2:end]) #convert to Float64 and replace empty by NaN
replace!(x2, -999.99 ⇒ NaN) #replace -999.99 by NaN

printmat(dN, x2)
```

1950-01-06	16.980	NaN
1950-01-09	17.080	NaN
1950-01-10	17.030	7.000
1950-01-11	17.090	8.000

### xlsx: Saving

openxlsx() will automatically create a sheet. Below, we rename it to “FirstSheet” and then populate it. A second sheet is created by XLSX.addsheet!().

```
x = [Date(1980) 1.1 1.2 1.3; #writing a matrix to an xlsx file
 Date(1981) 2.1 2.2 2.3]
y = [11 12]

XLSX.openxlsx("Results/NewxlsxFile.xlsx", mode="w") do xf
 XLSX.rename!(xf[1], "FirstSheet")
 xf["FirstSheet"]["C2"] = x #write to 1st sheet, matrix with upper left corner i
 XLSX.addsheet!(xf, "SecondSheet") #create 2nd sheet
 xf[2]["A1"] = y #write to 2nd sheet
```

```
end

printblue("NewxlsxFile.xlsx has been created in the subfolder Results")
```

NewxlsxFile.xlsx has been created in the subfolder Results

## hdf5 (extra)\*

### hdf5: loading

hdf5 files are used in many computer languages. They can store different types of data: integers, floats, strings, but *not* Julia Dates.

The basic syntax of the [HDF5.jl](#) package for reading is

```
h5open(fileName,"r") do fh #open for reading
 (x,y) = read(fh,"x","y")
end
```

Dates can, for instance, be saved in a Tx3 matrix (year, month day), and then converted to Julia dates by the `Date()` function.

(There is also the `h5read(filename,"x")` command that reads one variable from a file.)

```
using HDF5

h5open("Data/H5File.h5","r") do fh #open for reading
 println("\nVariables in h5 file: ",keys(fh)) #list contents of the file
 global x,B,ymd #to make sure they are exported to workspace
 (x,B,ymd) = read(fh,"x","B","ymd") #load some of the data
end

dN = Date.(ymd[:,1],ymd[:,2],ymd[:,3]) #reconstructing dates from a 3-column matrix

println("\ndates and x from h5 file is")
printmat([dN x])
```

Variables in h5 file: ["B", "C", "x", "ymd"]

dates and x from h5 file is

2019-05-14	1.100	1.200	1.300
2019-05-15	2.100	2.200	2.300

## hdf5: saving

To save dates, save either a matrix [y m d] or a date value from `Dates.value(date)`. See the tutorial on Dates for more details.

To save data, the syntax is

```
h5open(FileName,"w") do fh #open for writing
 write(fh,"x",x,"y",y)
end
```

(There is also the `h5write(filename,"x",x)` command that writes one variable to a file.)

```
x = [1.1 1.2 1.3;
 2.1 2.2 2.3]
ymd = [2019 5 14;
 2019 5 15]
B = 1
C = "Nice cat"

h5open("Results/NewH5File.h5","w") do fh #open file for writing
 write(fh,"x",x,"ymd",ymd,"B",B,"C",C)
end

printblue("NewH5File.h5 has been created in the subfolder Results")
```

NewH5File.h5 has been created in the subfolder Results

## mat Files (extra)\*

### mat: Loading

The [MAT.jl](#) package allows you to load/save (Matlab) mat files, which is a dialect of HDF5.



`matread()` reads in a file to a dictionary, while `matwrite()` writes a dictionary. (There is also a `matopen()` function that can be used with `write()` similar to the HDF5 approach discussed above.)

```
using MAT

D = matread("Data/MatFile.mat") #gives a Dict
println("variables in mat file: ",keys(D),"\n")

(dM,x) = (D["dM"],D["x"]) #extract some variables from Dict

printmat(dM,x)

println("The dates are from matlab's datenum(). Maybe convert?")
```

variables in mat file: ["B", "C", "dM", "x"]

737559	1.100	1.200	1.300
737560	2.100	2.200	2.300

The dates are from matlab's `datenum()`. Maybe convert?

## mat: Converting Dates

The next cell reuses function from the tutorial on Dates, in order to convert between matlab and Julia dates.

```
mlNum2jlDate(mlnum) = Date(rata2datetime(round(Int,mlnum) - 366)) #to Julia Date from matlab da
jlDate2mlNum(jldate) = datetime2rata(jldate) + 366.0;
```

```
d = mlNum2jlDate.(dM) #Matlab datenum to Julia date

println("\ndates from mat file is (before and after conversion)")
printmat(dM,d)
```

dates from mat file is (before and after conversion)

737559	2019-05-14
737560	2019-05-15

## mat: Writing

```
x = [1.1 1.2 1.3;
 2.1 2.2 2.3]
d = [Date(2019,5,14); #Julia dates
 Date(2019,5,15)]
B = 1
C = "Nice cat"

dM = jlDate2mNum.(d) #Julia Date to Matlab's datenum(), Float64

D = Dict{"x"⇒x,"B"⇒B,"dM"⇒dM,"C"⇒C}
matwrite("Results/NewMatFile.mat",D)

printblue("\nNewMatFile.mat has been created in the subfolder Results")
```

NewMatFile.mat has been created in the subfolder Results

# Strings

This notebook demonstrates some basic string commands.

## Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl");
```

## Quick Summary

of the (perhaps) most important commands.

```
x = 7
str1 = "Hello,\n" #\n is newline
str2 = "I am robot $x" #string interpolation
str3 = string(str1,str2) #combine into one string
println(str3)
```

```
Hello,
I am robot 7
```

## String Basics

The next few cells show how to test, replace, split and sort strings.

```

str4 = "Highway 62 Revisited"

if occursin("Highway",str4) #test if the string contains a substring
 println(str4)
 printblue("contains the word Highway")
end

str4 = replace(str4,"62" => "61") #replace part of a string
printblue("\nNew, better string after a replacement: ")
println(str4)

```

Highway 62 Revisited  
contains the word Highway

New, better string after a replacement:  
Highway 61 Revisited

```

words = split(str4)
printblue("split a string into a vector of words:")
printmat(words)

printblue("\nand join the words into a single string:")
println(join(words," "))

```

split a string into a vector of words:

Highway
61
Revisited

and join the words into a single string:  
Highway 61 Revisited

```

printblue("sort the words alphabetically:")
printmat(sort(words))

```

sort the words alphabetically:

61
Highway
Revisited

## Reading an Entire File as a String

The next cell reads a file into one single string. It keeps the formatting (spaces, line breaks etc).

```
txtFile = "Data/FileWithText.txt"
str = read(txtFile,String) #read as string
```

```
"Dogs are nicer\nthan cats.\n\n This\n is a\nfairly short file."
```

```
printblue("testing if str is a String: \n") #test if a String

if str isa String
 println(str)
end
```

testing if str is a String:

Dogs are nicer  
than cats.

This  
is a  
fairly short file.

## Reading all Lines of a File into a Vector of Strings

The next cell reads a file into a vector of strings: one string per line of the file. The second cell joins those lines into one string.

```
lines = readlines(txtFile)
printmat(lines)
```

Dogs are nicer  
than cats.

This  
is a  
fairly short file.

```
linesJoined = join(lines,"\n") #join the lines of the array,
println(linesJoined) # "\n" to create line breaks
```

Dogs are nicer  
than cats.

    This  
    is a  
fairly short file.

## Strings and Indexing (extra)\*

can be tricky when the string contains non-ascii characters. To test if it does, use `any(!isascii,strV)`.

Notice that you cannot change a string by indexing. For instance, `str[1] = "D"` does not work. However, you can *read* strings by indexing, if you are careful.

The next cell gives two versions of a string. `str2[2]` does not work since the first character takes more than one byte to store. Julia has `nextind()` to get around this.

```
str1 = "Dx = -0.9x"
str2 = "Δx = -0.9x"

println("Test if any non-ascii:", any(!isascii,str1), " ", any(!isascii,str2),"\n")

println(str1[2]) #works
#println(str2[2]) #uncomment to get an error
```

Test if any non-ascii:false true

x

```
i = 1
for c in str2 #alternatively, while i <= lastindex(str)
 #global i #only needed in script
 println(i," ",c)
 i = nextind(str2,i)
end

printred("\nNotice that index 2 is not listed: Δ takes to several bytes to store")
```

```

1 Δ
3 x
4
5 =
6
7 -
8 0
9 .
10 9
11 x

```

Notice that index 2 is not listed:  $\Delta$  takes to several bytes to store

## Creating a Long String (extra)\*

can be done with `string()`, but it is often quicker to write to an `IOBuffer()`.

Both approaches are demonstrated below by combining a vector of words into a string. This is just meant as an illustration since `join(txt, " ")` would be a better way to do this particular operation.

```

txt = ["The","highway","is","for","gamblers","better","use","your","sense\n", #a vector of words
 "Take","what","you","have","gathered","from","coincidence"];

```

```

BabyBlue1 = "" #an empty string
for i = 1:length(txt)
 #global BabyBlue1 #only needed in script
 BabyBlue1 = string(BabyBlue1," ",txt[i]) #add to the string
end
println(BabyBlue1)

```

The highway is for gamblers, better use your sense  
 Take what you have gathered from coincidence

```

iob = IOBuffer() #an IOBuffer
for i = 1:length(txt)
 write(iob," ",txt[i]) #write to the buffer
end
BabyBlue2 = String(take!(iob)) #convert to a string
println(BabyBlue2)

```

The highway is for gamblers, better use your sense  
Take what you have gathered from coincidence



# Printing

This notebook is focused on how to print (arrays of) numbers with formatting (width, number of digits shown, etc).

## Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl")
include("src/printTeXTable.jl");
```

## Basic Printing

is done with `println()` or `display()`.

```
x = [11.111 12.12; 21 22] #a matrix to print

println(x)
```

```
[11.111 12.12; 21.0 22.0]
```

```
display(x) #gives basic formatting
```

```
2×2 Matrix{Float64}:
 11.111 12.12
 21.0 22.0
```

## printmat() and printTeXTable

My simple function `printmat()` allows basic formatting and `printTeXTable()` prints a simple LaTeX table.

These functions use keyword arguments, for instance, `colNames=["A","B"]`. `printTeXTable()` is somewhat similar, but creates a string that can be used for a LaTeX table.

```
colNames = ["A","B"]
rowNames = ["row 1","row 2"]
printmat(x;colNames,rowNames,width=7,prec=2)
#printmat(x;colNames=colNames,rowNames=rowNames,width=7,prec=2) #this works too
```

	A	B
row 1	11.11	12.12
row 2	21.00	22.00

```
str = printTeXTable(x;colNames,rowNames,width=7,prec=3); #str can be printed to a file, see below
```

```
\begin{table}
 \begin{tabular}{lrr}
 & A & B \\ \hline
 row 1 & 11.111 & 12.120 \\
 row 2 & 21.000 & 22.000 \\
 \hline
\end{tabular}
\end{table}
```

## The PrettyTables.jl Package

The first cell illustrates some basic using for printing as text. The second cell demonstrates output to Markdown and html, while the third cell shows output to LaTeX.

```
using PrettyTables #PrettyTables version 3.x

pretty_table(x;column_labels=colNames,row_labels=rowNames)
println()

table_format = TextTableFormat(;@text__no_vertical_lines, @text__no_horizontal_lines)
pretty_table(x;row_labels=nothing,show_column_labels=false, table_format,formatters = [fmt__printf(
```

	A	B
row 1	11.111	12.12
row 2	21.0	22.0

```
11.11 12.12
21.00 22.00
```

```
pretty_table(x;column_labels=colNames,row_labels=rowNames,backend = :markdown) #PrettyTables vers
pretty_table(x;column_labels=colNames,row_labels=rowNames,backend = :html)
```

```
| | **A** | **B** |
|-----:|-----:|-----:|
| **row 1** | 11.111 | 12.12 |
| **row 2** | 21.0 | 22.0 |
```

	A	B
row 1	11.111	12.12
row 2	21.0	22.0

```
pretty_table(x;column_labels=colNames,row_labels=rowNames,backend = :latex)
```

```
\begin{tabular}{|r|r|r|}
\hline
& \textbf{A} & \textbf{B} \\
\hline
\textbf{row 1} & 11.111 & 12.12 \\
\textbf{row 2} & 21.0 & 22.0 \\
\hline
\end{tabular}
```

## Printing Output to a Text File

is simple. You open() do fh the file, write to it and close by end. The next cells demonstrate this.

```

open("Results/NewTxtFile.txt", "w") do fh #open the file, "w" for writing

 println(fh,"Dogs are ") #printing to the file, notice the fh
 println(fh,"nicer than cats.\n")

 printmat(fh,x,prec=2)
 printmat(fh,x,colNames=colNames,rowNames=rowNames,prec=2)
 printTeXTable(fh,x,colNames=colNames,rowNames=rowNames,prec=2)

 pretty_table(fh,x) #from PrettyTables.jl

end #closes the file

printblue("Results/NewTxtFile.txt has been closed. Check it out. The raw contents are:\n")
println(read("Results/NewTxtFile.txt",String))

```

Results/NewTxtFile.txt has been closed. Check it out. The raw contents are:

Dogs are  
nicer than cats.

```

11.11 12.12
21.00 22.00

```

```

 A B
row 1 11.11 12.12
row 2 21.00 22.00

```

```

\begin{table}
 \begin{tabular}{lrr}
 & A & B \\ \hline
 row 1 & 11.11 & 12.12 \\
 row 2 & 21.00 & 22.00 \\
 \hline
 \end{tabular}
\end{table}

```

Col. 1	Col. 2
11.111	12.12
21.0	22.0

--	--

## Redirecting the Output (extra)\*

You can also *redirect* the output from the screen to a file. This gives an simple mechanism for dumping all outputs to a file without having to insert the IO stream (called fh above) in all print statements.

To do that, use something like the next cell

```
redirect_stdio(stdout="Results/NewTxtFile2.txt") do #redirect the output to a file
 printmat(x,prec=2)
end #end redirection

printblue("Results/NewTxtFile2.txt has been closed. Check it out. The raw contents are:\n")
println(read("Results/NewTxtFile2.txt",String))
```

Results/NewTxtFile2.txt has been closed. Check it out. The raw contents are:

11.11	12.12
21.00	22.00

## Formatted Printing (extra)\*

by using the Printf library.

@printf() prints to a file or to the screen (and IOBuffers etc), while @sprintf() prints to a string. It is straightforward to use a static format string (eg. "%10.2f\n") to decide how the printing of, for instance, numbers should look like.

To print with *dynamic formatting* (that is, where the format is decided at run-time), use Printf.format(). The second cell below illustrates this.

```
@printf("%10.2f\n",pi) #constant format string, width 10, 2 digits, floating point

str = @sprintf("%5d %g",1828,exp(1)) #width 5 and integer, compact floating point
println("my string is just this: ",str)
```

3.14
my string is just this: 1828 2.71828

```
width = 10

for prec in 1:5 #dynamic formatting
 #local fmt,str #only needed in script
 fmt = Printf.Format("%$(width).$(prec)f")
 str = Printf.format(fmt, π)
 println(prec,str)
end
```

```
1 3.1
2 3.14
3 3.142
4 3.1416
5 3.14159
```

# Missing Values

are common in statistical data. This notebook gives an introduction to how to identify/handle them.

## Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl");
```

## NaN and missing

The NaN (Not-a-Number) can be used to indicate that a floating point number is missing or otherwise strange. For other types of data (for instance, integers), use `missing` instead.

Most computations involving NaN/missing give NaN or missing as a result.

Financial data is often on floating point form and NaN is easier than missing to work with (since NaN is part of the floating point specification, while missing is an add-on). This might suggest using NaN rather than missing.

```
println(2.0 + NaN, " ", 2 + missing)
```

NaN missing

## Loading Data

The next cell replaces -999.99 by NaN or missing in the matrix data. This is a common scenario when data has been loaded from a data set (csv file, say). See the tutorial on loading and saving data for more information.

```
data = [1.0 -999.99;
 3.0 13.0]

z = replace(data,-999.99⇒NaN) #replace -999.99 by NaN or missing
printblue("z: ")
printmat(z)
```

```
z:
 1.000 NaN
 3.000 13.000
```

## Testing for NaN/missing in an Array

You can test whether a number is NaN or missing by using `isunordered()`. (Use `isnan()` or `ismissing()` if you want to test specifically for one of them.)

```
if any(isunordered,z) #check if any NaN/missins
 println("data has some NaN/missing")
end
```

```
data has some NaN/missing
```

## Disregarding NaN/missing in a Vector

can often be done by just `filter()` the vector to keep only elements that are non-NaN/missing.

```
sum(filter(!isunordered,z)) #finds all elements that are not unordered, and sums them
```

```
17.0
```



## Prune All Rows (of a Matrix) with any NaN/missing

It is a (fairly) common procedure in statistics to throw out all cases with NaN/missing values. For instance, let  $z$  be a matrix and suppose  $z[t,:]$  contains one or more NaN/missing values. It is then common (for instance, in linear regressions) to throw out that entire row of the matrix.

The function `Cases2Keep(z,dims=2)` will create a bitvector with true for all rows in  $z$  that has no NaN/missing.

For statistical computations, you may also consider the [NaNStatistics.jl](#) package.

```
"""
 Cases2Keep

Indicate rows (or cols) without NaN/missing
"""
Cases2Keep(z,dims=2) = .!vec(any(isunordered,z;dims));

#z = [0 100;1 NaN;2 21.0] #try this too
z = [0 100;1 missing;2 21] #try this too

vc = Cases2Keep(z)
printblue("z and vc:")
printmat(z,vc;colNames=["col 1","col 2","vc"])

z2 = view(z,vc,:) #view of rows without NaN/missing, as an alternative do z[vc,:]
printblue("z2: view of all rows with without any NaN/missing:")
printmat(z2)
```

z and vc:

	col 1	col 2	vc
0	100		1
1	missing		0
2	21		1

z2: view of all rows with without any NaN/missing:

0	100
2	21

## Converting a View to a Standard Type (extra)\*

Once you have pruned all rows with missings, you may want to convert the matrix to, for instance, `Float64`. This might simplify some of the later code, in particular, if you plan to send this matrix to

R/Python/etc. Notice that if there were no missing (just NaN), then no conversion is needed, but you may still want to do `copy(z2)` to create an independent matrix.

As an alternative, consider `disallowmissing()` from the [Missings.jl](#) package.

```
println("The type of z2 is\n", typeof(z2))

z3 = convert.(Float64,z2) #z3 will be Float64 and a also an independent copy
z4 = copy(z2)
println("\nThe types of z3 and z4 are\n",typeof(z3),"\n",typeof(z4))
```

The type of z2 is

`SubArray{Union{Missing, Int64}, 2, Matrix{Union{Missing, Int64}}, Tuple{Vector{Int64}, Base.Slice{Base.OneTo{Int64}}}, true}`

The types of z3 and z4 are

`Matrix{Float64}`

`Matrix{Union{Missing, Int64}}`

# Downloading Files from Internet

is done by

```
import Downloads
Downloads.download(url)
```

## Loading Packages

```
using Downloads
```

```
if !isdir("Results")
 error("create the subfolder Results before running this program")
end
```

## Some Data from Kenneth French's Homepage

```
http = string("http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/",
 "F-F_Research_Data_Factors_daily_CSV.zip")

println("File to download: ",http)
result = Downloads.download(http,"Results/WhatIJustDownloaded.zip")

printstyled("\nDone. Check $result \n";color=:blue)
```

File to download: http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/F-F\_Research\_Data\_Factors\_daily\_CSV.zip

Done. Check Results/WhatIJustDownloaded.zip

# Threading

means using the multiples cores that modern CPUs have. Some things in Julia are automatically threaded, for instance, many linear algebra computations and the package manager. Also, many packages use are threaded. However, to get your own code threaded, there is some work to do (and some pitfalls to avoid).

## The Simplest Kind of Threading

in Julia is of this type:

```
using Base.Threads: @threads

Z = zeros(N)
@threads for i in 1:N
 local y
 y = SomeFunction(i,W)
 Z[i] = SomeOtherFunction(i,X,y,Z[i],but not Z[j])
end
U = YetAnotherFunction(Z)
```

In this case, `Z` is used for storing the results. Variables that are created in the loop (like `y`) should be declared `local` to avoid that they are shared across threads. Notice that is important to *not* let iteration `i` depend on results created by another iteration (`Z[j]`).

The `@threads` is a simple approach that works well when the iterations are similar (uniform). In contrast, if some iterations are much more costly or you want to use threads in a nested setting, the `@spawn`/`fetch`/`@sync` might be better (not discussed in this notebook).

Threading typically only pays off when the iterations involve *heavy* computations. Otherwise, the cost of setting of the threading might dominate. It is also important to avoid too many allocations (for instance, creating and deleting arrays) inside the threaded loop, or otherwise the memory allocation system (“gc”) will impact the performance.

## Activating Threading

Julia is (currently, as of version 1.12) started with two threads (one working, one interactive). To change that, set the environment variable `JULIA_NUM_THREADS=4` or replace 4 by `auto`. (It is often recommended to set it to the number of physical CPU cores).

In case you cannot change the environment variable, or simply want more fine-grained control, consider the following:

1. Start Julia from a command line as `julia --threads 4` to get 4 working threads (and, in addition, one thread for interaction), or replace 4 by `auto` to let Julia make a choice (often the number of CPU threads).
2. Set up Jupyter/lab by first doing the following in the REPL

```
using IJulia
installkernel("Julia (4 threads)", env=Dict("JULIA_NUM_THREADS"=>"4"))
```

or replace “4” by “auto”. Then pick the right kernel when running the notebook.

3. In VScode, go to the Julia extension settings and search for threads. Then, change to `"julia.NumThreads": 4` (or replace 4 by “auto”).

```
using Printf
using Base.Threads: @threads #load the threading functions

include("src/printmat.jl");
```

```
println("Number of threads: ", Threads.nthreads()) #check how many threads that are available
```

Number of threads: 4

## Comparing w/wo Threads

We will calculate

$$\mu_t = \lambda \mu_{t-1} + (1 - \lambda)x_{t-1}, \text{ for } t = 2 \dots T$$

on each column of a large matrix ( $t$  refers to rows).

The function `ExpMA(x, λ)` does the calculation for a vector (that is, a column in a bigger matrix). `ExpMA_1()` and `ExpMA_2()` loops over the columns without and with threading, respectively.

```

function ExpMA(x,λ) #exponential MA, creates a vector
 T = length(x)
 μ = zeros(T)
 for t in 2:T
 μ[t] = λ*μ[t-1] + (1-λ)*x[t-1]
 end
 return μ
end

```

ExpMA (generic function with 1 method)

```

function ExpMA_1(X,λ) #wrap ExpMA(X,λ) in loop over columns of X
 (T,N) = (size(X,1),size(X,2))
 result = zeros(T,N)
 for i in 1:N
 result[:,i] = ExpMA(X[:,i],λ)
 end
 return result
end

function ExpMA_2(X,λ) #wrap ExpMA(X,λ) but with threaded loop
 (T,N) = (size(X,1),size(X,2))
 result = zeros(T,N)
 @threads for i in 1:N
 result[:,i] = ExpMA(X[:,i],λ)
 end
 return result
end

```

ExpMA\_2 (generic function with 1 method)

```

λ = 0.94
T = 100_000

N = 500
X = randn(T,N)

Y1 = ExpMA_1(X,λ)
Y2 = ExpMA_2(X,λ)

println("Test if the same results (Y1==Y2): ", Y1==Y2)

```

Test if the same results (Y1==Y2): true

## Avoiding (Memory) Allocations

is often a good idea. But, it becomes perhaps even more important in threaded applications.

The next cell defines a new function ExpMAx!() which writes the result to an existing matrix (the first function argument,  $\mu$ ) instead of creating a new vector every time. Then, the function ExpMAx\_2() does a threaded loop over the columns in X. We verify that we get the same result as before.

```
function ExpMAx!(μ ,x, λ ,i) #exponential MA, writes to column in an existing matrix μ
 T = size(x,1)
 for t in 2:T
 μ [t,i] = λ * μ [t-1,i] + (1- λ)*x[t-1,i]
 end
 return μ
end

function ExpMAx_1(X, λ) #no threading, but create
 (T,N) = (size(X,1),size(X,2)) #fewer intermediate vectors
 result = zeros(T,N)
 for i in 1:N
 ExpMAx!(result,X, λ ,i)
 end
 return result
end

function ExpMAx_2(X, λ) #wrap but with threaded loop and creating
 (T,N) = (size(X,1),size(X,2)) #fewer intermediate vectors
 result = zeros(T,N)
 @threads for i in 1:N
 ExpMAx!(result,X, λ ,i)
 end
 return result
end

Y3 = ExpMAx_1(X, λ)
Y4 = ExpMAx_2(X, λ)
println("Test if the same results (Y1==Y3==Y4): ", Y1==Y3==Y4)
```

Test if the same results (Y1==Y3==Y4): true

## Speed Comparison

without thread, with threads (but creating intermediate vectors) and with threads but no intermediate vectors.

```
using BenchmarkTools #package for benchmarking

printblue("standard loop:")
@btime ExpMA_1($X,λ) #use $X to get more accurate timing

printblue("threaded loop:")
@btime ExpMA_2($X,λ)

printblue("standard loop, but without intermediate vectors:")
@btime ExpMAx_1($X,λ)

printblue("threaded loop, but without intermediate vectors:")
@btime ExpMAx_2($X,λ)

printmagenta("\n...reducing memory allocations and using threads can both speed up the code")
```

standard loop:

351.675 ms (3003 allocations: 1.12 GiB)

threaded loop:

152.582 ms (3031 allocations: 1.12 GiB)

standard loop, but without intermediate vectors:

209.078 ms (3 allocations: 381.47 MiB)

threaded loop, but without intermediate vectors:

93.013 ms (31 allocations: 381.47 MiB)

...reducing memory allocations and using threads can both speed up the code

## Things that Could Go Wrong with Threading (extra)\*

are often related to data races (different threads writing to the same memory location).

### Case 1: Several Threads Changing the Same Elements of an Array

When threads write to the same memory locations, anything can happen.



```

N = 10_000

x = [0]
@threads for i in 1:N
 x[1] = x[1] + 1 #all threads writing to x[1]
end

println("This ought to be $N, but it is ",x[])

```

This ought to be 10000, but it is 3403

## Case 2: Threads Writing to BitArrays

Code like in the next cell can also create unexpected results since the threads are trying write to the same BitArray (which has a packed format). Run a few times to see the problem.

This is solved by instead using `Z = fill(false,N)` which is an array of Booleans.

```

N = 10
Z = falses(N)
#Z = fill(false,N) #ucomment to solve the problem
@threads for i in 1:N
 Z[i] = true
 sleep(0.5) #give the thread something to do
end

println("All $N values in Z should be `true`, but only ", sum(Z)," are. If needed, rerun to see.")

```

All 10 values in Z should be `true`, but only 9 are. If needed, rerun to see.

## Case 3: @threads and Variable Scope

Code like this

```

v = 1:2
@threads for i in 1:N
 v = something
 x = SomeFunction(v)
end

```

can create unexpected results since the threads are sharing `v`. This is solved by declaring `v` inside the loop to be local.

```
using LinearAlgebra

function f2(N)
 v = falses(N+1)
 x = zeros{Int}(N,N)
 @threads for i = 1:N
 #local v #uncomment to solve the problem
 v = falses(N)
 v[i] = true
 x[v,i] .= i
 end
 return x
end

println("This should always be zero. Run a few times to check if that holds.\n")
M = 100
dev = zeros(M)
for i = 1:M
 dev[i] = maximum(abs,f2(i) - diagm{1,i})
end
println("All $M values should be 0, but only ", sum(dev.==0), " are.")
```

This should always be zero. Run a few times to check if that holds.

All 100 values should be 0, but only 10 are.

# A Module

...is, for instance, a collection of your own functions that you often use.

One of the advantages of a module is that you can do using `.TutorialModule` (or using `TutorialModule`, depending on the approach as discussed below) instead of having to `include()` every file. It is also likely to be quicker.

## Step 1: Create a Module File

Create a file that defines the module.

The next cell shows the contents of the module file. Change it, for instance, by exporting also `printred`.

```
printstyled("the contents of src/TutorialModule.jl:\n\n";color=:blue,bold=true)
println(read("src/TutorialModule.jl",String))
```

the contents of `src/TutorialModule.jl`:

```
module TutorialModule
```

```
import Printf #the module can use other packages
#import LinearAlgebra
```

```
export printmat, printlnPs, printblue, printTeXTable #available after `using .TutorialModule`
```

```
include("printmat.jl") #files with the functions
include("printTeXTable.jl") #could also type in the code here
```

```
end
```

## Step 2: Include and Use the Module

```
include("src/TutorialModule.jl")
using .TutorialModule #notice the dot(.)
```

```
x = [11 12;21 22]
```

```
printmat(x) #the printmat() function should now be available
```

```
11 12
21 22
```

## Steps 0-2: Alternative Approach to Creat/Use a Module (extra)

You should probably **restart the Julia kernel** before running the cells below, since we are going to load the same module, but using another approach.

0. Tell Julia where to find the files for the module by adding to `LOAD_PATH`. This has to be redone every time you run code using the module. (For instance, by putting it in your `startup.jl` file.)
  1. Use the same module file as above
  2. Run using `TutorialModule` (without the dot)

This approach has the advantage that the module will be available from everywhere and also *precompiled* (which implies quicker loading, except for the first time).

In case VS Code gives a strange message when you do using `TutorialModule` the first time, rerun the cell.

```
PathToMyFolder = joinpath(pwd(),"src") #change as needed
push!(LOAD_PATH,PathToMyFolder);
```

```
using TutorialModule #no dot, will precompile on the first run
```

```
x = [11 12;21 22]
```

```
printmat(x) #the printmat() function should now be available
```

```
11 12
21 22
```

# A Project

A project is folder where you can make specific package installations, without affecting other projects (or Julia, more generally).

When activating that project (see below for how), you have access to all the packages installed in the global environment (that is, those installed without specifying a project) plus the packages installed in this project.

## Step 1: Create the Project

Create the project folder (LittleProject).

```
dir = "LittleProject"
rm(dir, recursive=true, force=true) #delete any existing folder
mkdir(dir) #run once to create the project folder

printstyled("check that you now have an empty subfolder called $dir";color=:blue)
```

check that you now have an empty subfolder called LittleProject

## Step 2: Use the Project

1. Move to the project folder (by `cd()` or manually)
2. Do `Pkg.activate(".")` to activate the project. Needs to be done every time you work with that project
3. Optionally add packages. Because of point 2, the package installations will now be to the project, and not to the general Julia environment.
4. Write&run code from this folder.

```

printstyled("\nMove to the project folder and continue working from there.\n\n";color=:blue)
cd(dir) #alternatively, do this manually

import Pkg #run this in the LittleProject folder
Pkg.activate(".";io=devnull) #always run this, io=devnull to suppress printing here

Pkg.add("Missings";io=devnull) #optionally, install packages (do once)

printstyled("\nA Project.toml file has been created in $dir";color=:blue)

```

Move to the project folder and continue working from there.

A Project.toml file has been created in LittleProject

```

using Missings #use the (locally) installed package, run some code

x = [1,2]
x = allowmissing(x)
x[end] = missing
summary(x)

```

"2-element Vector{Union{Missing, Int64}}"

```

using BenchmarkTools #this has (probably) been installed to the general environment, so we

```

# A Local Package

A local package is a *project* with a subfolder `src` containing a module file with the same name as the project. The package can later be used from anywhere. It can be thought of as a library.

## Step 1: Create the Package in the Project Folder

To create a package in an empty subfolder `LittlePackage`, do `Pkg.generate("LittlePackage")`. This creates the subfolder, with `.toml` files (they define the packages you have added to the project—so far none) and a `src` subfolder with a skeleton module file.

```
dir = "LittlePackage"
rm(dir, recursive=true, force=true) #delete any existing folder

import Pkg
Pkg.generate(dir; io=devnull) #run this once, io=devnull to suppress printing

printstyled("The folder $dir has been created, it has a Project.toml file and a subfolder src"; color=:green)
```

The folder `LittlePackage` has been created, it has a `Project.toml` file and a subfolder `src`

## Step 2: Create/Transfer Files to the Package

Copy or create files to/in `LittlePackage/src`. Yes, overwrite the existing file there.

```
files = ["LittlePackage.jl", "printmat.jl", "printTeXTable.jl"]
for file in files
 cp(joinpath("src", file), joinpath(dir, "src", file); force=true)
end

println("contents of $dir: ", readdir(dir))
```

contents of `LittlePackage`: `["Project.toml", "src"]`

### Step 3: Set up Dependencies of the Package

1. Move to the `LittlePackage` folder.
2. Do `Pkg.activate(".")` to activate the project.
3. Add dependencies according to the `src/LittlePackage.jl` file. In this case, just `Printf`.
4. Go back to the default (global) environment, or restart Julia

```
cd(dir)

import Pkg
Pkg.activate(".";io=devnull) #make LittlePackage the environment
Pkg.add("Printf";io=devnull)

println()
Pkg.activate(;io=devnull) #back to the default (global) environment, or restart Julia
```

### Use the Package

...from another place

1. Make sure Julia knows how to find the package, by either adding the location to `LOAD_PATH` or by `develop(path=PathToMyFolder)`. In the former case, it has to be done every time you want to use it, while in the latter case, the package will be permanently added to your environment (can later be removed by `Pkg.remove()`.)
2. Use it.

```
cd("../") #move to another folder, change as needed
PathToMyFolder = joinpath(pwd(),"LittlePackage") #change as needed

#do one of A or B
push!(LOAD_PATH,PathToMyFolder) #A, do this
#import Pkg; Pkg.develop(path=PathToMyFolder) #B, or this

using LittlePackage
printmat([1,2]) #check that it works
```



1  
2

# Solving Non-linear Equations

The [Roots.jl](#) package provides methods for solving a non-linear equation (one variable, one function).

For a system of non-linear equations, use [SimpleNonlinearSolve.jl](#) or perhaps [NLsolve.jl](#) for a more lightweight installation. This notebook also shows how to use Optim.jl as a simple tool.

## Load Packages and Extra Functions

```
using Printf, Roots, SimpleNonlinearSolve

include("src/printmat.jl")
```

@doc2

```
using Plots
default(size = (480,320),fmt = :png)
```

## Defining and Plotting the Function

In the next few cells, we define a fairly simple function and then plot it.

If possible, plot your function before trying to find the roots. Maybe you see something strange, perhaps there are several roots? It also helps you set the initial guesses (or brackets) for root solving.

```
fn1(x,c) = 2*(x - 1.1)^2 - c
```

fn1 (generic function with 1 method)

```
x = range(-1,3,length=41)

p1 = plot(x,fn1.(x,0.5),
 linecolor = :red,
 linewidth = 2,
 legend = nothing,
 title = "the fn1(x,0.5) function",
 xlabel = "x",
 ylabel = "y")
display(p1)
```

the fn1(x,0.5) function



There seems to be two roots: around 0.6 and 1.6.

## Solving a Non-Linear Equation

`find-zero()` accepts parameters for the function we want to solve, that is, values that we don't want to change. This could be like `c` in `fn1()`.

The syntax is

```
find_zero(fn1b,x0;p=[some parameters])
```

where  $p$  is a container (scalar, array, tuple, named tuple) of parameters/data. This follows the [SciML](#) convention, which is popular in the Julia ecosystem.

The approach below defines a wrapper function `fn1b`. One advantage with a wrapper function is that it handles the conversion from the original function `fn1` to the format that the `Roots.jl` package wants. This means that the original function can be written in a general way, for convenient usage in other contexts.

It is still possible to use a slightly older syntax `find_zero(x→fn1(x,0.5),(-1,1))`, where `x→fn1(x,0.5)` defines an anonymous function of one input only.

```
fn1b(x,p) = fn1(x,p.c) #wrapper function, to fit `find_zero()` syntax
p = (;c=0.5) #here just one parameter, could be eg. (;c=0.5,x=zeros(10,2))

xRoot3 = find_zero(fn1b,(-1,1);p) #searches in the bracket (-1,1)
printlnPs("at which x is fn1(x,0.5) = 0, using p? ",xRoot3)

xRoot3 = find_zero(fn1b,2;p) #searches with start at 2
printlnPs("at which x is fn1(x,0.5) = 0, using p? ",xRoot3)
```

```
at which x is fn1(x,0.5) = 0, using p? 0.600
at which x is fn1(x,0.5) = 0, using p? 1.600
```

## Finding Many Roots (extra)\*

Instead, running

```
find_zeros(x→fn1(x,0.5),x0,x1)
```

searches for all roots between  $x_0$  and  $x_1$ . (Notice the  $s$  in `find_zeros`.)

The syntax for `find_zeros` does not (yet) support the parameter container, so we instead convert the 2-argument function to an anonymous function by `x→fn1(x,0.5)`.

```
xRootsAll = find_zeros(x→fn1(x,0.5),-1,3) #find_zeros (notice the "s")
printlnPs("at which x is fn1(x,0.5) = 0? ",xRootsAll)
```

```
at which x is fn1(x,0.5) = 0? 0.600 1.600
```

## Solving a System of Non-Linear Equations\*

The SimpleNonlinearSolve.jl package has many options. The cells below illustrate a very simple case (2 non-linear equations with 2 unknowns, no information about the derivatives).

The two equations are

$$y - x^2 - c = 0$$

$$y - x - c = 0$$

We will use  $c = 1$  as a way to illustrate how the function can use data/parameters.

The roots are at  $(x, y) = (0, 1)$  and also at  $(1, 2)$ . The solution will depend on the starting guess.

### Remarks on the code

- `prob = NonlinearProblem{false}(fn2b,x0,p)` defines a problem to be solved. `false` indicates that any arrays will be created anew, instead of reusing already allocated arrays. `fn2b` is the function to solve, `x0` is the starting guess, and `p` is (here) a named tuple with parameters and potentially more. In some cases, this could be empty.
- `Sol = solve(prob,SimpleNewtonRaphson())` solves for the roots.

```
function fn2(u,c) #u is a vector with 2 elements, the output too
 (x,y) = (u[1],u[2])
 z = [y-x^2-c,y-x-c] #should be equal to [0,0]
 return z
end

fn2b(u,p) = fn2(u,p.c) #wrapper function, to use c from NamedTuple p, as p.c
```

fn2b (generic function with 1 method)

```
p = (;c=1) #parameters of the function
x0 = [0.0,0.5] #starting guess

prob = NonlinearProblem{false}(fn2b,x0,p) #set up the problem
Sol = solve(prob,SimpleNewtonRaphson()) #solve
printlnPs("There is a solution at ",Sol.u)

x0 = [1.0,0.0] #another starting value, using fn2c
prob = NonlinearProblem{false}(fn2b,x0,p)
Sol2c = solve(prob,SimpleNewtonRaphson())
printlnPs("There is a another solution at ",Sol2c.u)
```

```

There is a solution at 0.000 1.000
There is a another solution at 1.000 2.000

```

## A Q&D Root Finder by Using Optim.jl\*

For a quick root finding, Optim.jl can often do the job, which may be convenient if you already use Optim and want to avoid installing another package.

The approach below is to minimize  $\text{sum}(\text{dev}.^2)$ , where dev is a vector of deviations from zero (for each equation).

```
using Optim
```

```

fn2c(u,c) = sum(abs2,fn2(u,c)) #minimize sum(dev.^2), wrapper fn, for convenience

Sol4 = optimize(u→fn2c(u,1),[0.0,0.5],BFGS()) #u→fn2c(u,1) creates an anonymous function
u4 = Optim.minimizer(Sol4)
res4 = fn2c(u4,1)
printlnPs("solution at (x,y)= ",u4)
printlnPs("max|dev| ",maximum(abs,res4))

```

```

solution at (x,y)= -0.000 1.000
max|dev| 0.000

```

# Optimization

of linear-quadratic problems, using the [OSQP.jl](#) package.

See `Ch23_5a_Optimization_Quadratic.ipynb` in the `ModernFinanceTheory` repository.

# Numerical Optimization

This notebook uses the [Optim.jl](#) package for unconstrained and box constrained (lower and upper bounds).

For other types of constraints, consider [Optimization.jl](#), [NLOpt.jl](#), or [JuMP.jl](#).

See `Ch23_5b_Optimization_General.ipynb` in the `ModernFinanceTheory` repository.



# Numerical Integration and Differentiation

This notebook illustrates how to perform numerical integration and differentiation.

There are several packages for doing this. Here, the focus is on the `QuadGK.jl` and `FiniteDiff.jl` packages.

Towards the end of the notebook, there are also some (extra) comments about the `ForwardDiff.jl` and `ReverseDiff.jl` packages.

## Load Packages and Extra Functions

```
using Printf

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

## Numerical Integration

with the [QuadGK.jl](#) package.

There are many alternative packages (for instance, [HCubature.jl](#)), but they are not discussed in this notebook.

```
using QuadGK
```

## The Pdf of the $N()$ Distribution

As a simple illustration, the next cells plot and integrate the  $N(0, \sigma)$  pdf.

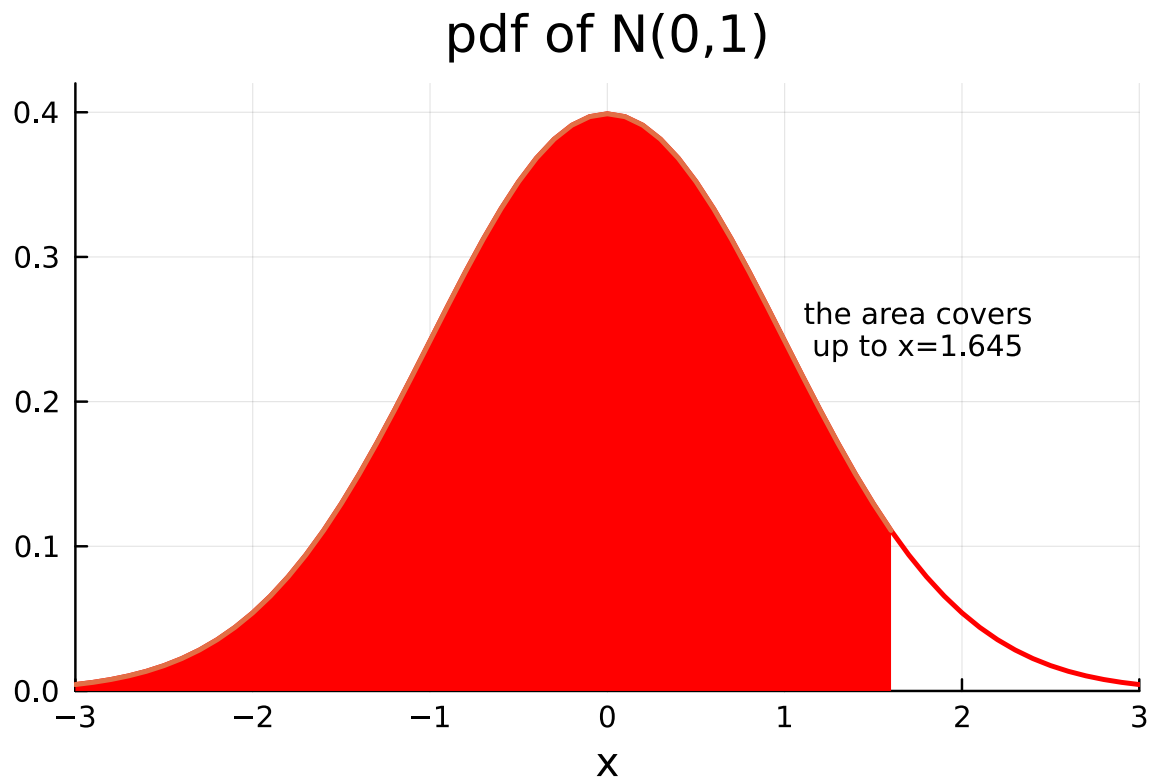
Notice: the function takes  $\mu$  and  $\sigma$  (not  $\sigma^2$ ) as inputs, similar to the `Distributions.jl` package.

```
function ϕ NS(x, μ =0, σ =1) #pdf of N(μ , σ), defaults to N(0,1)
 z = (x - μ)/ σ
 pdf = exp(-0.5*z^2)/(sqrt(2*pi)* σ)
 return pdf
end
```

$\phi$ NS (generic function with 3 methods)

```
x = -3:0.1:3
xb = filter(<=(1.645),x) #same as x[x.<=1.645]

p1 = plot(x, ϕ NS.(x),
 linecolor = :red,
 linewidth = 2,
 ylims = (0,0.42),
 xlims = (-3,3),
 legend = nothing,
 title = "pdf of N(0,1)",
 xlabel = "x",
 ylabel = "",
 annotation = (1.75,0.25,text("the area covers\nup to x=1.645",8)))
plot!(xb, ϕ NS.(xb),fillcolor=:red,linewidth=2,legend=nothing,fill=(0,:red))
display(p1)
```



### Calculating $\text{Prob}(x \leq 1.645)$

Let  $\phi(x, \mu, \sigma)$  be the pdf of an  $N(\mu, \sigma)$  variable. The next cell calculates

$$\int_{-\infty}^{1.645} \phi(x, 0, \sigma) dx,$$

by numeric integration.

The input to `quadgk` should be a function with only one argument. We do that by creating an anonymous function  $x \rightarrow \phi_{NS}(x, 0, 2)$  or just  $\phi_{NS}$  if we want to use the default values of the  $(\mu, \sigma)$  inputs.

```
cdf1, = quadgk(φNS,-Inf,1.645) #N(0,1), default values
printlnPs("\nPr(x<=1.645) according to N(0,1):", cdf1)

cdf2, = quadgk(x→φNS(x,0,2),-Inf,1.645) #N(0,σ=2)
printlnPs("\nPr(x<=1.645) according to N(0,2):", cdf2)
```

```
Pr(x<=1.645) according to N(0,1): 0.950
```

$\Pr(x \leq 1.645)$  according to  $N(0,2)$ : 0.795

## Numerical Derivatives

We use the [FiniteDiff.jl](#) package to calculate numerical derivatives.

The `FiniteDiff.jl` package does not export its functions and the names are very long. In the next cell we therefore rename the relevant functions to  $\delta$  and  $\nabla$ .

```
using FiniteDiff: finite_difference_derivative as δ , finite_difference_gradient as ∇
```

### A Derivative of a Function with One Input and One Output

The next cells find the derivative of a function with only one argument and one output. First, at a single point, then at several points. It is a simple case, so we can compare with the analytical result

```
function fn1(x,a) #a simple function, to be differentiated
 return (x - 1.1)^2 - a #see below for a plot
end

x0 = 2.0
dydx_A = $\delta(x \rightarrow fn1(x, 0.5), x_0)$ #derivative, make sure x0 is a Float64

printblue("The derivative at x=$x0 is:\n")
printmat(dydx_A, 2*(x0-1.1); colNames=["numerical", "exact"])
```

The derivative at  $x=2.0$  is:

numerical	exact
1.800	1.800

```
x = -3:6/99:6 #calculate the derivative at many points

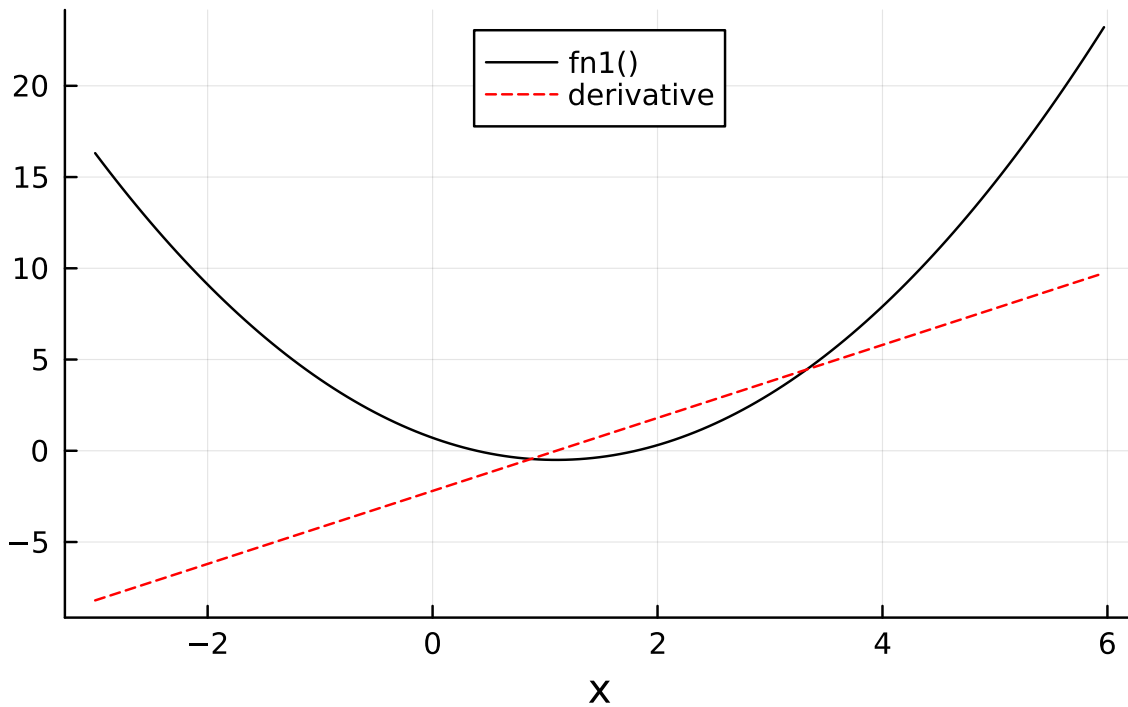
dydx_A = [$\delta(z \rightarrow fn1(z, 0.5), x_i)$ for x_i in x]; # x_i should be Float64
```

```

p1 = plot([x x],[fn1.(x,0.5) dydx_A],
 line = [:solid :dash],
 linecolor = [:black :red],
 label = ["fn1()" "derivative"],
 legend = :top,
 title = "fn1() and its derivative",
 xlabel = "x",
 ylabel = "")
display(p1)

```

## fn1() and its derivative



### A Derivative of a Function with a Vector of Inputs and One Output (Gradient)

The next cell finds the vector of partial derivatives (the gradient) of a function that takes a vector of inputs and gives one output. It's a simple case, so we can also compare with the an analytic result.

```

fn2(w,Σ) = w'*Σ*w #a quadratic form. The derivative wrt w is 2Σw
Σ = [1 0.5]

```

```

0.5 2]
w0 = [1.0,2.0] #floats

grad = ∇(z→fn2(z,Σ),w0)

printblue("The gradient is:\n")
printmat(grad,2*Σ*w0;colNames=["numerical","exact"])

```

The gradient is:

numerical	exact
4.000	4.000
9.000	9.000

## Comments on Using the ForwardDiff or ReverseDiff Packages (extra)\*

The [ForwardDiff.jl](#) and [ReverseDiff.jl](#) packages are exact and fast. (Use the former for functions with few inputs and many outputs, and the latter for vice versa.)

However, they require that your code can handle also other types than Floats. In most cases, that is not a problem, but you may have to watch out if you create arrays to store (intermediate?) results inside the function. See the examples below.

```

using ForwardDiff: ForwardDiff.gradient as ForwardDiff_∇
using ReverseDiff: ReverseDiff.gradient as ReverseDiff_∇

grad_B = ForwardDiff_∇(z→fn2(z,Σ),w0) #this works
grad_C = ReverseDiff_∇(z→fn2(z,Σ),w0)

printmat(grad_B,grad_C,2*Σ*w0;colNames=["ForwardDiff","ReverseDiff","exact"],width=15)

```

ForwardDiff	ReverseDiff	exact
4.000	4.000	4.000
9.000	9.000	9.000

## An Illustration of the Pitfall of ForwardDiff.jl and ReverseDiff.jl

```

function fnDoesNotWork(b,a)
 z = zeros(length(b)) #will not work with ForwardDiff, since
 for i in 1:length(z) #since z cannot store dual numbers
 z[i] = b[i]*i
 end
 return sum(z) + a
end

function fnDoesWork(b,a)
 z = similar(b) #will work with ForwardDiff, since
 z .= 0 #when b is a dual number, so is z
 for i in 1:length(z)
 z[i] = b[i]*i
 end
 return sum(z) + a
end

```

fnDoesWork (generic function with 1 method)

```

b0 = [2] #should be a vector for use with the gradient functions

grad_x1 = ForwardDiff_∇(b→fnDoesWork(b,1),b0)
#grad_x1 = ForwardDiff_∇(b→fnDoesNotWork(b,1),b0) #uncomment to get an error

grad_x2 = ReverseDiff_∇(b→fnDoesWork(b,1),b0)
#grad_x2 = ReverseDiff_∇(b→fnDoesNotWork(b,1),b0) #uncomment to get an error

printlnPs(grad_x1,grad_x2)

```

1            1

# Interpolation

This notebook uses the [Interpolations.jl](#) package. As alternatives, consider [Dierckx.jl](#) and [DataInterpolations.jl](#). They might handle irregular grids better.

## Load Packages and Extra Functions

```
using Printf, Interpolations

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

## Interpolation of $y = f(x)$

Interpolations are particularly useful when *(a)* we repeatedly want to evaluate a function  $f(x)$  that is time consuming to calculate but we are willing to accept approximate results, or *(b)* when we only know  $f(x)$  for a grid of  $x$  values but we are willing to assume that the function is pretty smooth (perhaps even linear over short intervals).

In either case, we do something like this:

1. Calculate  $f(x)$  values for a grid of  $x$ . This creates a “look-up” table.
2. Replace the calculation of  $f(z)$  by interpolating/extrapolating from the “look-up” table.



## Interpolate

The next cell calls on `cubic_spline_interpolation()` to create the “look-up” table (more precisely, create an interpolation object).

To use a cubic spline it is required that the  $x_i$  grid is *uniformly spaced* (for instance, 0.1, 0.2, ...). Actually, it has to be a range. The case of a non-uniformly spaced  $x$  grid is discussed later.

The option `extrapolation_bc=...` determines how extrapolation beyond the range of the  $x_i$  grid is done.

The second cell interpolates and extrapolates  $y$  at some specific  $z$  values.

```
xGrid = range(-pi, 3.0, length=101) #uniformly spaced grid
yGrid = sin.(xGrid); #y values at xGrid

itp = cubic_spline_interpolation(xGrid, yGrid, extrapolation_bc=Flat());

z = [0.25, 0.75, 1.25] #to interpolate the y values at

y_interpolated = itp(z)
printmat([z y_interpolated sin.(z)], colNames=["z", "interpolated y", "true y"], width=17)

z2 = [pi+0.1, pi+0.5] #to extrapolate the y values at
y_extrapolated = itp(z2)

printmat([z2 y_extrapolated], colNames=["z2", "extrapolated y"], width=17)
```

z	interpolated y	true y
0.250	0.247	0.247
0.750	0.682	0.682
1.250	0.949	0.949

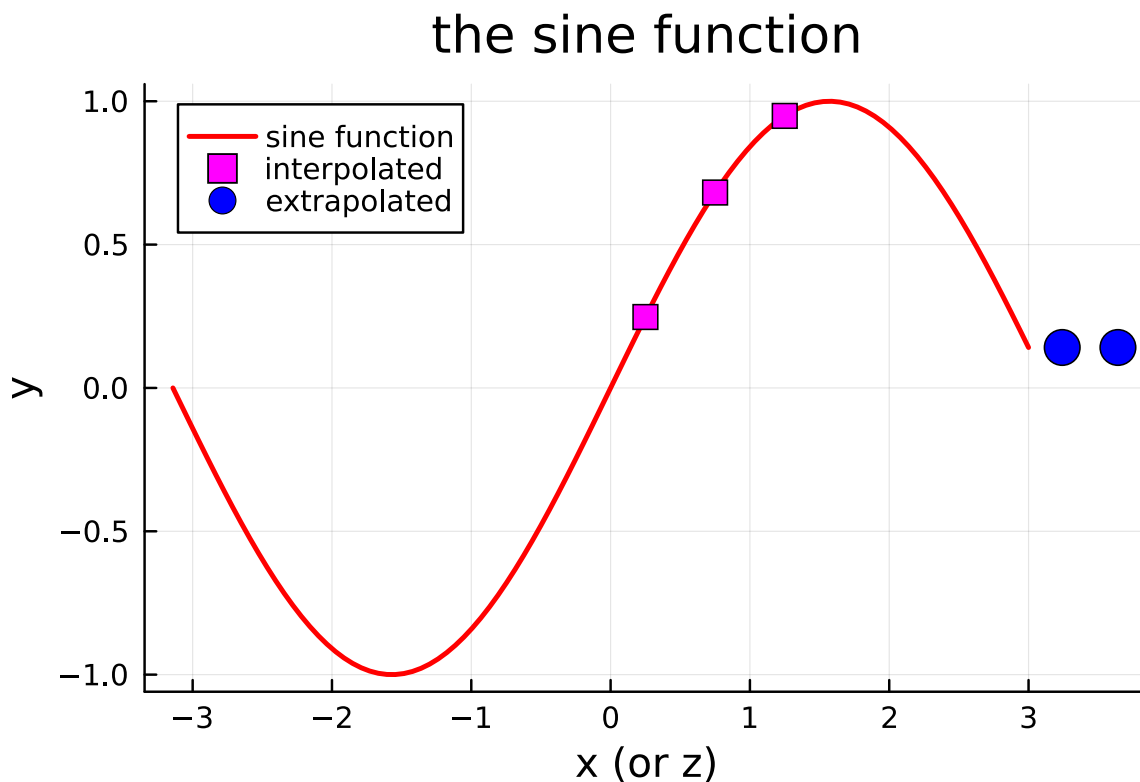
z2	extrapolated y
3.242	0.141
3.642	0.141

## Plotting the Results

```

p1 = plot(xGrid,yGrid,
 linecolor = :red,
 linewidth = 2,
 label = "sine function",
 title = "the sine function",
 xlabel = "x (or z)",
 ylabel = "y")
scatter!(z,y_interpolated,markercolor=:magenta,markersize=5,marker=:square,label="interpolated")
scatter!(z2,y_extrapolated,markercolor=:blue,markersize=8,label="extrapolated")
display(p1)

```



## Interpolation with General x Vectors

...when we cannot guarantee that the look-up table of  $y_i = f(x_i)$  is from uniformly spaced  $x_i$  values (the grid is irregular). This is often the case, for instance, when we have empirical data on  $(x_i, y_i)$ .

The approach works similar to before, except that the `cubic_spline_interpolation` must be replaced by `linear_interpolation`. Alternatively, consider some of the other packages mention at the top of

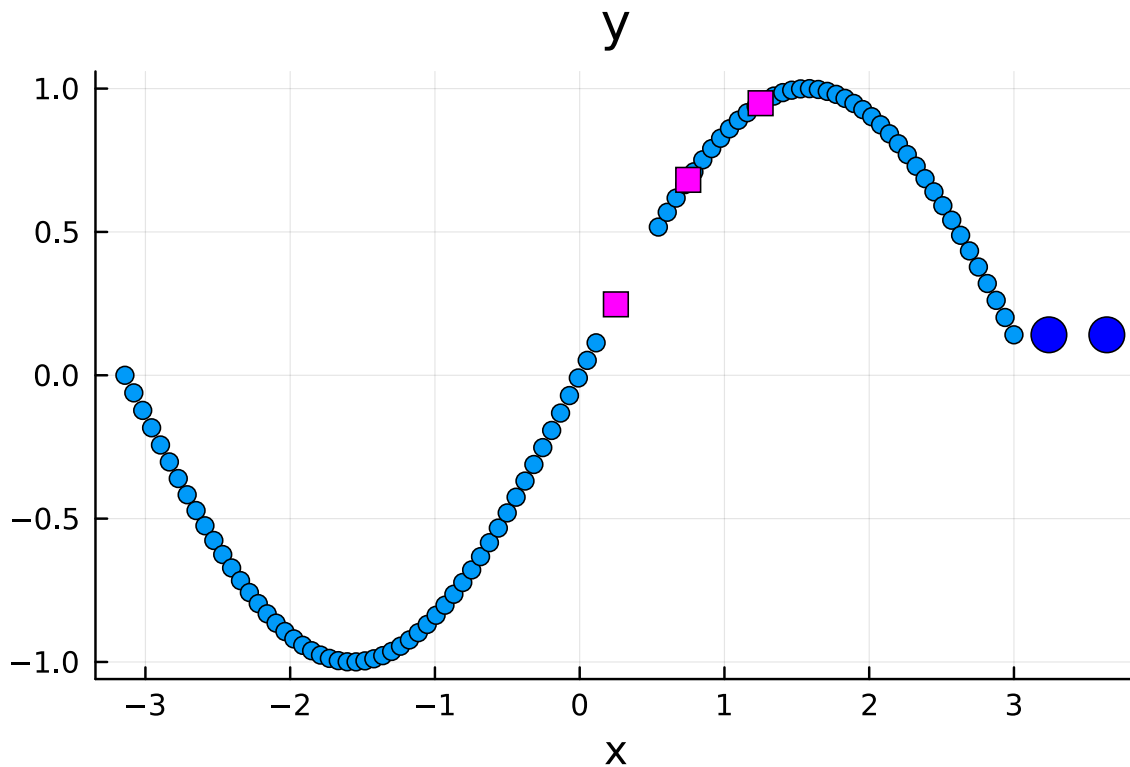
the notebook.

```
xGrid2 = deleteat!(collect(xGrid),55:60) #non-uniformly spaced grid
yGrid2 = sin.(xGrid2)

itp2 = linear_interpolation(xGrid2,yGrid2,extrapolation_bc=Flat());

y_interpolated = itp(z)
y_extrapolated = itp2(z2)

p1 = scatter(xGrid2,yGrid2,
 linecolor = :red,
 legend = false,
 title = "y",
 xlabel = "x")
scatter!(z,y_interpolated,markercolor=:magenta,markersize=5,marker=:square,label="interpolated")
scatter!(z2,y_extrapolated,markercolor=:blue,markersize=8,label="extrapolated")
display(p1)
```



# Running Python, C or R Code from Julia

This notebook first provides a basic introduction to how to run Python, using the `PythonCall.jl` package and also C code (requires no package) from Julia. Please see the [PythonCall.jl](#) homepage for instructions for how to either use an existing Python installation, or let the package make one.

An alternative package (not used here) for running Python is [PyCall.jl](#), which is a slightly older approach.

The second part of the notebook runs R code, using the [RCall.jl](#) package, from Julia. See the [RCall.jl](#) homepage (the installation instructions) for how to use your existing R installation or for letting `RCall.jl` install R for you.

```
using Printf, DelimitedFiles
include("src/printmat.jl");
```

## Load Data

```
x = readlm("Data/MyData.csv",',',skipstart=1) #reading the csv file

(Rme,Rf,R) = (x[:,2],x[:,3],x[:,4]) #creating variables from columns of x
y = R - Rf #do R .- Rf if R has several columns

c = ones(length(Rme))
x = [c Rme]

b = x\y
println("OLS coeffs according to Julia")
printmat(b)
```

```
OLS coeffs according to Julia
-0.504
1.341
```

## Python

You need Python's statsmodels package installed to run the code below. If you have let PythonCall install Python for you, use [CondaPkg.jl](#) to add packages: `import CondaPkg; CondaPkg.add("statsmodels")`.

In the next cells we (a) load the PythonCall.jl package and activates the (Python) package statsmodels; (b) call some functions (eg. `OLS()`) from statsmodels.

In many cases, you can use Julia arrays (like `y` and `x` in the cells below) in a call on a Python function. In other cases (like here), you need to convert them to numpy arrays, by using `Py(y).to_numpy()`.

```
using PythonCall
sm = pyimport("statsmodels.api") #activate this package and call it `sm`
println("")
```

```
resultsP = sm.OLS(Py(y).to_numpy(), Py(x).to_numpy()).fit() #np.array() to convert to Python
println(resultsP.summary())
```

```

 OLS Regression Results
=====
Dep. Variable: y R-squared: 0.519
Model: OLS Adj. R-squared: 0.518
Method: Least Squares F-statistic: 416.2
Date: Mon, 24 Aug 2026 Prob (F-statistic): 2.72e-63
Time: 20:45:42 Log-Likelihood: -1241.7
No. Observations: 388 AIC: 2487.
Df Residuals: 386 BIC: 2495.
Df Model: 1
Covariance Type: nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-0.5042	0.305	-1.654	0.099	-1.103	0.095
x1	1.3410	0.066	20.401	0.000	1.212	1.470

```
=====
Omnibus: 259.682 Durbin-Watson: 1.870
Prob(Omnibus): 0.000 Jarque-Bera (JB): 5249.944
Skew: 2.482 Prob(JB): 0.00
Kurtosis: 20.323 Cond. No. 4.68
=====
```

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
#foreach{println,pydir(resultsP)) #run this to see all attributes

b_P1 = PyArray(resultsP.params) #a Julia view of the Python array,
b_P2 = pyconvert(Array{Float64}, resultsP.params) #or copy/convert to a Julia array

printblue("Comparing the estimates in Julia and Python:")
printmat(b,b_P1,b_P2;colNames=["Julia","Julia view","Py→Ju"],width=15)
```

Comparing the estimates in Julia and Python:

Julia	Julia view	Py→Ju
-0.504	-0.504	-0.504
1.341	1.341	1.341

We can also run Python code (quoted in “” “” “”) directly as in the next cell.

```
@pyexec (x=x, y=y) ⇒ """
import numpy as np
x = np.array(x)
y = np.array(y)
xx = np.matmul(x.T, x)
xy = np.matmul(x.T, y)
b_p = np.linalg.solve(xx, xy)
""" ⇒ b_p

b_P3 = PyArray(b_p)
printmat(b_P3)
```

```
-0.504
1.341
```

## C

This section shows some simple examples of how to call a C function. The functions are in the file `My_C_Stuff.c` (printed in the next cell). The first function `c_dot` defines a dot product between two vectors and the second function `c_ols` a simple linear regression.

```
#println(read("Data/My_C_Stuff.c",String)) #uncomment to see the c file
```

To compile to a dynamic library (dll on windows), I use gcc (for x86\_64) from [mingw-64](#) and run the following in the mingw terminal

```
gcc -shared -fPIC My_C_Stuff.c -o My_C_Stuff.dll
```

To call the C functions, place the dll file in the current folder and then run the following cells.

## A Function which Returns a Number

In the next example, we use the function `c_dot` in `My_C_Stuff.dll`. The function calculates the inner product of two vectors.

The details are: 1. `mylibc.c_dot` is the library.function 2. `length(y)::Csize_t` is the first input and its type (an integer indicating the number of elements in `y`) 3. `y::Ptr{Float64}` is the second input (a pointer to an array of Floats) and similarly for `x2` 4. `Float64` is the type of the output

(We could potentially wrap this in a Julia function that checks for the right input types and outputs the result.)

```
mylibc = "My_C_Stuff.dll"

x2 = x[:,2]; #get a vector with the regressor values

z = @ccall mylibc.c_dot(length(y)::Csize_t, y::Ptr{Float64}, x2::Ptr{Float64})::Float64

printlnPs("The inner product of x2 and y in Julia and C: ",x2'y," ",z)
```

```
The inner product of x2 and y in Julia and C: 11071.648 11071.648
```

## A Function which Returns a Vector

The details are as above, except that 1. `mylibc.c_ols` is the library.function 2. `Cvoid` is the type of the output, which here indicates that the function does not have an output. Rather, the function modifies the vector `b_c` by putting the OLS results there.

```

b_c = zeros(2) #where C will store the regression results

@ccall mylibc.c_ols(length(y)::Csize_t, y::Ptr{Float64}, x2::Ptr{Float64}, b_c::Ptr{Float64})::Cvoid

println("Comparing the estimates in Julia and C")
printmat([b b_c];colNames=["Julia","C"])

```

Comparing the estimates in Julia and C

Julia	C
-0.504	-0.504
1.341	1.341

## R

To avoid naming conflicts with the Python packages, *restart the kernel* and work from here.

The first cell loads the data into Julia (again). The subsequent cells load the RCall.jl package, transfers the data from Julia to R (@rput) and then run an OLS regression (reval()).

```

using Printf, DelimitedFiles
include("src/printmat.jl") #quickly redoing the Julia part

x = readlm("Data/MyData.csv",'',skipstart=1) #reading the csv file
(Rme,Rf,R) = (x[:,2],x[:,3],x[:,4]) #creating variables from columns of x
y = R - Rf #do R .- Rf if R has several columns
c = ones(length(Rme))
x = [c Rme]
b = x \ y
println("OLS coeffs according to Julia")
printmat(b)

```

OLS coeffs according to Julia

-0.504
1.341

```
using RCall
```

```

@rput x y #send x and y to R

resultsR = reval("summary(mod <- lm(y ~ x-1))") #run R code
println(resultsR) #and print output

```



```
RObject{VecSxp}
```

```
Call:
```

```
lm(formula = y ~ x - 1)
```

```
Residuals:
```

	Min	1Q	Median	3Q	Max
	-17.981	-3.131	-0.359	2.281	52.361

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t )
x1	-0.50416	0.30483	-1.654	0.099 .
x2	1.34105	0.06573	20.401	<2e-16 ***

```

```

```
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 5.954 on 386 degrees of freedom
```

```
Multiple R-squared: 0.5194, Adjusted R-squared: 0.5169
```

```
F-statistic: 208.6 on 2 and 386 DF, p-value: < 2.2e-16
```

```
println(names(resultsR)) #print all keys (field names)
```

```
[:call, :terms, :residuals, :coefficients, :aliases, :sigma, :df, Symbol("r.squared"), Symbol("adj.r.squared")]
```

```
b_R = rcopy(resultsR[:coefficients]) #the numerical results are now a Julia array
```

```
printblue("Comparing the estimates in Julia and R")
```

```
printmat([b b_R[:,1]];colNames=["Julia","R"])
```

```
Comparing the estimates in Julia and R
```

Julia	R
-0.504	-0.504
1.341	1.341

```
R"model <- lm(y ~ x-1)" #an alternative approach
```

```
resultsR2 = rcall(:summary, R"model")
```

```
b_R2 = rcopy(resultsR2[:coefficients])
```

```
printmat([b b_R[:,1] b_R2[:,1]];colNames=["Julia","R","R ver, 2"])
```

Julia	R	R ver, 2
-0.504	-0.504	-0.504
1.341	1.341	1.341

We can also run blocks of R code `R"""` code `"""` like in the cell below. Do `@rput x y` or use `$x` and `$y` in the code below.

```
#solve x'x*b = x'y
R"""
xx <- t(x)%*%x
xy <- t(x)%*%y
b_R <- solve(xx,xy)
"""
```

```
RObject{RealSxp}
 [,1]
[1,] -0.5041626
[2,] 1.3410486
```

# PythonPlot

This file demonstrates how to create plots in Julia by using the [PythonPlot.jl](#) package.

PythonPlot relies on the [matplotlib library](#), which is part of Python. If you have Python installed, then it will be used as is (provided you tell Julia where to find Python). Otherwise, PythonPlot will help installing a Python version. See PythonPlot's homepage for instructions.

PythonPlot seems to behave better (on Windows, at least), if you run `ENV["MPLBACKEND"]="TkAgg"` before running using PythonPlot. One possibility is to put that command in the `startup.jl` file.

A collections of examples are available at [Plot Examples](#).

## Load Packages and Extra Functions

```
using Dates

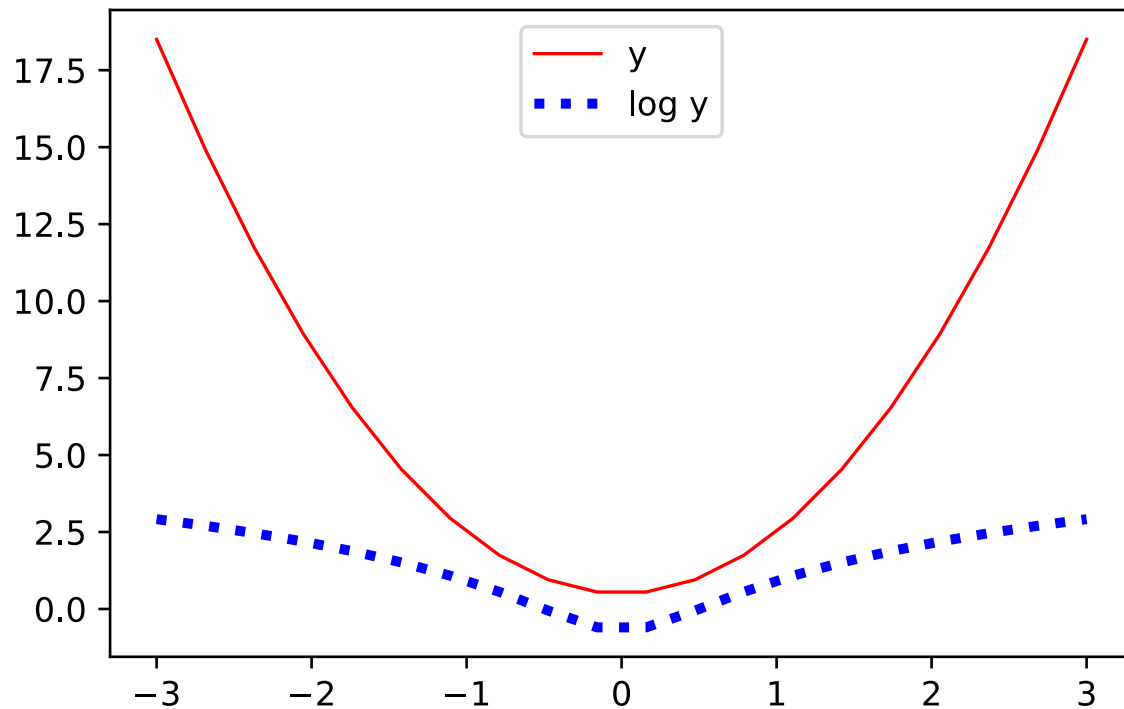
using PythonPlot
PythonPlot.svg(true); #prettier figures if `true`, but Github may not render it
```

## A First Plot

The next figure shows several curves, with legends.

```
x = range(-3,3,length=20)
y = 2*x.^2 .+ 0.5

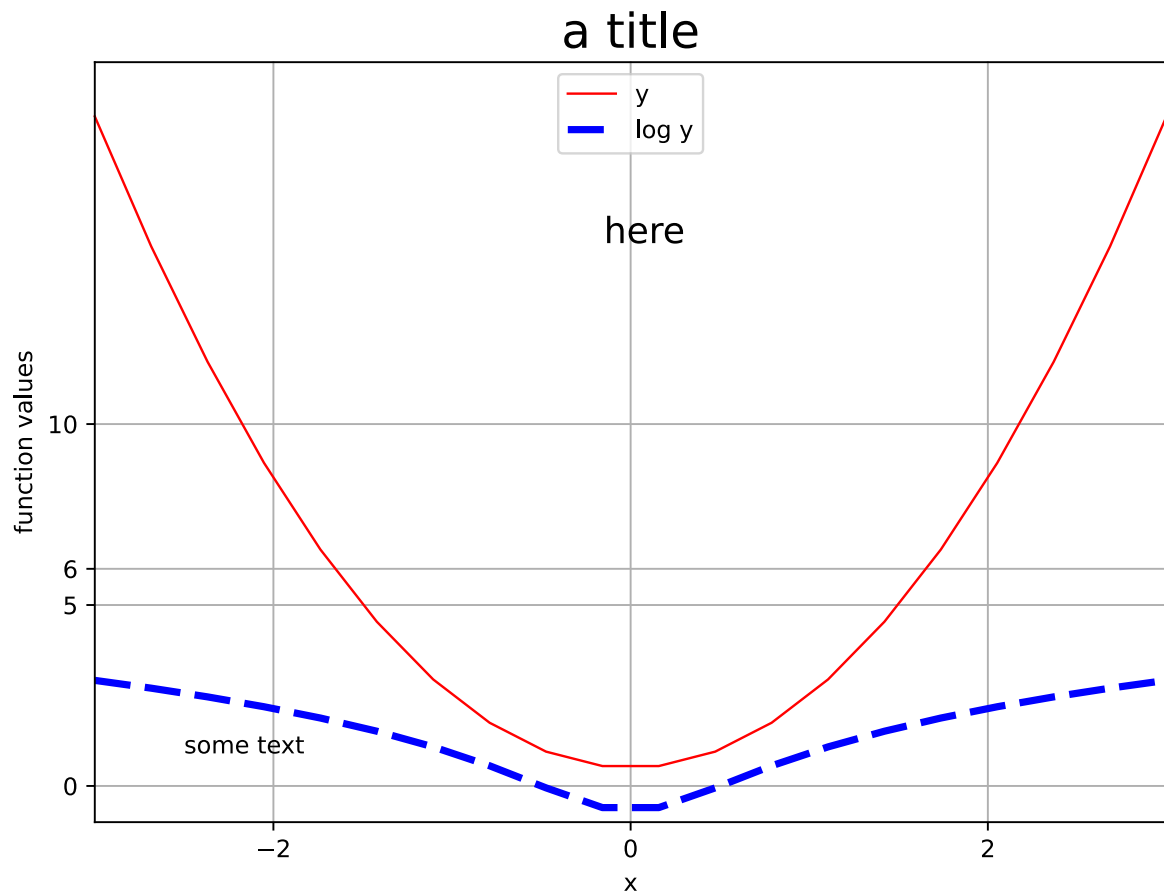
figure(figsize=(5.5,3.5)) #width and height, in inches
plot(x,y,linestyle="-",color="r",linewidth=1.0,label="y")
plot(x,log.(y),linestyle=":",color="b",linewidth=3.0,label="log y")
legend(loc="upper center")
#display(gcf()) #uncomment if the plot is not shown (eg. in REPL or VS Code)
```



Python: <matplotlib.legend.Legend object at 0x000002E26F29E120>

#now with title, labels and more - and larger

```
figure(figsize=(8,5.71))
plot(x,y,linestyle="-",color="r",linewidth=1.0,label="y")
plot(x,log.(y),linestyle=(0,(5,2)),color="b",linewidth=3.0,label="log y")
xticks([-2;0;2])
yticks([0;5;6;10])
grid(true)
title("a title",fontsize=20)
xlim(-3,3) # set limits of the x-axis
ylim(-1,20) # set limits of the y-axis
xlabel("x")
ylabel("function values")
text(-2.5,0.9,"some text")
text(-0.15,15,"here",fontsize=15)
legend(loc="upper center")
savefig("AFirstPlot.pdf") #save pdf file of the plot
#display(gcf())
```



Python: None

## Subplots

are easily constructed using the `subplot()` command. For a the first subplot in a 2x2 grid, use `subplot(2,2,1)`.

The `fig.set_size_inches(16,10.7,forward=false)` before the `savefig()` command (see below) makes sure the size of the saved graphics is not cut by my (smallish) screen size.

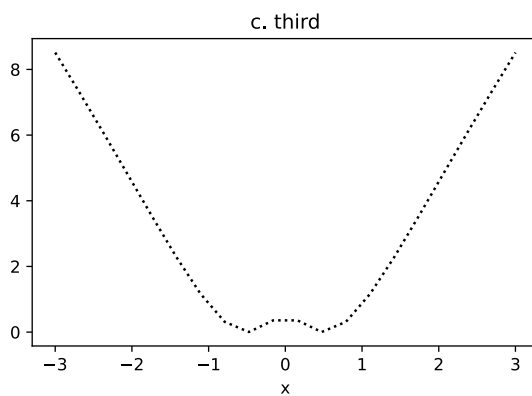
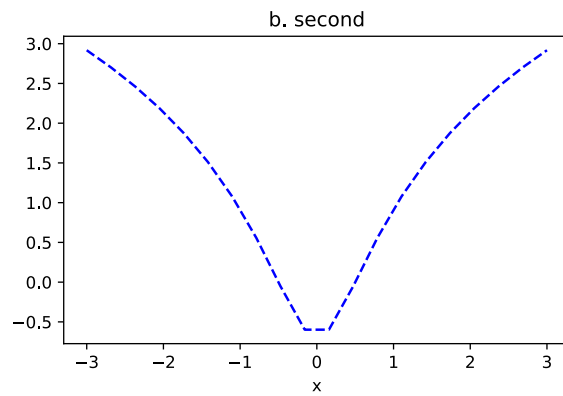
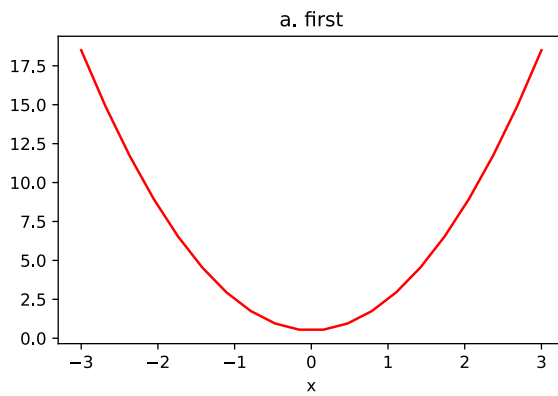
```
fig = figure(figsize=(7,5)) #subplots
subplot(2,2,1)
plot(x,y,"r-")
title("a. first")
xlabel("x")
```

```

subplot(2,2,2)
plot(x,log.(y),"b--")
title("b. second")
xlabel("x")
subplot(2,2,3)
plot(x,log.(y).^2,"k:")
title("c. third")
xlabel("x")
subplots_adjust(hspace = 0.4) #to give more vertical space between "row" 1 and 2

fig.set_size_inches(12,8,forward=false) #to save really large figure
savefig("ASecondPlot.pdf")
#display(gcf())

```

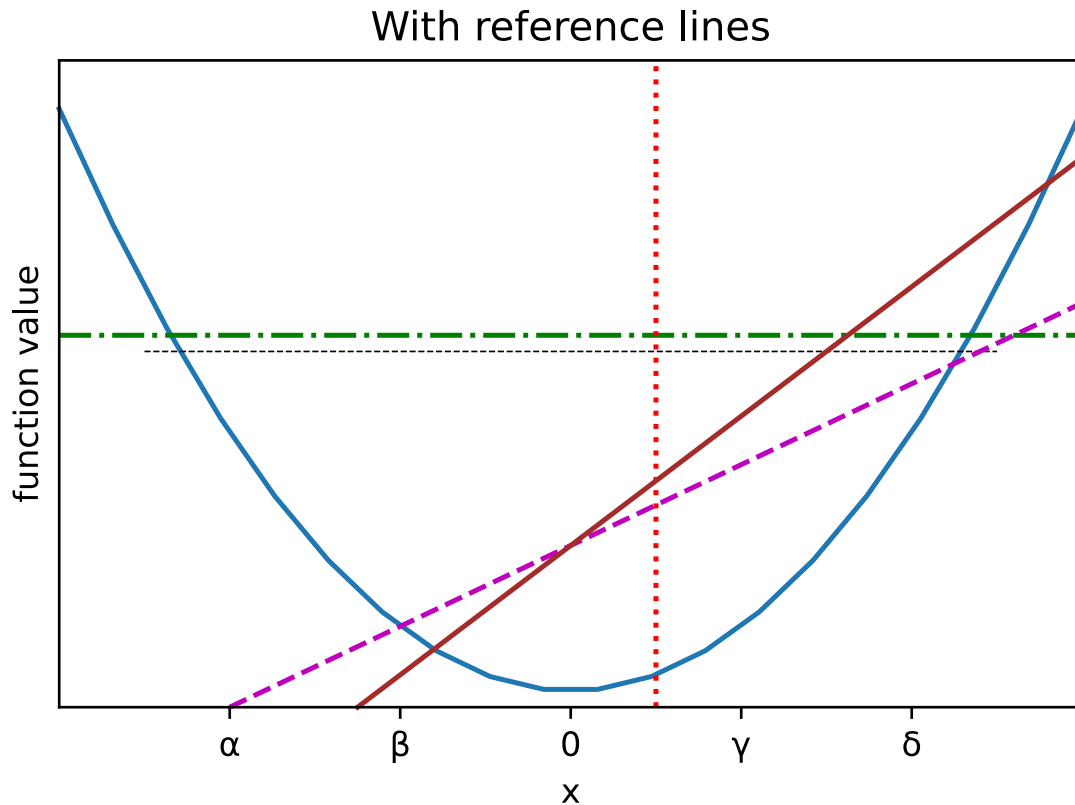


Python: None

## Adding Lines

as well as changing the x-tick marks and suppressing the y-tick marks.

```
figure(figsize=(5.5,3.5))
plot(x,y)
xlim(-3,3)
ylim(0,20)
title("With reference lines")
xlabel("x")
ylabel("function value")
xticks(-2:2,["α","β","0","γ","δ"])
yticks(Float64[]) #no tick marks on y axis, notice Float64[]
hlines(11,-2.5,2.5,linestyle="--",linewidth=0.5,color="black") #stretches over x=[-2.5;2.5]
axhline(11.5,linestyle="-.",color="g") #stretches over all x
vlines(0.5,0,20,linestyle=":",color="r")
axline((0,5),(2,10),linestyle="--",color="m")
axline((0,5),slope=4,linestyle="-",color="brown")
#display(gcf())
```



Python: <matplotlib.lines.AxLine object at 0x000002E27D8CF610>

## LaTeX in the Figure

PythonPlot calls on LaTeXStrings, so you can use, for instance, `xlabel(L"$\alpha$")` to get the  $x$  label in LaTeX. You could also change the general font to “cm” (see below) to make the various text elements look more similar.

To insert the value of the variable  $z$  into the LaTeX string (“string interpolation”), use `%%z` instead of the standard `$z`.

```
PythonPlot.matplotlib.rcParams["mathtext.fontset"] = "cm" #setting the math font
PythonPlot.matplotlib.rcParams["font.family"] = "serif" #setting the text font family

z = 4 #a value to be used inside the string

figure(figsize=(5.5,3.5))
```

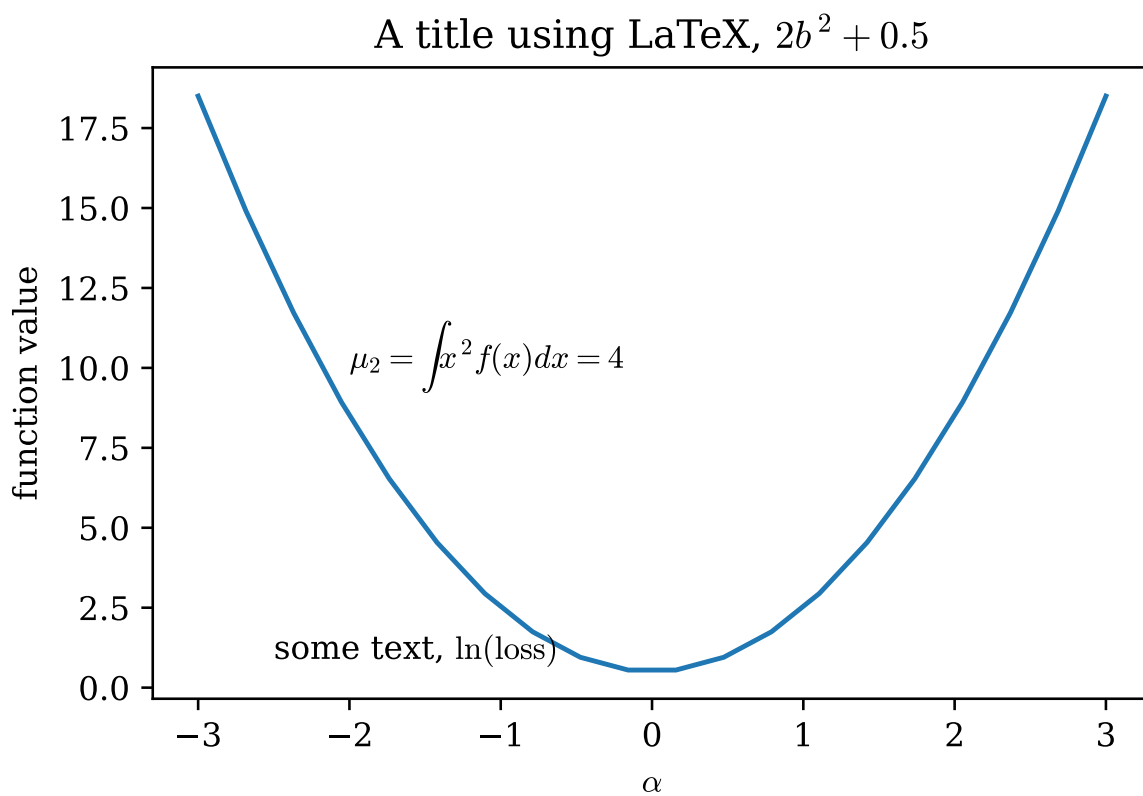


```

plot(x,y)
title(L"A title using LaTeX, $2b^2 + 0.5$ ")
xlabel(L" α ")
ylabel("function value")
text(-2.5,0.9,L"some text, $\ln(\mathrm{loss})$ ")
text(-2.0,10,L" $\mu_2 = \int x^2 f(x) dx = %z$ ") #notice: %z
#display(gcf())

PythonPlot.matplotlib.rcParams["mathtext.fontset"] = "dejavusans" #resetting the fonts
PythonPlot.matplotlib.rcParams["font.family"] = "sans-serif" #after the LaTeXing
PythonPlot.matplotlib.rcParams["font.size"] = 10;

```



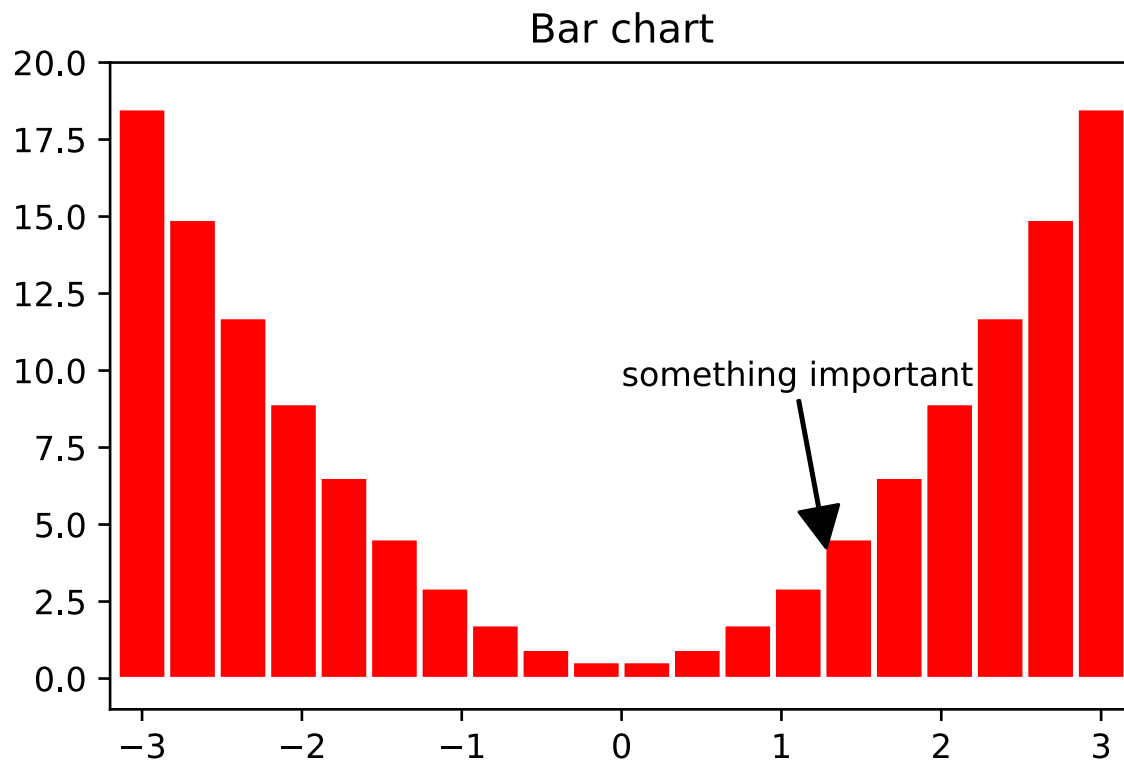
## Bars and Stairs

(with some annotations)

```

figure(figsize=(5.5,3.5))
bar(x,y,facecolor="red",edgecolor="white",align="center",width=0.3)
xlim(-3.2,3.2)
ylim(-1,20)
title("Bar chart")
txt = "something important"
annotate(txt,xy=[0.7;0.25],xycoords="axes fraction",xytext=[0.5;0.5],
 textcoords="axes fraction",arrowprops=Dict("facecolor"⇒"black","width"⇒0.5))
#display(gcf())

```

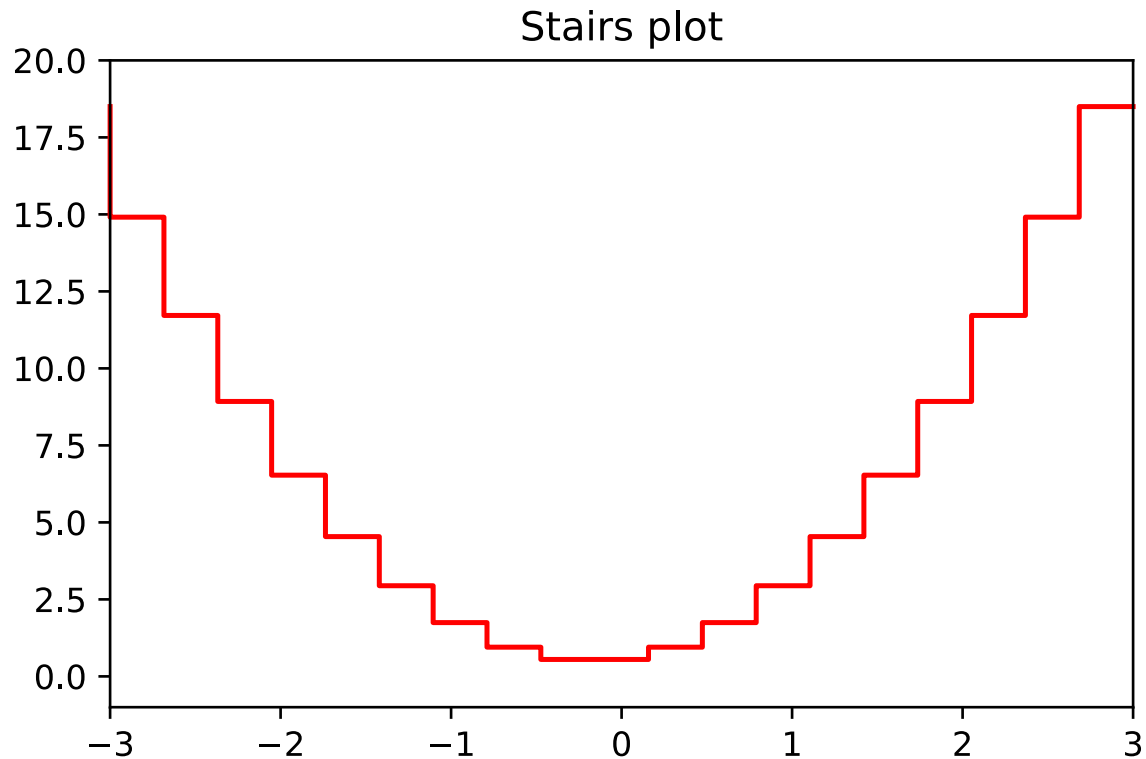


Python: Text(0.5, 0.5, 'something important')

```

figure(figsize=(5.5,3.5))
plotstep(x,y,linewidth=1.5,color="r") #not step()
xlim(-3,3)
ylim(-1,20)
title("Stairs plot")
#display(gcf())

```



Python: `Text(0.5, 1.0, 'Stairs plot')`

## Surface (3D) Plots

The next plot creates a surface plot. Notice that if  $x$  is a 20 vector,  $y$  is a 25 vector, and  $z$  is  $20 \times 25$ , then you need to use `surf(x,y,z')`.

In the subsequent plot, we rotate the view and also customize the axes and tick marks. This requires a slightly more involved approach (see below).

```
x = range(-3,3,length=20) #create some "data" to plot
y = range(1,7,length=25)

z = 2*x.^2 .+ (y' .-4).^2

#notice the arguments: x,y,z'
println(size(x),size(y),size(z'))
```

```

PythonPlot.matplotlib.rcParams["font.size"] = 16
fig = figure(figsize=(11,7))
ax = subplot(1,1,1,projection="3d") #ax = gca(projection="3d") in old matplotlib versions
 surf(x,y,z',rstride=1,cstride=1,cmap=ColorMap("cool"),alpha=0.8)
 xlim(-3,3) #change rstride and cstride to improve the look
 ylim(1,7)
 zlim(0,30)
 xlabel("x")
 ylabel("y")
 title("Surface plot")
#display(gcf())

println("\nthe aspect ratio: ",gca().get_box_aspect())
println("\nthe elevation and azimuth: ",gca().elev," ",gca().azim)

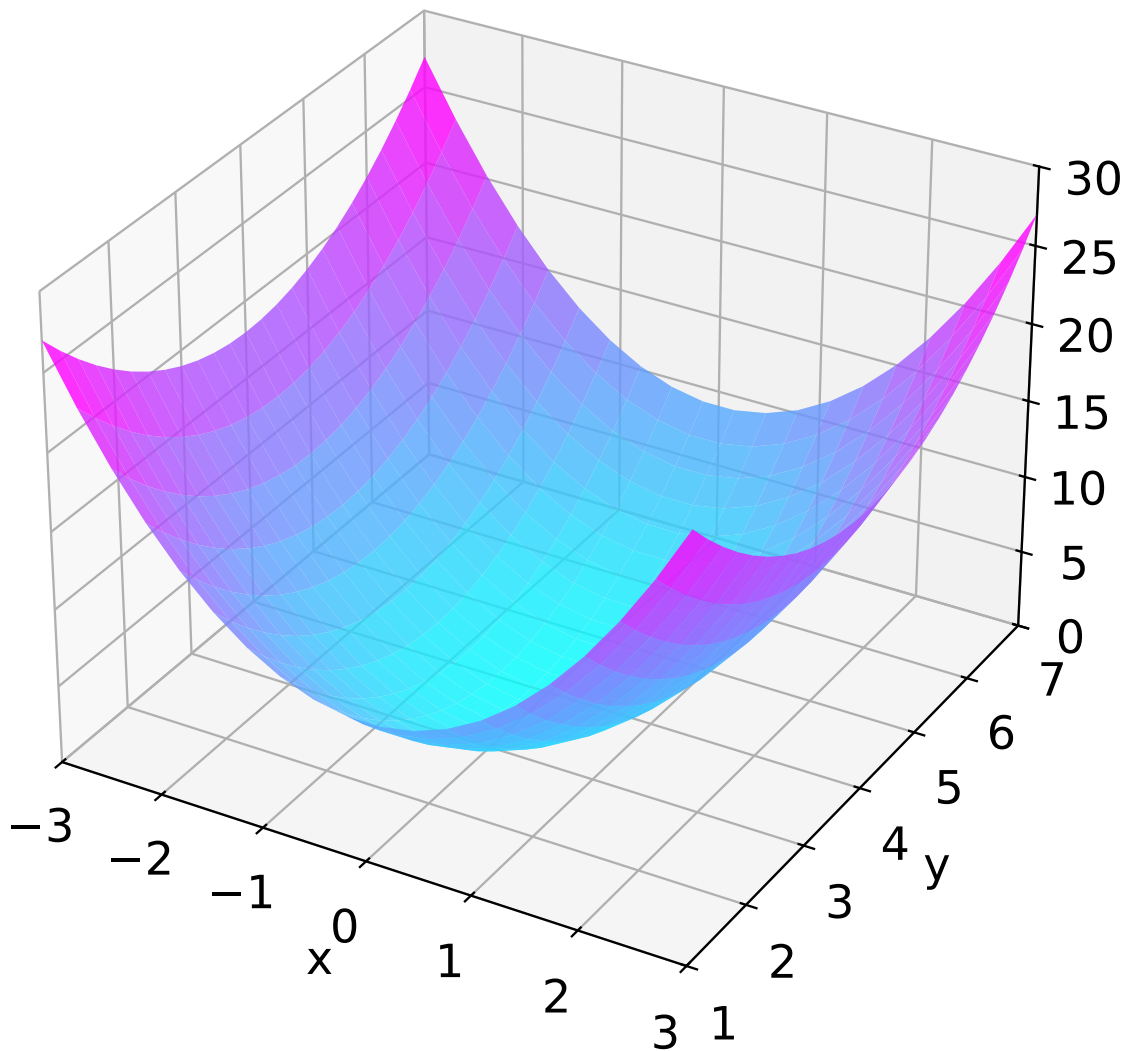
```

(20,)(25,)(25, 20)

the aspect ratio: [1.19047619 1.19047619 0.89285714]

the elevation and azimuth: 30 -60

# Surface plot



```
PythonPlot.matplotlib.rcParams["font.size"] = 16

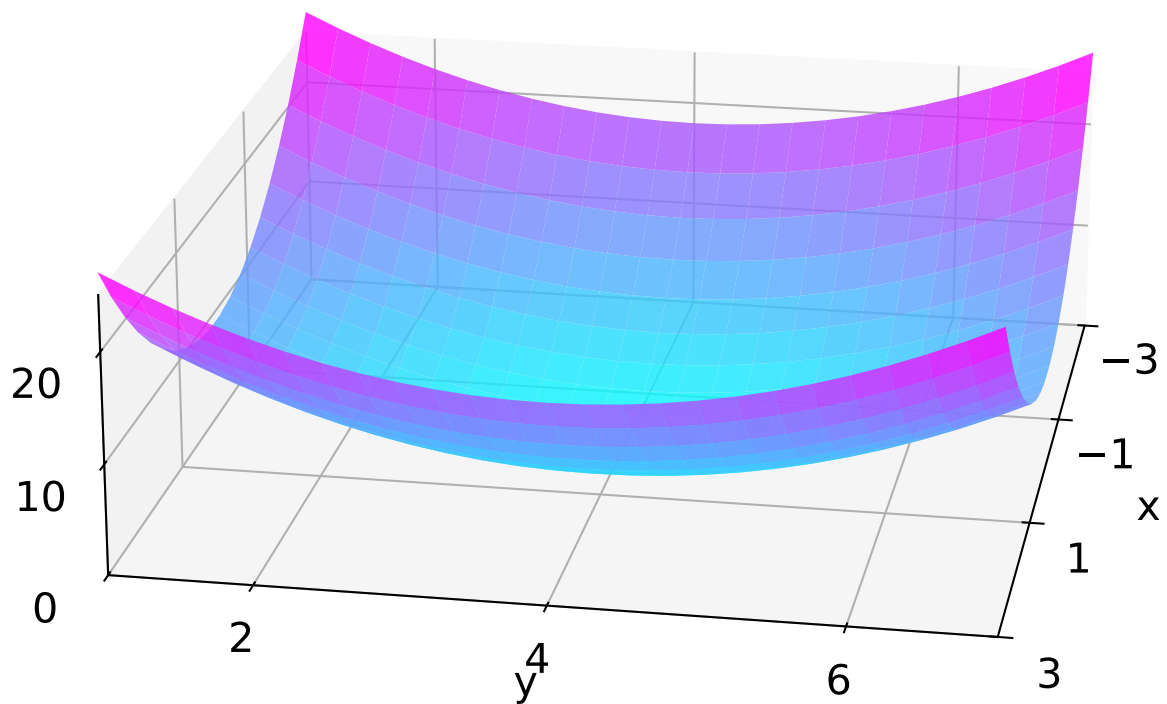
figure(figsize=(11,7))
ax = subplot(1,1,1,projection="3d") #ax = gca(projection="3d") in old matplotlib versions
surf(x,y,z',rstride=1,cstride=1,cmap=ColorMap("cool"),alpha=0.8)
xlim(-3,3)
ylim(1,7)
```

```

zlim(0,30)
ax.view_init(elev=20.0,azim=10) #notice: elevation and azimuth
gca().set_box_aspect((6.0,6.0,2.0),zoom=1.2)
xticks(-3:2:3)
yticks(2:2:6)
zlim([0,25])
zticks([0;10;20])
xlabel("x")
ylabel("y")
title("Surface plot, rotated and flatter")
#display(gcf())

```

## Surface plot, rotated and flatter

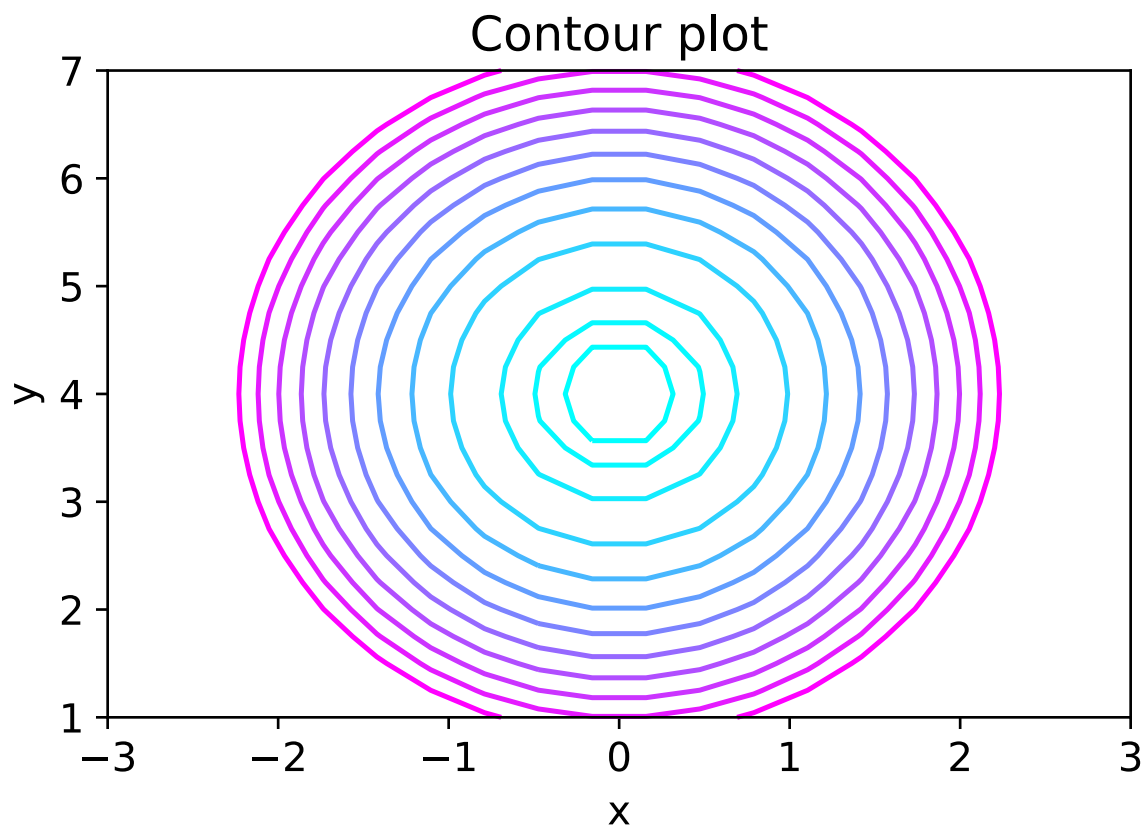


Python: `Text(0.5, 1.0, 'Surface plot, rotated and flatter')`

## Contour Plot

As an alternative to a surface plot, consider a contour plot. The syntax is `contour(x,y,z',...)`.

```
PythonPlot.matplotlib.rcParams["font.size"] = 12
fig1 = figure(figsize=(5.5,3.5))
lev = [0.25;0.5;1:1:10]
contour(x,y,z',lev,cmap=ColorMap("cool"))
xlim(-3,3)
ylim(1,7)
xlabel("x")
ylabel("y")
title("Contour plot")
#display(gcf())
```



Python: Text(0.5, 1.0, 'Contour plot')

## Scatter and Histogram

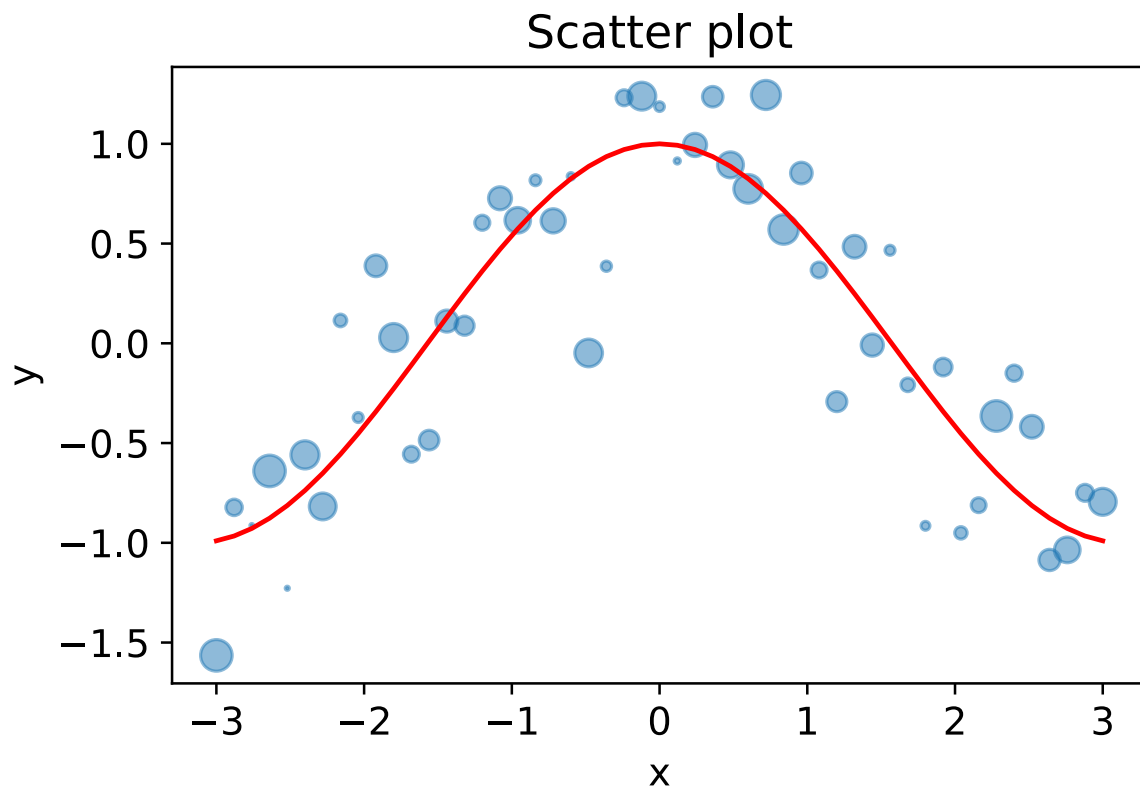
```
N = 51
x = range(-3,3,length=N)
y = cos.(x) + randn(N)/3
areas = rand(51)*100

figure(figsize=(5.5,3.5))
scatter(x,y,s=areas,alpha=0.5)
plot(x,cos.(x),"r-")
title("Scatter plot")
xlabel("x")
ylabel("y")
#display(gcf())
```

#SCATTER, HISTOGRAM

#size of the scatter points, could be a scalar

#s is the size of the circles

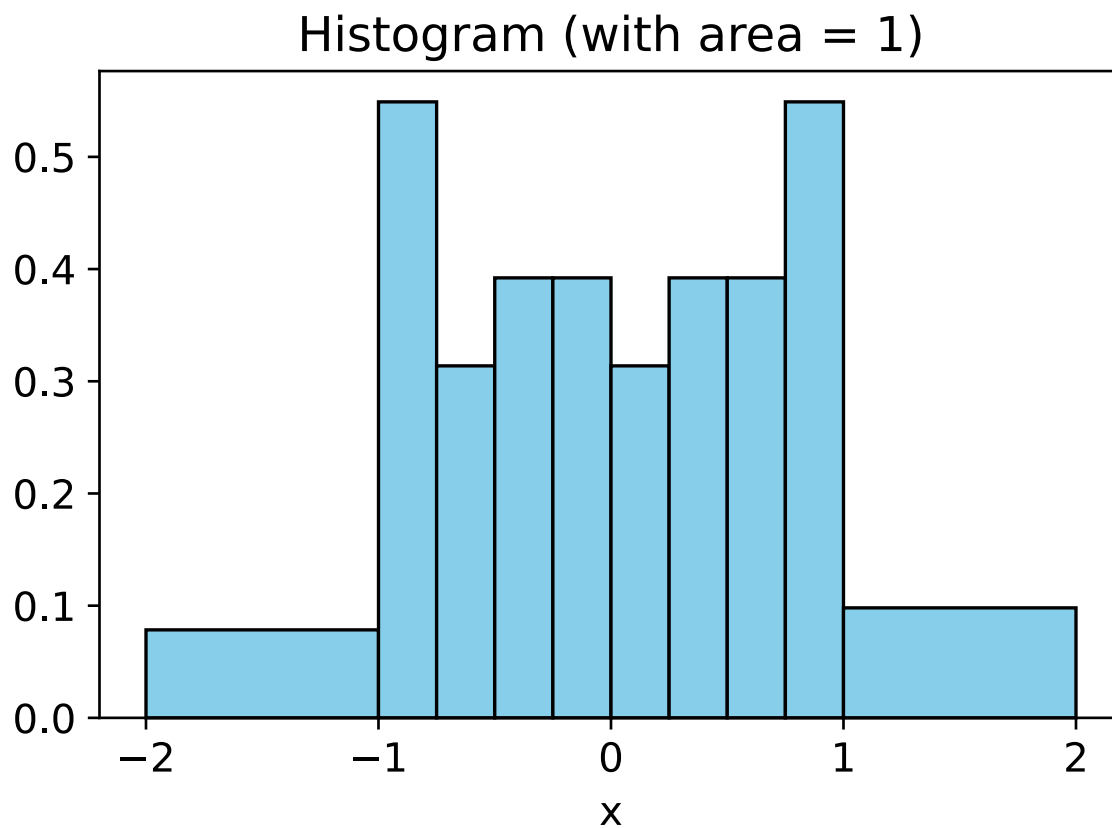




```
Python: Text(26.666666666666667, 0.5, 'y')
```

```
bins = vcat([-2,-1],[-0.75:0.25:1,2]) #bin edges

figure(figsize=(5.5,3.5))
hist(y,bins=bins,density=True,color="skyblue",edgecolor="k")
title("Histogram (with area = 1)")
xlabel("x")
#display(gcf())
```



```
Python: Text(0.5, 26.666666666666667, 'x')
```

## Time Series Plots

The (time) tick marks in time series plots often need tweaking. The cell below does that by first creating a vector of dates (xTicks) where I want the time marks, and then calls on `xticks(xTicks)`.

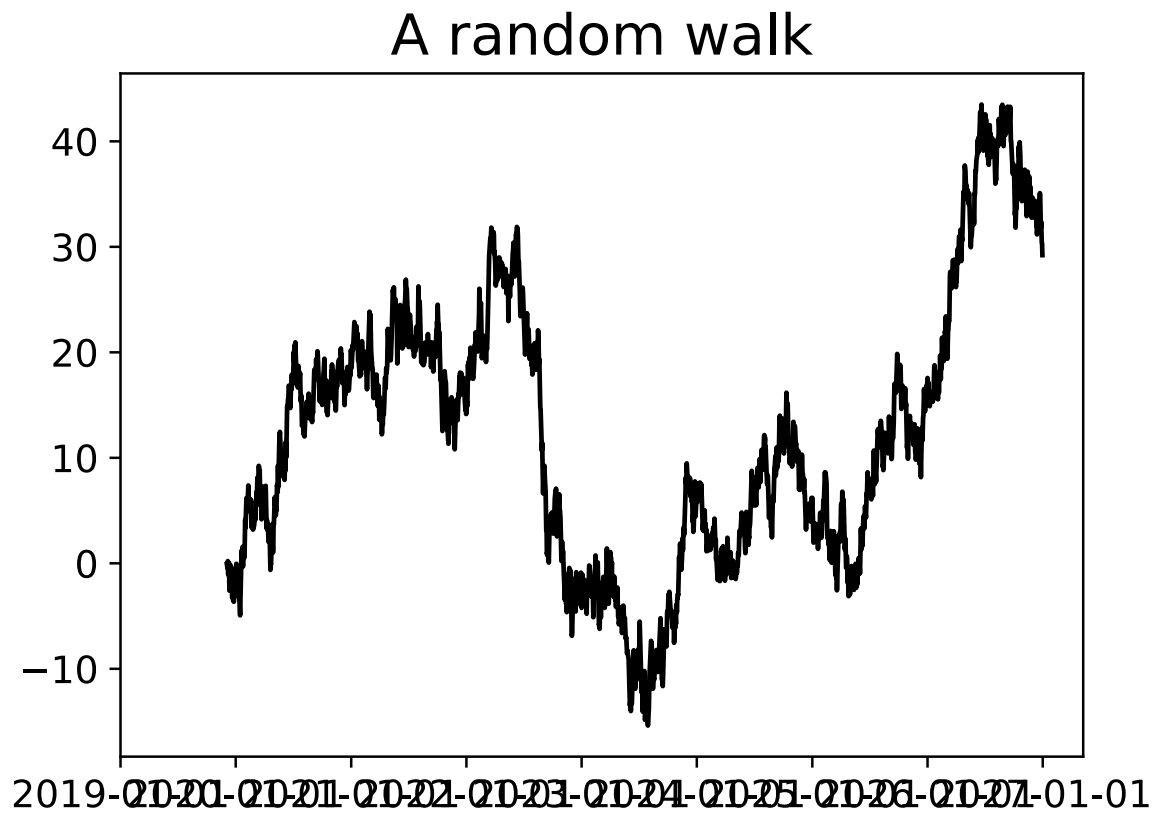
When the x variable is Julia dates, then the usual `plot()` function works well. (Otherwise, consider `plot_date()`).

The second figure fixed the formatting of the dates.

```
dN = Date(2019,12,4):Day(1):Date(2026,12,31) #just faking some dates
y = randn(length(dN)) #random numbers to plot

xTicks = Date(2019,1,1):Year(1):Date(2027,1,1) #tick marks on x axis

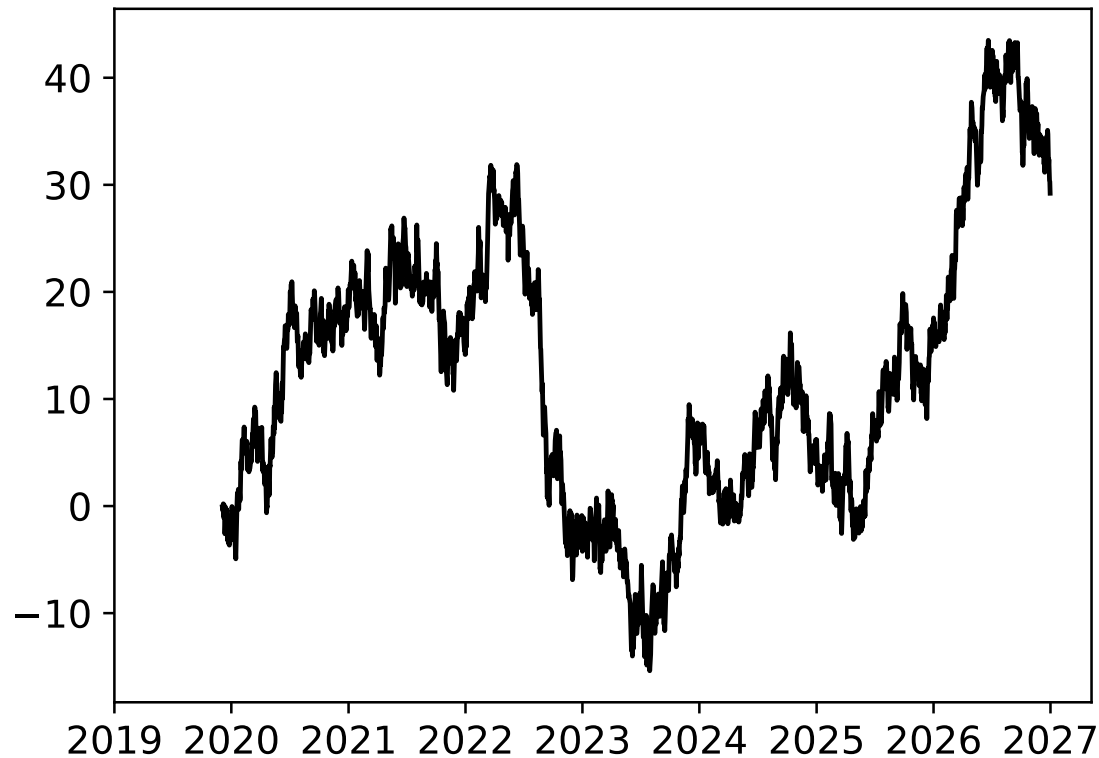
figure(figsize=(5.5,5.5/1.4)) #basic time series plot
plot(dN,cumsum(y),"k-")
xticks(xTicks)
title("A random walk",fontsize=18)
#display(gcf())
```



Python: `Text(0.5, 1.0, 'A random walk')`

```
figure(figsize=(5.5,5.5/1.4))
plot(dN,cumsum(y),"k-")
xticks(xTicks)
gca().xaxis.set_major_formatter(matplotlib.dates.DateFormatter("%Y"))
title("A random walk, with better tick marks",fontsize=18)
#display(gcf())
```

## A random walk, with better tick marks



Python: Text(0.5, 1.0, 'A random walk, with better tick marks')

### Adjusting the Resolution (extra)

In case you use PythonPlot in a script (not a notebook), then you may experience that your figures do not work well with the resolution of your screen (the figures are too small/big and the elements look garbled). It sometimes help to change the resolution of your figures by adding the following to your script

```
PythonPlot.matplotlib.rcParams["figure.dpi"] = 50 #experiment with different numbers
```

# DataFrames

This notebook illustrates the *basics* of [DataFrames.jl](#). There are many packages that work with such data frames, for instance, [TidierData.jl](#) which sticks close to R syntax. To do R style regressions, combine with [StatsModels.jl](#) and [GLM.jl](#).

As an alternative to `DataFrames.jl`, you may want to consider [InMemoryDatasets.jl](#).

## Load Packages

```
using Printf, Dates, Statistics, DataFrames, CSV
include("src/printmat.jl");
```

## Loading Some Data with CSV.jl

The [CSV.jl](#) package is very powerful - and also quick for large files.

(You could also use `DelimitedFiles.jl`, but it is not as well integrated with the `DataFrames.jl` as `CSV.jl`.)

```
println("the raw contents of Data/dfData1.csv:")
println(read("Data/dfData1.csv",String))
```

the raw contents of Data/dfData1.csv:

Date,	id,	B,	C,	D,	E
01/05/2019,	1,	9.998,	0.000,	The,	true
01/05/2019,	2,	9.994,	0.037,	highway,	true
01/05/2019,	3,	10.044,	0.092,	is,	true
02/05/2019,	1,	10.061,	0.098,	for,	true
02/05/2019,	2,	10.076,	0.078,	gamblers,	true
02/05/2019,	3,	10.064,	0.061,	Better,	false

To use CSV.jl to create a dataframe on the fly, do as in the cell below. Use normalizenames to get names that can be used in Julia as variables names and specify the dateformat used in the csv file (to convert to proper Julia dates).

```
df1 = CSV.read("Data/dfData1.csv",DataFrame,normalizenames=true,dateformat="dd/mm/yyyy")
show(df1;allrows=true,summary=false) #to explicitly print the df, can be configured
```

Row	Date	id	B	C	D	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	1	9.998	0.0	The	true
2	2019-05-01	2	9.994	0.037	highway	true
3	2019-05-01	3	10.044	0.092	is	true
4	2019-05-02	1	10.061	0.098	for	true
5	2019-05-02	2	10.076	0.078	gamblers	true
6	2019-05-02	3	10.064	0.061	Better	false

## Picking out Rows and Columns

```
df1[:,B] #create a traditional Julia vector
#df1.B #works too
```

6-element Vector{Float64}:

```
9.998
9.994
10.044
10.061
10.076
10.064
```

```
df1b = df1[(df1.id.>=2) .& (df1.E==true), :] #create new df by picking out some rows, print
```

	Date	id	B	C	D	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	2	9.994	0.037	highway	1
2	2019-05-01	3	10.044	0.092	is	1
3	2019-05-02	2	10.076	0.078	gamblers	1

```
df1b = df1[2:4,[:Date,:B]] #creates a new df by picking out some rows and cols
show(stdout,MIME("text/csv"),df1b) #explicit printing, here as csv
show(stdout,MIME("text/html"),df1b;summary=false,eltypes=false) #and as html
```

```
"Date","B"
"2019-05-01",9.994
"2019-05-01",10.044
"2019-05-02",10.061
```

Row	Date	B
1	2019-05-01	9.994
2	2019-05-01	10.044
3	2019-05-02	10.061

## Changing a DataFrame

```
df1[2:3,:C] = [100,101] #changing some values in :C
show(df1)
```

6×6 DataFrame

Row	Date	id	B	C	D	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	1	9.998	0.0	The	true
2	2019-05-01	2	9.994	100.0	highway	true
3	2019-05-01	3	10.044	101.0	is	true
4	2019-05-02	1	10.061	0.098	for	true
5	2019-05-02	2	10.076	0.078	gamblers	true
6	2019-05-02	3	10.064	0.061	Better	false

```
rename!(df1,:D ⇒ :BabyBlue) #renaming a column
```

	Date	id	B	C	BabyBlue	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	1	9.998	0.0	The	1
2	2019-05-01	2	9.994	100.0	highway	1
3	2019-05-01	3	10.044	101.0	is	1
4	2019-05-02	1	10.061	0.098	for	1
5	2019-05-02	2	10.076	0.078	gamblers	1
6	2019-05-02	3	10.064	0.061	Better	0

```
df1c = select(df1,[:BabyBlue,:C]) #new df
show(df1c)

println("\n")
df1b = select(df1,Not[:BabyBlue,:C])) #new df, select and Not: all cols except those
show(df1b)
```

6x2 DataFrame

Row	BabyBlue	C
	String15	Float64
1	The	0.0
2	highway	100.0
3	is	101.0
4	for	0.098
5	gamblers	0.078
6	Better	0.061

6x4 DataFrame

Row	Date	id	B	E
	Date	Int64	Float64	Bool
1	2019-05-01	1	9.998	true
2	2019-05-01	2	9.994	true
3	2019-05-01	3	10.044	true
4	2019-05-02	1	10.061	true
5	2019-05-02	2	10.076	true
6	2019-05-02	3	10.064	false



## Converting to and from a (traditional) Julia Matrix

```
arraydata = Matrix(df1[:,[:id,:B,:E]])
```

```
6×3 Matrix{Float64}:
```

```
1.0 9.998 1.0
2.0 9.994 1.0
3.0 10.044 1.0
1.0 10.061 1.0
2.0 10.076 1.0
3.0 10.064 0.0
```

```
df_from_matrix = DataFrame(arraydata,:auto)
```

	x1	x2	x3
	Float64	Float64	Float64
1	1.0	9.998	1.0
2	2.0	9.994	1.0
3	3.0	10.044	1.0
4	1.0	10.061	1.0
5	2.0	10.076	1.0
6	3.0	10.064	0.0

## Reshuffling a DataFrame (Groupby)

We now create a group for each id. These groups can be referred to as `dataG1[key]`.

We list the keys below. For instance `(id=2,)` is one of the keys. This is a tuple with a single element (similar to `(x,)`) where the element is `id=2`.

```
dataG1 = groupby(df1,:id) #grouped by id
```

GroupedDataFrame with 3 groups based on key: id

First Group (2 rows): id = 1

	Date	id	B	C	BabyBlue	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	1	9.998	0.0	The	1
2	2019-05-02	1	10.061	0.098	for	1
...						

Last Group (2 rows): id = 3

	Date	id	B	C	BabyBlue	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	3	10.044	101.0	is	1
2	2019-05-02	3	10.064	0.061	Better	0

```
dataG2 = vcat(dataG1...) #put together again, but now id=1 first, then id=2
```

	Date	id	B	C	BabyBlue	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	1	9.998	0.0	The	1
2	2019-05-02	1	10.061	0.098	for	1
3	2019-05-01	2	9.994	100.0	highway	1
4	2019-05-02	2	10.076	0.078	gamblers	1
5	2019-05-01	3	10.044	101.0	is	1
6	2019-05-02	3	10.064	0.061	Better	0

```
println("The keys of the grouped DataFrame are: ")
printmat(DataFrames.GroupKeys(dataG1))
```

The keys of the grouped DataFrame are:

GroupKey: (id = 1,)

GroupKey: (id = 2,)

GroupKey: (id = 3,)

```
dataG1[(id=2,)] #the group for (id=2,)
```

	Date	id	B	C	BabyBlue	E
	Date	Int64	Float64	Float64	String15	Bool
1	2019-05-01	2	9.994	100.0	highway	1
2	2019-05-02	2	10.076	0.078	gamblers	1

## Merging DataFrames

It is possible to stack DataFrames, if they are comformable.

More generally, we can use one of the many join functions, for instance, `inner join()`.

```
df3 = [df1 DataFrame(ff=1:6)] #stacking 2 dataframes horizontally
```

	Date	id	B	C	BabyBlue	E	ff
	Date	Int64	Float64	Float64	String15	Bool	Int64
1	2019-05-01	1	9.998	0.0	The	1	1
2	2019-05-01	2	9.994	100.0	highway	1	2
3	2019-05-01	3	10.044	101.0	is	1	3
4	2019-05-02	1	10.061	0.098	for	1	4
5	2019-05-02	2	10.076	0.078	gamblers	1	5
6	2019-05-02	3	10.064	0.061	Better	0	6

```
#loading another DataFrame, it has another order of observations
df2 = CSV.read("Data/dfData2.csv",DataFrame,normalizenames=true,dateformat="dd/mm/yyyy")
```

	Date	id	G
	Date	Int64	Int64
1	2019-05-01	1	11
2	2019-05-02	1	12
3	2019-05-01	2	21
4	2019-05-02	2	22
5	2019-05-01	3	31
6	2019-05-02	3	32

```
df3 = innerjoin(df1,df2,on=[Date,id]) #joining 2 DataFrames, match both :Date and :id
#innerjoin() for intersection of data points
```

	Date	id	B	C	BabyBlue	E	G
	Date	Int64	Float64	Float64	String15	Bool	Int64
1	2019-05-01	1	9.998	0.0	The	1	11
2	2019-05-02	1	10.061	0.098	for	1	12
3	2019-05-01	2	9.994	100.0	highway	1	21
4	2019-05-02	2	10.076	0.078	gamblers	1	22
5	2019-05-01	3	10.044	101.0	is	1	31
6	2019-05-02	3	10.064	0.061	Better	0	32

```
df3[:,c] .= 1 #add a constant to the DataFrame
show(df3;summary=false)
```

Row	Date	id	B	C	BabyBlue	E	G	c
	Date	Int64	Float64	Float64	String15	Bool	Int64	Int64
1	2019-05-01	1	9.998	0.0	The	true	11	1
2	2019-05-02	1	10.061	0.098	for	true	12	1

3	2019-05-01	2	9.994	100.0	highway	true	21	1
4	2019-05-02	2	10.076	0.078	gamblers	true	22	1
5	2019-05-01	3	10.044	101.0	is	true	31	1
6	2019-05-02	3	10.064	0.061	Better	false	32	1

## Using the Data

```
median(df1[:, :B])
```

10.0525

```
combine(groupby(df1, :id), :B.⇒[mean, std]) #mean and std of B for each id
```

	id	B_mean	B_std
	Int64	Float64	Float64
1	1	10.0295	0.0445477
2	2	10.035	0.0579828
3	3	10.054	0.0141421

# Tricky Stuff

This file highlights some tricky aspects of Julia (from the perspective of an old Matlab user).

## Load Packages and Extra Functions

```
using Printf, LinearAlgebra

include("src/printmat.jl");
```

## An Nx1 Matrix Is Not a Vector

and it sometimes matters.

Julia has both vectors and  $N \times 1$  matrices (the latter being a special case of  $N \times M$  matrices). They can often be used interchangeably, but not always.

```
v = ones{Int,2} #a vector with two elements
v2 = ones{Int,2,1} #a 2x1 matrix (Array)

println("v and v2 'look' similar:")
printmat(v)
printmat(v2)
println("but they have different sizes: ",size(v)," ",size(v2))
```

v and v2 'look' similar:

```
1
1

1
```

1

but they have different sizes: (2,) (2, 1)

## **X[1,:] Gives a (Flat) Vector**

If  $X$  is a  $T \times n$  matrix, then  $X[1,:]$  gives a flat vector, *not* a  $1 \times n$  matrix (or row vector).

```
X = [11 12; 21 22]

x1 = X[1,:] #this gives a flat vector
println("size of X[1,:]: ", size(x1))
printmat(x1)

x1b = X[1:1,:] #however, this gives a 1x2 matrix
println("size of X[1:1,:]: ", size(x1b))
printmat(x1b)
```

size of X[1,:]: (2,)

11  
12

size of X[1:1,:]: (1, 2)

11      12

## **Array .+ Scalar Requires a Dot (.)**

```
y = [1;2] .+ 1 #do not forget the dot (.)
printmat(y)
```

2  
3

## **Creating Variables in a Loop**

```

for i = 1:5
 global Tor #without this, Tor is not seen outside the loop
 Tor = cos(i)
end
println("Tor: $Tor")

Oden = Inf #define 'Oden' before the loop
for i = 1:5
 #global Oden #only needed in script
 Oden = sin(i) #will overwrite an existing value
end
println("Oden: $Oden")

```

```

Tor: 0.28366218546322625
Oden: -0.9589242746631385

```

## An Array of Arrays

can be initialized by comprehension (see below).

Do **not** use `fill(array,n)` for this, since all `n` arrays will refer to the same underlying array (changing one changes all).

```

x = [zeros(2,2) for i=1:2] #a vector of two matrices
x[1][1,1] = -99 #change an element of x[1]

println("x[1]")
printmat(x[1])

println("x[2]")
printmat(x[2])

```

```

x[1]
-99.000 0.000
 0.000 0.000

```

```

x[2]
 0.000 0.000
 0.000 0.000

```

## Arrays Are Different...

Vectors and matrices (arrays) can take lots of memory space, so **Julia is designed to avoid unnecessary copies of arrays**. This means that if A and B are names of the same underlying array, then changing an element of B changes also A.

See the chapter on Arrays for more details.